

```
=== FILE: .\db_setup.py ===
```

```
"""
```

```
db_setup.py – Initialize ECommerce Database Tables
```

```
Run once: python db_setup.py
```

```
"""
```

```
from core.db import Database
```

```
db = Database()
```

```
schema = """
```

```
-- Products Table
```

```
CREATE TABLE IF NOT EXISTS products (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL,  
    description TEXT,  
    price REAL NOT NULL,  
    image_url TEXT,  
    category TEXT DEFAULT 'general',  
    stock INTEGER DEFAULT 100,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- Cart Table
```

```
CREATE TABLE IF NOT EXISTS cart (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    session_id TEXT NOT NULL,  
    product_id INTEGER NOT NULL,  
    quantity INTEGER DEFAULT 1,  
    added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (product_id) REFERENCES products(id)  
);
```

```
-- Orders Table
```

```
CREATE TABLE IF NOT EXISTS orders (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    session_id TEXT,  
    name TEXT NOT NULL,  
    email TEXT NOT NULL,  
    address TEXT NOT NULL,  
    phone TEXT,  
    total REAL NOT NULL,  
    status TEXT DEFAULT 'pending',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
-- Order Items Table
```

```
CREATE TABLE IF NOT EXISTS order_items (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    order_id INTEGER NOT NULL,  
    product_id INTEGER NOT NULL,  
    quantity INTEGER NOT NULL,  
    unit_price REAL NOT NULL,  
    FOREIGN KEY (order_id) REFERENCES orders(id),  
    FOREIGN KEY (product_id) REFERENCES products(id)  
);
```

```
"""
```

```
db.executescript(schema)
```

```
print(" Tables created successfully.")
```

```
# Seed dummy products
```

```
existing = db.fetchall("SELECT COUNT(*) as cnt FROM products;")
```

```
if existing[0]['cnt'] == 0:
```

```
    products = [  
        ("Gravity Pro Headphones", "Premium wireless headphones with 40-hour battery and ANC technology.  
        Crystal clear sound with deep bass.", 149.99, "/static/img/headphones.svg", "Electronics"),
```

```

        ("NovaPulse Smartwatch", "Track your fitness, get notifications, and look great doing it. 7-day battery life.", 299.99, "/static/img/watch.svg", "Electronics"),

        ("AeroFloat Sneakers", "Lightweight performance sneakers with memory foam insole. Perfect for running or casual wear.", 89.99, "/static/img/sneakers.svg", "Footwear"),

        ("ZenCore Backpack", "Weather-resistant backpack with laptop compartment, USB charging port, and 30L capacity.", 69.99, "/static/img/backpack.svg", "Bags"),

        ("LuminaDesk LED Keyboard", "RGB mechanical keyboard with anti-ghosting, 100% rollover, and customizable macros.", 119.99, "/static/img/keyboard.svg", "Electronics"),

        ("CloudPillow Travel Set", "Memory foam travel pillow + eye mask + earplugs combo for the ultimate travel comfort.", 34.99, "/static/img/pillow.svg", "Travel"),

        ("SolarCharge Power Bank", "20000mAh solar power bank with dual USB-C and fast charging. Works in direct sunlight.", 59.99, "/static/img/powerbank.svg", "Electronics"),

        ("FrostBrew Coffee Maker", "Smart drip coffee maker with app control, built-in grinder, and thermal carafe.", 189.99, "/static/img/coffee.svg", "Kitchen"),

    ]

    db.executemany(

        "INSERT INTO products (name, description, price, image_url, category) VALUES (?, ?, ?, ?, ?)",

        products

    )

    print(f" Seeded {len(products)} products.")

else:

    print("  Products already seeded.")


print("\n Database ready. Run: python main.py")

```

```

=====

```

```

=== FILE: .\main.py ===

```

```

# AUTO-GENERATED BY NODE EDITOR COMPILER

```

```

import socketserver

```

```

from nodes.server_node import ServerNode

```

```

from nodes.http_requests_node import HTTPRequestsNode

```

```

from nodes.url_node import URLNode

```

```

from nodes.template_node import RenderNode

```

```

from nodes.logic_node import LogicNode

```

```
from nodes.route_node import RouterNode

from nodes.context_node import ContextNode

from plugins.logger import ActionLoggerNode

from plugins.security import AntiBotNode

from plugins.security import RateLimitNode

from plugins.security import CSRFNode

from nodes.response import Response


# Initialize Database if needed


# Nodes Instantiation

servernode_node_42f0f91d = ServerNode(host='127.0.0.1', port=8000)

httprequestsnodes_node_463dc491 = HTTPRequestsNode()

actionlogger_node_630a0f20 = ActionLoggerNode()

ratelimit_node_4899a088 = RateLimitNode()

antibot_node_789fa014 = AntiBotNode()

csrf_node_5a10accc = CSRFNode()

url_node_ea5e3b4a = URLNode('/')

url_node_aa37ab65 = URLNode('/shop')

url_node_f928646e = URLNode('/product/<id>')

url_node_b1bb4012 = URLNode('/cart')

url_node_baa46610 = URLNode('/cart/add')

url_node_50dd4297 = URLNode('/cart/add_from_shop')

url_node_172c8a35 = URLNode('/cart/remove')

url_node_bdd5e5ef = URLNode('/checkout')

url_node_37cf26cc = URLNode('/checkout/place')

url_node_a89ed2e0 = URLNode('/order/success')

url_node_66d287f6 = URLNode('/api/cart/add')

url_node_acff5eda = URLNode('/api/cart-count')

url_node_fb230ff9 = URLNode('/test-py')

url_node_297859fd = URLNode('/demo-frontend')

url_node_297861 = URLNode('/about')
```

```

from static.helpers import db, get_session_id, compute_cart_totals

def home_logic(req):

    category    = req.query_params.get('cat', '')

    session_id  = get_session_id(req)

    if category:

        products = db.fetchall(

            "SELECT * FROM products WHERE category=? ORDER BY id",

            (category,)

        )

    else:

        products = db.fetchall("SELECT * FROM products ORDER BY id")

    product_count = db.fetchall("SELECT COUNT(*) as cnt FROM products")[0]['cnt']

    cart_count_row = db.fetchall(

        "SELECT COALESCE(SUM(quantity),0) as cnt FROM cart WHERE session_id=?",

        (session_id,)

    )

    return {

        'products':        products,

        'product_count':   product_count,

        'current_category': category,

        'cart_count':      cart_count_row[0]['cnt'],

        'csrf_token':      req.context.get('csrf_token', ''),

    }

```

```

logicnode_node_a2378860 = LogicNode(home_logic)

```

```

from static.helpers import db, get_session_id, compute_cart_totals

def product_detail_logic(req):

    pid        = req.get_param('id') or req.url_params.get('id', 0)

    session_id = get_session_id(req)

    rows       = db.fetchall("SELECT * FROM products WHERE id=?", (int(pid),))

    if not rows:

        return {'product_name': 'Not Found', 'product_id': 0, 'product_price': '0',

```

```

        'product_description': '', 'product_image': '', 'product_category': '',
        'cart_count': 0, 'csrf_token': ''}

p = rows[0]

cart_count_row = db.fetchall(

    "SELECT COALESCE(SUM(quantity),0) as cnt FROM cart WHERE session_id=?",

    (session_id,)

)

return {

    'product_id':          p['id'],

    'product_name':        p['name'],

    'product_price':        f"{p['price']:.2f}",

    'product_description': p['description'],

    'product_image':        p['image_url'],

    'product_category':    p['category'],

    'cart_count':          cart_count_row[0]['cnt'],

    'csrf_token':          req.context.get('csrf_token', ''),

}

}

logicnode_node_ada28e0d = LogicNode(product_detail_logic)

from static.helpers import db, get_session_id, compute_cart_totals

def cart_logic(req):

    session_id = get_session_id(req)

    cart_items = db.fetchall("""

        SELECT c.id as cart_id, c.quantity, p.name, p.price, p.image_url, p.category

        FROM cart c JOIN products p ON c.product_id = p.id

        WHERE c.session_id = ?

        ORDER BY c.id

    """, (session_id,))

    totals = compute_cart_totals(cart_items)

    flash_success = req.query_params.get('added', '')

    flash_name = req.query_params.get('name', '')

    totals.update({

```

```

        'cart_items':    cart_items,
        'cart_count':    totals['item_count'],
        'csrf_token':    req.context.get('csrf_token', ''),
        'flash_success': f"'{flash_name}' added to cart!" if flash_success else '',
        'flash_error':    '',
    })

    return totals

```

```

logicnode_node_e2a12a36 = LogicNode(cart_logic)

```

```

from static.helpers import db, get_session_id, compute_cart_totals

```

```

def cart_add_logic(req):

    session_id = get_session_id(req)

    product_id = req.get_param('product_id')

    quantity    = int(req.get_param('quantity') or 1)

    if not product_id:

        return Response.redirect('/cart')

    product_id = int(product_id)

    # Check if product exists

    rows = db.fetchall("SELECT * FROM products WHERE id=?", (product_id,))

    if not rows:

        return Response.redirect('/cart')

    product = rows[0]

    # Check if already in cart (same session)

    existing = db.fetchall(

        "SELECT id, quantity FROM cart WHERE session_id=? AND product_id=?",

        (session_id, product_id)

    )

    if existing:

        new_qty = existing[0]['quantity'] + quantity

        db.execute("UPDATE cart SET quantity=? WHERE id=?", (new_qty, existing[0]['id']))

    else:

        db.execute(

```

```

        "INSERT INTO cart (session_id, product_id, quantity) VALUES (?, ?, ?)",
        (session_id, product_id, quantity)
    )

# Redirect back with flash message

name_enc = urllib.parse.quote(product['name'])

return Response.redirect(f'/cart?added=1&name={name_enc}')

logicnode_node_8e4cf37f = LogicNode(cart_add_logic)

from static.helpers import db, get_session_id, compute_cart_totals

def cart_add_from_shop_logic(req):
    session_id = get_session_id(req)
    product_id = req.get_param('product_id')
    quantity = int(req.get_param('quantity') or 1)

    if not product_id:
        return Response.redirect('/shop')

    product_id = int(product_id)

    # Check if product exists
    rows = db.fetchall("SELECT * FROM products WHERE id=?", (product_id,))

    if not rows:
        return Response.redirect('/shop')

    # Check if already in cart
    existing = db.fetchall(
        "SELECT id, quantity FROM cart WHERE session_id=? AND product_id=?",
        (session_id, product_id)
    )

    redirect_url = req.get_param('redirect_url') or '/shop'

```



```

if existing:

    new_qty = existing[0]['quantity'] + quantity

    db.execute("UPDATE cart SET quantity=? WHERE id=?", (new_qty, existing[0]['id']))

else:

    db.execute(

        "INSERT INTO cart (session_id, product_id, quantity) VALUES (?,?,?)",

        (session_id, product_id, quantity)

    )

# Redirect back locally

return Response.redirect(redirect_url)

```

```

logicnode_node_ea26aefb = LogicNode(cart_add_from_shop_logic)

```

```

from static.helpers import db, get_session_id, compute_cart_totals

def cart_remove_logic(req):

    cart_id = req.get_param('cart_id')

    if cart_id:

        db.execute("DELETE FROM cart WHERE id=?", (int(cart_id),))

    return Response.redirect('/cart')

```

```

logicnode_node_38f5c7ca = LogicNode(cart_remove_logic)

```

```

from static.helpers import db, get_session_id, compute_cart_totals

def checkout_logic(req):

    session_id = get_session_id(req)

    cart_items = db.fetchall("""

        SELECT c.id as cart_id, c.quantity, p.name, p.price, p.image_url, p.category

        FROM cart c JOIN products p ON c.product_id = p.id

        WHERE c.session_id = ?

        ORDER BY c.id

    """, (session_id,))

    if not cart_items:

```

```

        return Response.redirect('/cart')

totals = compute_cart_totals(cart_items)

totals.update({

    'cart_items': cart_items,

    'cart_count': totals['item_count'],

    'csrf_token': req.context.get('csrf_token', ''),

    'flash_error': req.query_params.get('error', ''),

})

return totals

```

```

logicnode_node_4215f0d1 = LogicNode(checkout_logic)

```

```

from static.helpers import db, get_session_id, compute_cart_totals

```

```

def checkout_place_logic(req):

```

```

    session_id = get_session_id(req)

    name      = req.get_param('name', '').strip()

    email     = req.get_param('email', '').strip()

    address   = req.get_param('address', '').strip()

    phone     = req.get_param('phone', '').strip()

```

```

    if not name or not email or not address:

```

```

        return Response.redirect('/checkout?error=Please+fill+all+required+fields')

```

```

# Get cart items

```

```

cart_items = db.fetchall("""

```

```

    SELECT c.id as cart_id, c.quantity, p.id as product_id, p.name, p.price

    FROM cart c JOIN products p ON c.product_id = p.id

    WHERE c.session_id = ?

```

```

""", (session_id,))

```

```

if not cart_items:

```

```

    return Response.redirect('/cart?error=Cart+is+empty')

```

```

totals = compute_cart_totals(cart_items)

total = totals['total_raw']


# Insert order

db.execute(

    "INSERT INTO orders (session_id, name, email, address, phone, total) VALUES (?, ?, ?, ?, ?, ?)",

    (session_id, name, email, address, phone, total)

)

# Get the order id

order_row = db.fetchall("SELECT id FROM orders WHERE session_id=? ORDER BY id DESC LIMIT 1",
(session_id,))

order_id = order_row[0]['id'] if order_row else '??'


# Insert order items

items_data = [(order_id, item['product_id'], item['quantity'], item['price']) for item in cart_items]

db.executemany(

    "INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES (?, ?, ?, ?)",

    items_data

)


# Clear the cart

db.execute("DELETE FROM cart WHERE session_id=?", (session_id,))


# Store order info in context for redirect

name_enc = urllib.parse.quote(name)

email_enc = urllib.parse.quote(email)

addr_enc = urllib.parse.quote(address)

total_enc = urllib.parse.quote(f"{total:.2f}")

return Response.redirect(

    f'/order/success?order_id={order_id}&name={name_enc}'

    f'&email={email_enc}&address={addr_enc}&total={total_enc}'

)


logicnode_node_699b71b9 = LogicNode(checkout_place_logic)

```

```

from static.helpers import db, get_session_id, compute_cart_totals

def order_success_logic(req):

    session_id = get_session_id(req)

    order_id    = req.query_params.get('order_id', '???')

    name        = req.query_params.get('name', 'Customer')

    email        = req.query_params.get('email', '')

    address      = req.query_params.get('address', '')

    total        = req.query_params.get('total', '0.00')

    return {

        'order_id':        order_id,

        'customer_name':   name,

        'customer_email':  email,

        'customer_address': address,

        'order_total':     total,

        'cart_count':      0,

        'csrf_token':      req.context.get('csrf_token', ''),

    }

```

```

logicnode_node_f193a041 = LogicNode(order_success_logic)

```

```

from static.helpers import db, get_session_id, compute_cart_totals

def api_cart_add_logic(req):

    session_id = get_session_id(req)

    product_id = req.get_param('product_id')

    quantity    = int(req.get_param('quantity') or 1)

    if not product_id:

        return json.dumps({'error': 'No product_id'})

    product_id = int(product_id)

    rows = db.fetchall("SELECT * FROM products WHERE id=?", (product_id,))

    if not rows:

        return json.dumps({'error': 'Product not found'})

    existing = db.fetchall(

```

```

        "SELECT id, quantity FROM cart WHERE session_id=? AND product_id=?",
        (session_id, product_id)
    )
    if existing:
        new_qty = existing[0]['quantity'] + quantity
        db.execute("UPDATE cart SET quantity=? WHERE id=?", (new_qty, existing[0]['id']))
    else:
        db.execute(
            "INSERT INTO cart (session_id, product_id, quantity) VALUES (?, ?, ?)",
            (session_id, product_id, quantity)
        )
    cart_count = db.fetchall(
        "SELECT COALESCE(SUM(quantity),0) as cnt FROM cart WHERE session_id=?",
        (session_id,)
    )[0]['cnt']
    return Response.json({'cart_count': cart_count, 'status': 'added'})

```

```

logicnode_node_32a2c2e6 = LogicNode(api_cart_add_logic)

```

```

from static.helpers import db, get_session_id, compute_cart_totals

```

```

def api_cart_count_logic(req):
    session_id = get_session_id(req)
    cnt = db.fetchall(
        "SELECT COALESCE(SUM(quantity),0) as cnt FROM cart WHERE session_id=?",
        (session_id,)
    )[0]['cnt']
    return Response.json({'count': cnt})

```

```

logicnode_node_9b18f0c1 = LogicNode(api_cart_count_logic)
rendernode_node_a5fac370 = RenderNode('test_pyscript.html')
rendernode_node_b29ad28e = RenderNode('demo_frontend.html')

```

```

def node_logic(request):

```

```

import time

from datetime import datetime

from nodes.response import StreamingResponse


def live_clock():

    # Ye loop lagatar chalta rahega jab tak user page open rakhega

    while True:

        # Current time nikal kar format karein (Hours:Minutes:Seconds)

        current_time = datetime.now().strftime("%H:%M:%S")

        # HTML span bhej kar time dikhayein

        yield f"<span style='font-family: monospace; font-size: 2rem; color: #10b981; font-weight: bold;'>{current_time}</span>"

        # Har naya time bhejne ke baad 1 second hold karein

        time.sleep(1)

    # 'result' key mein stream ko bhej dein taaki template {{ result | safe }} usko catch karle

    return {"result": StreamingResponse(live_clock())}


contextnode_node_297865 = ContextNode(node_logic)

rendernode_node_5b707f69 = RenderNode('shop.html')

rendernode_node_4439bbdd = RenderNode('product.html')

rendernode_node_47db0e8b = RenderNode('cart.html')

rendernode_node_b5e3eb77 = RenderNode('checkout.html')

rendernode_node_ee0994d4 = RenderNode('order_success.html')

rendernode_node_297862 = RenderNode('About.html')


# Node Connection Chain

servernode_node_42f0f91d.connect(httprequestsnode_node_463dc491)

httprequestsnode_node_463dc491.connect(actionloggernode_node_630a0f20)

```

```

actionloggernode_node_630a0f20.connect(ratelimitnode_node_4899a088)

ratelimitnode_node_4899a088.connect(antibotnode_node_789fa014)

antibotnode_node_789fa014.connect(csrfnode_node_5a10accc)

router_node_auto_1 = RouterNode([urlnode_node_ea5e3b4a, urlnode_node_aa37ab65, urlnode_node_f928646e,
urlnode_node_b1bb4012, urlnode_node_baa46610, urlnode_node_50dd4297, urlnode_node_172c8a35,
urlnode_node_bdd5e5ef, urlnode_node_37cf26cc, urlnode_node_a89ed2e0, urlnode_node_66d287f6,
urlnode_node_acff5eda, urlnode_node_fb230ff9, urlnode_node_297859fd, urlnode_node_297861])

csrfnode_node_5a10accc.connect(router_node_auto_1)

urlnode_node_ea5e3b4a.connect(logicnode_node_a2378860)

urlnode_node_aa37ab65.connect(logicnode_node_a2378860)

urlnode_node_f928646e.connect(logicnode_node_ada28e0d)

urlnode_node_b1bb4012.connect(logicnode_node_e2a12a36)

urlnode_node_baa46610.connect(logicnode_node_8e4cf37f)

urlnode_node_50dd4297.connect(logicnode_node_ea26aefb)

urlnode_node_172c8a35.connect(logicnode_node_38f5c7ca)

urlnode_node_bdd5e5ef.connect(logicnode_node_4215f0d1)

urlnode_node_37cf26cc.connect(logicnode_node_699b71b9)

urlnode_node_a89ed2e0.connect(logicnode_node_f193a041)

urlnode_node_66d287f6.connect(logicnode_node_32a2c2e6)

urlnode_node_acff5eda.connect(logicnode_node_9b18f0c1)

urlnode_node_fb230ff9.connect(rendernode_node_a5fac370)

urlnode_node_297859fd.connect(rendernode_node_b29ad28e)

urlnode_node_297861.connect(contextnode_node_297865)

logicnode_node_a2378860.connect(rendernode_node_5b707f69)

logicnode_node_ada28e0d.connect(rendernode_node_4439bbdd)

logicnode_node_e2a12a36.connect(rendernode_node_47db0e8b)

logicnode_node_4215f0d1.connect(rendernode_node_b5e3eb77)

logicnode_node_f193a041.connect(rendernode_node_ee0994d4)

contextnode_node_297865.connect(rendernode_node_297862)

# Start Server

print('Node Editor Generated Server starting...')

print(f' -> http://{servernode_node_42f0f91d.host}:{servernode_node_42f0f91d.port}')

from nodes.server_node import FrameworkHandler

FrameworkHandler.server_node = servernode_node_42f0f91d

```

```

FrameworkHandler._middleware_chain = None # Reset cache on redeploy

from http.server import HTTPServer

from socketserver import ThreadingMixIn

class ThreadedHTTPServer(ThreadingMixIn, HTTPServer): pass

with ThreadedHTTPServer((servernode_node_42f0f91d.host, servernode_node_42f0f91d.port), FrameworkHandler)
as httpd:

    try:

        httpd.serve_forever()

    except KeyboardInterrupt:

        pass

```

=====

```

=== FILE: .\settings.py ===

```

```

import os

```

```

# -----

```

```

# Base Directories

```

```

# -----

```

```

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

```

```

TEMPLATES_DIR = os.path.join(BASE_DIR, 'templates')

```

```

STATIC_URL = '/static/'

```

```

STATIC_ROOT = os.path.join(BASE_DIR, 'static')

```

```

def get_env(key, default=None, required=False, cast=str):

```

```

    # Step 1: Check environment variable

```

```

    value = os.environ.get(key)

```

```

    # Step 2: Check .env file

```

```

    if value is None:

```

```

        env_file = os.path.join(BASE_DIR, '.env')

```

```

        if os.path.exists(env_file):

```



```

with open(env_file, 'r') as f:
    for line in f:
        line = line.strip()
        # Skip comments and blanks
        if not line or line.startswith('#'):
            continue
        if '=' in line:
            k, _, v = line.partition('=')
            if k.strip() == key:
                value = v.strip()
                # Remove quotes
                if value.startswith(('\'', '\"')) and value.endswith(('\'', '\"')):
                    value = value[1:-1]
                break

# Step 3: Use default
if value is None:
    if required:
        raise RuntimeError(
            f"Required environment variable '{key}' is not "
            f"set. Add it to .env file or set as env variable."
        )
    value = default

# Step 4: Cast to correct type
if value is None:
    return None

try:
    if cast == bool:
        return str(value).lower() in ('true', '1', 'yes', 'on')
    return cast(value)
except (ValueError, TypeError):

```

```
return default
```

```
# -----  
  
# Server  
  
# -----  
  
DEBUG = get_env('DEBUG', 'True', cast=bool)  
  
HOST  = get_env('HOST', '127.0.0.1')  
  
PORT  = get_env('PORT', '8000', cast=int)  
  
ALLOWED_HOSTS = ['*']  
  
  
# -----  
  
# Secret Key  
  
# -----  
  
def get_secret_key():  
    """  
    Reads secret key from .secret_key file.  
    Generate it by running: python setup_project.py  
    """  
    secret_file = os.path.join(BASE_DIR, '.secret_key')  
    try:  
        with open(secret_file, 'r') as f:  
            return f.read().strip()  
    except FileNotFoundError:  
        raise RuntimeError(  
            "Secret key not found. Run: python setup_project.py"  
        )  
  
SECRET_KEY = get_secret_key()  
  
  
# -----  
  
# Installed Node Packages  
  
# -----  
  
INSTALLED_NODES = [
```

```

    'nodes',
    'plugins',
]

# -----

# Middleware Chain (import path strings, executed top → bottom)
# -----

MIDDLEWARE = [
    'nodes.middleware.common.SimpleLoggingMiddleware',
    'nodes.middleware.common.SecurityMiddleware',
]

# -----

# Database
# -----

DATABASES = {
    'default': {
        'ENGINE': 'sqlite3',
        'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),
    }
}

# -----

# Security
# -----

SECURITY = {
    'RATE_LIMIT_ENABLED': False,
    'RATE_LIMIT_MAX': 500,          # requests per window
    'RATE_LIMIT_WINDOW': 60,       # seconds
    'CSRF_ENABLED': True,
    'ANTI_SCRAPING_ENABLED': True, # User-Agent checks
    'SCREEN_PROTECTION_ENABLED': True, # Black screen on blur/printscreens
}

```

```

# -----
# Logging
# -----

LOGGING = {
    'ENABLED': True,
    'LOG_DIR': os.path.join(BASE_DIR, 'core', 'logs'),
    'FORMAT': ' [{timestamp}] {method} {path} | UA: {user_agent}',
}

=====

=== FILE: .\setup_project.py ===

"""
setup_project.py – Gravity-Flow Framework Setup Script
Run this ONCE before starting the server:

    python setup_project.py
"""

import secrets
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
SECRET_FILE = os.path.join(BASE_DIR, '.secret_key')
DB_FILE = os.path.join(BASE_DIR, 'db.sqlite3')
LOG_DIR = os.path.join(BASE_DIR, 'core', 'logs')

def generate_secret_key():
    if os.path.exists(SECRET_FILE):
        print(" [skip] Secret key already exists.")
        return
    key = secrets.token_urlsafe(64)

```

```

with open(SECRET_FILE, 'w') as f:

    f.write(key)

print(" [ok] Secret key generated → .secret_key")


def create_log_directory():

    os.makedirs(LOG_DIR, exist_ok=True)

    print(f" [ok] Log directory ready → core/logs/")


def create_database():

    import sqlite3

    conn = sqlite3.connect(DB_FILE)

    conn.execute("PRAGMA journal_mode=WAL;")

    conn.close()

    print(f" [ok] Database initialized → db.sqlite3")


def setup_all():

    print("\n Gravity-Flow Framework – Project Setup")

    print("=" * 45)

    generate_secret_key()

    create_log_directory()

    create_database()

    print("=" * 45)

    print(" Setup complete! Run: python main.py\n")


if __name__ == "__main__":

    setup_all()

```

```

=====

```

```
=== FILE: .\test_logger.py ===
```

```
import os
```

```
import sys
```

```
sys.path.insert(0, '.')
```

```
from core.logging import get_logger
```

```
import threading
```

```
logger = get_logger()
```

```
# Test basic levels
```

```
logger.debug("debug message")
```

```
logger.info("info message")
```

```
logger.warn("warn message")
```

```
logger.error("error message")
```

```
# Test structured request logic
```

```
logger.request('GET', '/shop', 200, 45, '127.0.0.1', 'Chrome')
```

```
# Test rotation (reduce max bytes for rapid testing)
```

```
logger.max_bytes = 100
```

```
def flood():
```

```
    for i in range(200):
```

```
        logger.info(f"flood {i} thread {threading.get_ident()}")
```

```
t1 = threading.Thread(target=flood)
```

```
t2 = threading.Thread(target=flood)
```

```
t1.start()
```

```
t2.start()
```

```
t1.join()
```

```
t2.join()
```

```
print("Log directory contents:")
print(os.listdir(logger.log_dir))
```

```
=====
```

```
=== FILE: .\test_zoom.html ===
```

```
<!DOCTYPE html>

<html>

<head>

<style>

#container { width: 500px; height: 500px; border: 1px solid black; overflow: hidden; position: relative; }
#layer { width:100%; height:100%; background: lightpink; transform-origin: 0 0; }
.node { width: 50px; height: 50px; background: red; position: absolute; top: 200px; left: 200px; }

</style>

</head>

<body>

<div id='container'><div id='layer'><div class='node'></div></div></div>

<script>

let panX=0, panY=0, scale=1;

let container = document.getElementById('container');

let layer = document.getElementById('layer');

container.addEventListener('wheel', e => {

e.preventDefault();

let zoomIntensity = 0.05;

let delta = e.deltaY > 0 ? 1 : -1;

let factor = Math.exp(delta * zoomIntensity);

let newScale = scale * factor;

const rect = container.getBoundingClientRect();
```

```

const mouseX = e.clientX - rect.left;

const mouseY = e.clientY - rect.top;


const canvasX = (mouseX - panX) / scale;

const canvasY = (mouseY - panY) / scale;


scale = newScale;

panX = mouseX - canvasX * scale;

panY = mouseY - canvasY * scale;


layer.style.transform =      ranslate(px, px) scale();

console.log(panX, panY, scale);

});

</script>

</body>

</html>

```

=====

=== FILE: .\wsgi.py ===

"""

wsgi.py – Gravity-Flow Framework WSGI Adapter

WSGI = Web Server Gateway Interface

Isse framework Gunicorn, uWSGI, Apache, Nginx ke saath chal sakta hai.

Usage (Production):

```
gunicorn wsgi:application --workers 4 --bind 0.0.0.0:8000
```

Usage (Dev, same as before):

```
python main.py
```


How it works:

WSGI standard ek function chahta hai: `application(environ, start_response)`

Ye module `main.py` ka `ServerNode` graph lete hai aur us pe WSGI wrapper lagata hai.

Full deployment example (Windows/Linux):

```
pip install gunicorn
```

```
gunicorn wsgi:application --workers 4 --bind 0.0.0.0:8000 --reload
```

```
"""
```

```
import io
```

```
import sys
```

```
import os
```

```
# Add framework root to path
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
if BASE_DIR not in sys.path:
```

```
    sys.path.insert(0, BASE_DIR)
```

```
import settings
```

```
# -----
```

```
# WSGI Request Wrapper
```

```
# -----
```

```
class WSGIRequestWrapper:
```

```
    """
```

```
    Wraps a WSGI environ dict to look like our RequestWrapper.
```

```
    So all existing nodes work unchanged with WSGI servers.
```

```
    """
```

```
    import urllib.parse as _up
```

```
    import json as _json
```

```

def __init__(self, environ):
    import urllib.parse

    self.environ = environ

    self.method = environ.get('REQUEST_METHOD', 'GET').upper()

    self.path = environ.get('PATH_INFO', '/')

    self.headers = WSGIHeaders(environ)

    self.url_params = {}

    self.context = {}

    # Query string
    qs = environ.get('QUERY_STRING', '')

    self.query_params = {
        k: v[0] if len(v) == 1 else v
        for k, v in urllib.parse.parse_qs(qs).items()
    }

    # Body
    self.body_bytes = b''

    self.params = {}

    self._files = {}

    if self.method in ('POST', 'PUT', 'PATCH', 'DELETE'):
        try:
            length = int(environ.get('CONTENT_LENGTH', 0) or 0)

            if length > 0:
                self.body_bytes = environ['wsgi.input'].read(length)

                ct = environ.get('CONTENT_TYPE', '')

                if 'application/x-www-form-urlencoded' in ct:
                    self.params = urllib.parse.parse_qs(
                        self.body_bytes.decode('utf-8', errors='replace')
                    )

        except Exception:
            pass

```

```
# Fake handler for middleware/security nodes that do client_address
```

```
self.handler = _FakeHandler(envIRON)
```

```
@property
```

```
def content_type(self):
```

```
    return self.envIRON.get('CONTENT_TYPE', '').lower()
```

```
def get_param(self, key, default=None):
```

```
    if key in self.url_params:
```

```
        return self.url_params[key]
```

```
    v = self.params.get(key)
```

```
    return v[0] if v else default
```

```
def get_json(self):
```

```
    import json
```

```
    try:
```

```
        return json.loads(self.body_bytes.decode('utf-8'))
```

```
    except Exception:
```

```
        return None
```

```
def get_file(self, key):
```

```
    return self._files.get(key)
```

```
class WSGIHeaders:
```

```
    """Dict-like wrapper for WSGI environ HTTP headers."""
```

```
def __init__(self, environ):
```

```
    self._env = environ
```

```
def get(self, name, default=None):
```

```
    key = 'HTTP_' + name.upper().replace('-', '_')
```

```
    # Special cases
```

```

        if name.lower() == 'content-type':
            key = 'CONTENT_TYPE'

        elif name.lower() == 'content-length':
            key = 'CONTENT_LENGTH'

        return self._env.get(key, default)

def __contains__(self, name):
    return self.get(name) is not None

def __getitem__(self, name):
    v = self.get(name)

    if v is None:
        raise KeyError(name)

    return v

class _FakeHandler:
    """Mimics http.server handler for nodes that access handler.client_address."""

    def __init__(self, environ):
        addr = environ.get('REMOTE_ADDR', '127.0.0.1')
        self.client_address = (addr, 0)

# -----
# WSGI Application Factory
# -----

def _load_server_node():
    """
    Import main.py's server_node graph.

    main.py is auto-generated by the Node Editor.

    """
    import importlib.util

```

```

main_path = os.path.join(BASE_DIR, 'main.py')

spec = importlib.util.spec_from_file_location('main', main_path)

mod = importlib.util.module_from_spec(spec)

spec.loader.exec_module(mod)

# main.py sets FrameworkHandler.server_node – grab it
from nodes.server_node import FrameworkHandler

return FrameworkHandler.server_node


def create_app(server_node=None):
    """
    Create and return a WSGI-compatible application callable.

    Args:
        server_node : The root ServerNode. If None, loads from main.py.

    Returns:
        callable(envron, start_response) – standard WSGI app
    """
    from nodes.response import Response

    if server_node is None:
        server_node = _load_server_node()

    def application(envron, start_response):
        """WSGI entry point."""
        request = WSGIRequestWrapper(envron)

        # Run the node graph
        try:
            result = server_node.start_flow(request)
        except Exception as e:
            import traceback

```

```
tb = traceback.format_exc()

print(f"[WSGI] Unhandled error: {e}\n{tb}")

result = Response.server_error(str(e))
```

```
# Normalize result to Response
```

```
if result is None:
```

```
    # Check static file
```

```
    path = environ.get('PATH_INFO', '/')
```

```
    if path.startswith(settings.STATIC_URL):
```

```
        result = _serve_static(path)
```

```
    else:
```

```
        result = Response.not_found(f"No route matched: {path}")
```

```
elif isinstance(result, str):
```

```
    result = Response(body=result)
```

```
# Serialize
```

```
body_bytes = result.to_bytes()
```

```
headers = [
```

```
    ('Content-Type', result.content_type),
```

```
    ('Content-Length', str(len(body_bytes))),
```

```
    ('X-Content-Type-Options', 'nosniff'),
```

```
    ('X-Frame-Options', 'SAMEORIGIN'),
```

```
]
```

```
for k, v in result.headers.items():
```

```
    headers.append((k, v))
```

```
start_response(f"{result.status} {_status_text(result.status)}", headers)
```

```
return [body_bytes]
```

```
return application
```

```
def _serve_static(path):
```

```

"""Serve a static file from settings.STATIC_ROOT."""

from nodes.response import Response

rel_path = path[len(settings.STATIC_URL):]

full_path = os.path.join(settings.STATIC_ROOT, rel_path)

if not os.path.isfile(full_path):

    return Response.not_found(f"Static file not found: {rel_path}")

ext_map = {

    '.css': 'text/css', '.js': 'application/javascript',

    '.png': 'image/png', '.jpg': 'image/jpeg', '.svg': 'image/svg+xml',

    '.ico': 'image/x-icon', '.woff2': 'font/woff2',

}

ext = os.path.splitext(full_path)[1].lower()

ct = ext_map.get(ext, 'application/octet-stream')

with open(full_path, 'rb') as f:

    body = f.read()

return Response(body=body, content_type=ct)


def _status_text(code):

    _CODES = {

        200: 'OK', 201: 'Created', 204: 'No Content',

        301: 'Moved Permanently', 302: 'Found',

        400: 'Bad Request', 401: 'Unauthorized', 403: 'Forbidden',

        404: 'Not Found', 405: 'Method Not Allowed', 429: 'Too Many Requests',

        500: 'Internal Server Error', 503: 'Service Unavailable',

    }

    return _CODES.get(code, 'Unknown')


# -----

# Module-level application (for Gunicorn: gunicorn wsgi:application)

# -----

# NOTE: Only auto-loads when running via Gunicorn or direct call.

```

```
# Importing this module for testing does NOT start the server.

application = None

def _lazy_load():
    """Called by Gunicorn worker on first request if application is None."""
    global application
    if application is None:
        try:
            application = create_app()
        except Exception as _e:
            print(f"[WSGI] Warning: Could not load app: {_e}")
    return application

# Auto-load only when run directly (not on import)

if __name__ == '__wsgi__' or (
    '__name__' in dir() and
    os.environ.get('SERVER_SOFTWARE', '').startswith('gunicorn')
):
    _lazy_load()
```

```
=====
```

```
=== FILE: .\core\ai_graph.py ===
```

```
"""
```

```
core/ai_graph.py – AI-Readable Graph Format
```

Ye module graph.json ko ek clear, structured format mein export karta hai

jisse AI agents (GPT, Claude, Gemini) directly padhh ke samjhein:

- Konsa node konse se connected hai
- Har node ki responsibility kya hai

- Request kahan se aati hai, kahan jaati hai
- Agar error ho, kahan fix karna hai

Usage:

```
from core.ai_graph import GraphReader, GraphSummary

# node_editor/graph.json se load karo
reader = GraphReader.from_file('node_editor/graph.json')
summary = reader.to_ai_summary()

print(summary)          # Human-readable
print(reader.to_json()) # Machine-readable for AI

# Live running nodes se bhi describe kar sako
from core.ai_graph import describe_live_graph
describe_live_graph(server_node) # prints chain

"""

import json
import os
import settings

# -----
# Node Type Descriptions (for AI context)
# -----

_NODE_DESCRIPTIONS = {
    'ServerNode'      : 'HTTP server root. Listens on host:port. Entry point of the graph.',
    'HTTPRequestsNode' : 'Parses raw HTTP request into RequestWrapper (path, method, params, body).',
    'URLNode'          : 'Route matcher. Passes request forward only if path matches. Supports
/path/<param>.',
    'RouterNode'       : 'Distributes request to multiple URL branches. Acts as a hub.',
    'LogicNode'        : 'Executes a Python function. Takes request, returns dict context. Business logic
lives here.',
    'ContextNode'      : 'Updates request.context dict with function result. Used for chained data
building.'
```

```

'RenderNode'          : 'Reads an HTML template, injects context, returns final HTML string.',
'ModelNode'           : 'Runs a SQL query. Results stored in request.context[key]. Reads or writes DB.',
'ActionLoggerNode'    : 'Logs every request to core/logs/{ip}.txt. Transparent pass-through.',
'AntiBotNode'         : 'Blocks known bots and scrapers by User-Agent header.',
'RateLimitNode'       : 'Blocks IPs exceeding request rate. Config in settings.SECURITY.',
'CSRFNode'            : 'Validates CSRF token on POST requests. Generates token on GET.',
'ScreenProtectionNode': 'Injects JS to disable right-click, screenshots, and text selection.',
}

```

```

def _describe_node_type(node_type):
    return _NODE_DESCRIPTIONS.get(node_type, f'{node_type} – custom node.')

```

```

# -----
# GraphReader – parses graph.json
# -----

```

```

class GraphReader:
    """
    Reads and understands a node_editor graph.json file.

    graph.json structure:
    {
        "nodes": [{"id": "node-10", "type": "ServerNode", "config": {...}}, ...],
        "connections": [{"source": "node-10", "target": "node-11"}, ...]
    }
    """

    def __init__(self, graph_data):
        self.nodes = {n['id']: n for n in graph_data.get('nodes', [])}
        self.connections = graph_data.get('connections', [])
        self._out_map = self._build_out_map()

```

```

@classmethod
def from_file(cls, path=None):
    if path is None:
        path = os.path.join(settings.BASE_DIR, 'node_editor', 'graph.json')
    with open(path, 'r', encoding='utf-8') as f:
        return cls(json.load(f))

```

```

@classmethod
def from_dict(cls, data):
    return cls(data)

```

```

def _build_out_map(self):
    out = {}
    for c in self.connections:
        out.setdefault(c['source'], []).append(c['target'])
    return out

```

```

def _find_root(self):
    """Find the ServerNode (root of the graph)."""
    for nid, node in self.nodes.items():
        if node['type'] == 'ServerNode':
            return nid
    return next(iter(self.nodes)) if self.nodes else None

```

```

def _bfs_order(self):
    """Return node IDs in BFS order from root."""
    root = self._find_root()
    if not root:
        return []
    visited = []
    queue = [root]
    seen = set()

```

```

while queue:

    curr = queue.pop(0)

    if curr in seen:

        continue

    seen.add(curr)

    visited.append(curr)

    queue.extend(self._out_map.get(curr, []))

return visited

```

```

# -----
# AI Summary outputs
# -----

```

```

def to_ai_summary(self):

    """

    Generate a human + AI readable text summary of the graph.

    This can be pasted into an AI prompt for context.

    """

    order = self._bfs_order()

    lines = [

        "=" * 55,

        " GRAVITY-FLOW GRAPH SUMMARY",

        " (AI-readable node-by-node description)",

        "=" * 55,

        f" Total Nodes      : {len(self.nodes)}",

        f" Total Connections: {len(self.connections)}",

        "=" * 55,

        "",

        " REQUEST FLOW (top to bottom):",

        "",

    ]

    for i, nid in enumerate(order):

```

```

node      = self.nodes[nid]

ntype     = node['type']

config    = node.get('config', {})

targets   = self._out_map.get(nid, [])


lines.append(f" [{i+1}] {ntype} (id: {nid})")

lines.append(f"      Role   : {_describe_node_type(ntype)}")


# Config details

if ntype == 'ServerNode':

    lines.append(f"      Config : host={config.get('ip', '127.0.0.1')}
port={config.get('port', 8000)}")

elif ntype == 'URLNode':

    lines.append(f"      Config : path={config.get('path', '/')}")

elif ntype == 'RenderNode':

    lines.append(f"      Config : template={config.get('filename', 'auto')}")

elif ntype == 'ModelNode':

    lines.append(f"      Config : query={config.get('query', '')[:50]}")

elif ntype in ('LogicNode', 'ContextNode'):

    code_preview = config.get('code', '')[:80].replace('\n', ' ')

    lines.append(f"      Logic   : {code_preview}...")


# Connections

if targets:

    target_types = [self.nodes[t]['type'] for t in targets if t in self.nodes]

    lines.append(f"      Next    : {' + '.join(target_types)}")

else:

    lines.append(f"      Next    : [END - response returned here]")

lines.append("")


lines.append("=" * 55)

lines.append("  HOW TO EDIT:")

lines.append("  - Open node_editor/index.html to visually modify the graph")

lines.append("  - Or edit node_editor/graph.json directly")

```

```

lines.append(" - Then Deploy to regenerate main.py")

lines.append("=" * 55)

return '\n'.join(lines)

```

```
def to_json(self):
```

```
    """
```

```
    Return a structured JSON string for AI agents to parse.
```

```
    Contains: nodes, connections, flow_order, node_descriptions.
```

```
    """
```

```
    order = self._bfs_order()
```

```
    output = {
```

```
        "framework"      : "Gravity-Flow Node Graph Framework",
```

```
        "total_nodes"    : len(self.nodes),
```

```
        "total_connections": len(self.connections),
```

```
        "flow_order"     : [
```

```
            {
```

```
                "step"      : i + 1,
```

```
                "id"       : nid,
```

```
                "type"     : self.nodes[nid]['type'],
```

```
                "description": _describe_node_type(self.nodes[nid]['type']),
```

```
                "config"    : self.nodes[nid].get('config', {}),
```

```
                "connects_to": [
```

```
                    self.nodes[t]['type']
```

```
                    for t in self._out_map.get(nid, [])
```

```
                    if t in self.nodes
```

```
                ],
```

```
            }

```

```
        for i, nid in enumerate(order)
```

```
    ],
```

```
    "raw_connections": self.connections,
```

```
    "ai_instructions": (
```

```
        "This is a node graph web framework. Each node has ONE responsibility. "
```

```
        "To fix an error in a specific node, locate the node by 'id' or 'type' "
```

"in flow_order and edit its 'config'. "

"To add a route: add a URLNode → LogicNode → RenderNode chain and connect it. "

"After editing graph.json, Deploy from the Node Editor to regenerate main.py."

),

}

return json.dumps(output, indent=2, ensure_ascii=False)

def routes(self):

"""Return all URLNode routes for quick reference."""

return [

{

 'id' : nid,

 'path': node.get('config', {}).get('path', '?'),

}

for nid, node in self.nodes.items()

if node['type'] == 'URLNode'

]

def print_routes(self):

print("\n Registered Routes:")

for r in self.routes():

 print(f" GET/POST {r['path']} (node: {r['id']})")

print()

Live Graph Descriptor (from running nodes, not graph.json)

def describe_live_graph(root_node, indent=0):

"""

Print the live node graph structure from a running ServerNode.

Works on the actual Python objects (not graph.json).

Usage (in main.py or shell):

```
from core.ai_graph import describe_live_graph

describe_live_graph(server_node)

"""

chain = root_node.describe_chain()

print(f"\n{'-'*50}")

print(f"  Live Node Graph")

print(f"{'-'*50}")

for i, node_info in enumerate(chain):

    prefix = '  ' + ('└' if i == len(chain) - 1 else '├')

    name    = node_info.get('name', '?')

    ntype   = node_info.get('type', '?')

    nxt     = node_info.get('next_name', None)

    print(f"{prefix} [{i+1}] {ntype} ({name})")

    if i < len(chain) - 1:

        print(f"  {'|'} if i < len(chain) - 2 else ' ')

print(f"{'-'*50}\n")
```

=====

=== FILE: .\core\db.py ===

```
import sqlite3

import threading

import os

import settings

from contextlib import contextmanager
```

```
class Database:
```

```
    """
```


Thread-safe SQLite database wrapper using `threading.local()` for connections.

Each thread gets its OWN persistent connection – created once, reused forever:

- WAL mode + foreign keys configured ONCE per thread connection
- No `check_same_thread=False` (removed – safe by design now)
- No `conn.close()` in hot-path methods (`execute/fetchall`)
- `rollback()` on errors to maintain consistent state
- Singleton so all logic files share the same Database object

Thread lifecycle:

Thread starts → first query → `get_connection()` creates conn → stored in `_local.conn`

Thread ends → Python GC closes connection automatically

(or call `close_connection()` explicitly for clean shutdown)

"""

`_instance = None`

`_local = threading.local()` # Each thread gets its own namespace

`def __new__(cls):`

`if cls._instance is None:`

`cls._instance = super(Database, cls).__new__(cls)`

`cls._instance.db_path = os.path.join(settings.BASE_DIR, 'db.sqlite3')`

`return cls._instance`

`# -----`

`# Connection Management`

`# -----`

`def get_connection(self) -> sqlite3.Connection:`

`"""`

Return this thread's persistent connection, creating it if needed.

Configuration (WAL, foreign keys, `row_factory`) is applied ONCE.

`"""`

```

if not hasattr(self._local, 'conn') or self._local.conn is None:

    conn = sqlite3.connect(

        self.db_path,

        check_same_thread=True, # Safe: each thread has its own conn

        timeout=30,

    )

    # Configure once per connection

    conn.execute("PRAGMA journal_mode=WAL")

    conn.execute("PRAGMA foreign_keys=ON")

    conn.row_factory = sqlite3.Row

    self._local.conn = conn

return self._local.conn

```

```

def close_connection(self):

```

```

    """

```

```

    Explicitly close this thread's connection.

```

```

    Call after a request ends if running in a threaded server.

```

```

    """

```

```

    if hasattr(self._local, 'conn') and self._local.conn is not None:

```

```

        try:

```

```

            self._local.conn.close()

```

```

        except Exception:

```

```

            pass

```

```

        self._local.conn = None

```

```

# -----

```

```

# Core Query Methods (use persistent thread-local connection)

```

```

# -----

```

```

def execute(self, query, params=()):

```

```

    """

```

```

    Execute a write query (INSERT / UPDATE / DELETE / DDL).

```

Uses the thread's persistent connection – does NOT close it.

Rolls back on error.

"""

```
conn = self.get_connection()
```

```
try:
```

```
    cursor = conn.cursor()
```

```
    cursor.execute(query, params)
```

```
    conn.commit()
```

```
    return cursor
```

```
except Exception as e:
```

```
    conn.rollback()
```

```
    print(f"Database Error: {e}")
```

```
    raise e
```

```
def executemany(self, query, params_list):
```

```
    """Bulk insert/update optimization. Uses persistent connection."""
```

```
    conn = self.get_connection()
```

```
    try:
```

```
        conn.executemany(query, params_list)
```

```
        conn.commit()
```

```
    except Exception as e:
```

```
        conn.rollback()
```

```
        print(f"Database Error (Bulk): {e}")
```

```
        raise e
```

```
def executescript(self, script):
```

```
    """
```

```
    Run a raw SQL script (good for migrations/DDL/triggers).
```

```
    Uses a temporary dedicated connection because executescript()
```

```
    auto-commits and cannot run inside an existing transaction.
```

```
    """
```

```
    conn = sqlite3.connect(self.db_path, timeout=30)
```

```
    try:
```

```

        conn.executescript(script)

except Exception as e:

    print(f"Database Error (Script): {e}")

    raise e

finally:

    conn.close() # Temporary connection – always close


def fetchall(self, query, params=()):
    """
    Execute a SELECT and return a list of dicts.

    Uses the thread's persistent connection – does NOT close it.
    """
    conn = self.get_connection()

    try:

        cursor = conn.cursor()

        cursor.execute(query, params)

        return [dict(row) for row in cursor.fetchall()]

    except Exception as e:

        print(f"Database Error: {e}")

        return []


# -----
# Transaction Context Manager
# -----


@contextmanager
def transaction(self):
    """
    Atomic transaction context manager.

    Usage:

        with db.transaction() as conn:

            conn.execute(q1, params1)

```

```

        conn.execute(q2, params2)

    """

    conn = self.get_connection()

    try:

        yield conn

        conn.commit()

    except Exception as e:

        conn.rollback()

        print(f"Transaction Rolled Back: {e}")

        raise e

    # No conn.close() – thread keeps its connection

# -----

# Utility – Stored Procedure / Function Registration

# -----

def register_function(self, conn, name, num_params, func):

    """

    Registers a Python function as a SQL scalar function.

    Usage in SQL: SELECT my_func(col) FROM table

    """

    conn.create_function(name, num_params, func)

# -----

# DDL Convenience Methods

# -----

def setup_tables(self):

    create_schema = '''

    CREATE TABLE IF NOT EXISTS users (

        id INTEGER PRIMARY KEY AUTOINCREMENT,

        name TEXT NOT NULL,

        email TEXT UNIQUE,

```

```

        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

        is_premium BOOLEAN DEFAULT 0

    );
    ...

```

```

self.executescript(create_schema)

```

```

create_trigger = '''

CREATE TRIGGER IF NOT EXISTS validate_email_suffix

BEFORE INSERT ON users

BEGIN

    SELECT

    CASE

        WHEN NEW.email NOT LIKE '%@%' THEN

            RAISE (ABORT, 'Invalid email address')

    END;

END;

...

```

```

self.executescript(create_trigger)

```

```

def create_table(self, table_name, columns_def):

```

```

    """Creates a table if it doesn't exist. columns_def: 'id INTEGER PRIMARY KEY, name TEXT'"""

    self.execute(f"CREATE TABLE IF NOT EXISTS {table_name} ({columns_def});")

```

```

def alter_table(self, table_name, operation, details):

```

```

    """Alters a table. operation: 'ADD' or 'RENAME'. details: column definition."""

    op = operation.upper()

    if op == 'ADD':

        self.execute(f"ALTER TABLE {table_name} ADD {details};")

    elif op == 'RENAME':

        self.execute(f"ALTER TABLE {table_name} RENAME TO {details};")

    else:

        raise ValueError(f"Unsupported ALTER operation: {operation}")

```

```

def drop_table(self, table_name):
    """Drops a table if it exists."""
    self.execute(f"DROP TABLE IF EXISTS {table_name};")

def create_view(self, view_name, select_query):
    """Creates a view."""
    self.execute(f"CREATE VIEW IF NOT EXISTS {view_name} AS {select_query};")

def drop_view(self, view_name):
    """Drops a view."""
    self.execute(f"DROP VIEW IF EXISTS {view_name};")

def create_index(self, index_name, table_name, columns, unique=False):
    """Creates an index."""
    unique_clause = "UNIQUE" if unique else ""
    self.execute(
        f"CREATE {unique_clause} INDEX IF NOT EXISTS {index_name} "
        f"ON {table_name} ({columns});"
    )

```

=====

=== FILE: .\core\errors.py ===

"""

core/errors.py – Gravity-Flow Node Error Reporting System

Jab koi bhi node fail hota hai, yeh module clearly batata hai:

- Konsa node crash hua
- Kya input tha
- Kya error tha (readable)
- Kahan fix karna hai

AI-Friendly Design:

- Structured JSON format – AI directly parse kar sakta hai
- No ambiguous stack traces – pinpointed node name
- Context preserved – request path + method bhi log hota hai

"""

```
import traceback
```

```
import datetime
```

```
import json
```

```
import os
```

```
import settings
```

```
class NodeError(Exception):
```

```
    """
```

```
    Raised when a node fails during graph execution.
```

```
    Carries structured context so errors are easy to diagnose.
```

```
    """
```

```
    def __init__(self, node_name, node_type, original_error, input_data=None):
```

```
        self.node_name = node_name
```

```
        self.node_type = node_type
```

```
        self.original = original_error
```

```
        self.input_data = input_data
```

```
        self.timestamp = datetime.datetime.now().isoformat()
```

```
        super().__init__(str(original_error))
```

```
    def to_dict(self):
```

```
        return {
```

```
            "error": "NodeError",
```

```
            "node": self.node_name,
```

```
            "type": self.node_type,
```

```
            "message": str(self.original),
```

```
            "timestamp": self.timestamp,
```

```
            "input_type": type(self.input_data).__name__,
```



```
}
```

```
def __str__(self):
    return (
        f"\n{'='*55}\n"
        f"  NODE ERROR DETECTED\n"
        f"{'='*55}\n"
        f"  Node      : {self.node_name} ({self.node_type})\n"
        f"  Error      : {type(self.original).__name__}: {self.original}\n"
        f"  Time       : {self.timestamp}\n"
        f"{'='*55}\n"
        f"  FIX HINT : Check the '{self.node_name}' node logic.\n"
        f"{'='*55}"
    )
```

```
class NodeErrorReporter:
```

```
    """
```

```
    Central error reporting hub for the node graph.
```

```
    Features:
```

- Console output: colored, readable, pinpointed
- File logging: structured JSON per error
- HTML error page: shown in browser during DEBUG mode
- AI summary: machine-readable JSON for AI agents to parse

```
    """
```

```
LOG_DIR = os.path.join(settings.BASE_DIR, 'core', 'logs', 'errors')
```

```
@classmethod
```

```
def _ensure_dir(cls):
```

```
    os.makedirs(cls.LOG_DIR, exist_ok=True)
```

```

@classmethod
def report(cls, node_name, node_type, error, request=None, tb_str=None):
    """
    Report an error from a node.

    Args:
        node_name : str    - node variable name (e.g. 'urlnode_node_12')
        node_type : str    - node class name   (e.g. 'URLNode')
        error      : Exception
        request    : RequestWrapper | None
        tb_str     : str | None - traceback string
    """
    # Build structured record
    record = {
        "timestamp" : datetime.datetime.now().isoformat(),
        "node"       : node_name,
        "node_type"  : node_type,
        "error_type" : type(error).__name__,
        "message"    : str(error),
        "request"    : {
            "method": getattr(request, 'method', 'UNKNOWN'),
            "path"  : getattr(request, 'path', 'UNKNOWN'),
        } if request else None,
        "traceback" : tb_str or traceback.format_exc(),
    }

    # 1. Console
    cls._print_console(record)

    # 2. File log
    cls._write_log(record)

    # 3. Return HTML error page or JSON

```

```
return cls._make_error_page(record)
```

```
@classmethod
```

```
def _print_console(cls, record):
```

```
    print(f"\n{'='*55}")
```

```
    print(f"    NODE ERROR")
```

```
    print(f"{'='*55}")
```

```
    print(f" Node      : {record['node']} ({record['node_type']})")
```

```
    print(f" Error      : {record['error_type']}: {record['message']}")
```

```
    print(f" Request   : {record['request']['method'] if record['request'] else 'N/A'}")
```

```
        f" {record['request']['path'] if record['request'] else ''}")
```

```
    print(f"{'='*55}")
```

```
    # Show only last 3 traceback lines to avoid noise
```

```
    if record['traceback']:
```

```
        lines = [l for l in record['traceback'].strip().splitlines() if l.strip()]
```

```
        for line in lines[-3:]:
```

```
            print(f" {line.strip()}")
```

```
    print(f"{'='*55}\n")
```

```
@classmethod
```

```
def _write_log(cls, record):
```

```
    try:
```

```
        cls._ensure_dir()
```

```
        # 1. Daily JSONL log
```

```
        date_str = datetime.date.today().isoformat()
```

```
        log_file = os.path.join(cls.LOG_DIR, f"{date_str}.jsonl")
```

```
        with open(log_file, 'a', encoding='utf-8') as f:
```

```
            f.write(json.dumps(record, ensure_ascii=False) + '\n')
```

```
        # 2. Live error state file – node editor reads this to highlight nodes red
```

```
        # Format: {"node_type": {"node": ..., "message": ..., "timestamp": ...}}
```

```
        state_file = os.path.join(settings.BASE_DIR, 'core', 'logs', 'error_state.json')
```

```
    try:
```

```

        with open(state_file, 'r', encoding='utf-8') as f:

            state = json.load(f)

    except Exception:

        state = {}

```

```

state[record['node_type']] = {

    "node"      : record['node'],

    "node_type" : record['node_type'],

    "message"   : record['message'],

    "error_type": record['error_type'],

    "timestamp" : record['timestamp'],

}

```

```

with open(state_file, 'w', encoding='utf-8') as f:

    json.dump(state, f, ensure_ascii=False, indent=2)

```

```

except Exception as e:

    print(f"[ErrorReporter] Could not write log: {e}")

```

@classmethod

def clear_errors(cls):

"""Clear the live error state (call on successful deploy)."""

state_file = os.path.join(settings.BASE_DIR, 'core', 'logs', 'error_state.json')

try:

with open(state_file, 'w', encoding='utf-8') as f:

json.dump({}, f)

except Exception:

pass

@classmethod

def _make_error_page(cls, record):

"""Return an HTML error page (DEBUG mode) or minimal 500 page."""

if not getattr(settings, 'DEBUG', False):

```

return (
    '<!DOCTYPE html><html><body style="font-family:sans-serif;text-align:center;'
    'padding:60px;background:#0f172a;color:#e2e8f0">'
    '<h1 style="color:#f43f5e;font-size:3rem">500</h1>'
    '<p>An internal error occurred.</p></body></html>'
)

```

```

tb_html = (record['traceback'] or '').replace('<', '&lt;').replace('>', '&gt;')
req      = record['request'] or {}
return f'<!DOCTYPE html>

```

```

<html lang="en">

```

```

<head>

```

```

<meta charset="UTF-8">

```

```

<title>Node Error – {record['node']}</title>

```

```

<style>

```

```

* {{ box-sizing: border-box; margin: 0; padding: 0; }}

```

```

body {{ background: #0f172a; color: #e2e8f0; font-family: 'Segoe UI', sans-serif; padding: 2rem; }}

```

```

.header {{ background: #f43f5e22; border: 1px solid #f43f5e55; border-radius: 12px;

```

```

padding: 1.5rem 2rem; margin-bottom: 1.5rem; }}

```

```

.badge {{ display: inline-block; background: #f43f5e; color: white; font-size: 0.75rem;

```

```

font-weight: bold; padding: 2px 10px; border-radius: 20px; margin-bottom: 0.5rem; }}

```

```

h1 {{ font-size: 1.6rem; color: #fca5a5; margin-bottom: 0.4rem; }}

```

```

.meta {{ color: #94a3b8; font-size: 0.85rem; }}

```

```

.card {{ background: #1e293b; border-radius: 10px; padding: 1.2rem 1.5rem; margin-bottom: 1rem; }}

```

```

.card h3 {{ font-size: 0.75rem; text-transform: uppercase; letter-spacing: 0.1em;

```

```

color: #64748b; margin-bottom: 0.6rem; }}

```

```

.node-name {{ font-size: 1.1rem; font-weight: bold; color: #38bdf8; }}

```

```

.error-msg {{ font-size: 1rem; color: #fca5a5; }}

```

```

pre {{ background: #0f172a; border-radius: 8px; padding: 1rem; overflow-x: auto;

```

```

font-size: 0.8rem; color: #94a3b8; line-height: 1.6; border: 1px solid #1e293b; }}

```

```

.fix-hint {{ background: #14532d33; border: 1px solid #16a34a55; border-radius: 10px;

```

```

padding: 1rem 1.5rem; color: #86efac; }}

```

```

.fix-hint h3 {{ color: #4ade80; margin-bottom: 0.4rem; font-size: 0.9rem; }}

```

```

</style>

</head>

<body>

  <div class="header">

    <div class="badge">NODE ERROR</div>

    <h1> {record['error_type']}: {record['message']}</h1>

    <div class="meta">{record['timestamp']} &nbsp;|&nbsp; {req.get('method','?')}
{req.get('path','?')}</div>

  </div>


  <div class="card">

    <h3>Which Node Failed</h3>

    <div class="node-name">{record['node']}</div>

    <div class="meta" style="margin-top:4px">Type: {record['node_type']}</div>

  </div>


  <div class="card">

    <h3>Traceback</h3>

    <pre>{tb_html}</pre>

  </div>


  <div class="fix-hint">

    <h3> How to Fix</h3>

    <p>Open the <strong>{record['node']}</strong> node in the Node Editor and check its
      logic. Each node is independent – only this node needs to be fixed.</p>

  </div>


  <div class="card" style="margin-top:1rem">

    <h3>AI-Readable JSON (for AI agents)</h3>

    <pre>{json.dumps(record, indent=2, ensure_ascii=False)}</pre>

  </div>

</body>

</html>"""

```

```

def wrap_node_process(node, method_name='process'):
    """
    Decorator utility – wraps a node's process() method with error reporting.
    Called automatically by BaseNode when ERROR_REPORTING is enabled in settings.

    Usage (manual):
        wrap_node_process(my_node)
    """
    original = getattr(node, method_name)

    def safe_process(data):
        try:
            return original(data)
        except Exception as e:
            tb = traceback.format_exc()
            node_name = getattr(node, '_node_name',
                                f"{type(node).__name__}_at_{id(node)}")
            return NodeErrorReporter.report(
                node_name = node_name,
                node_type = type(node).__name__,
                error      = e,
                request    = data if hasattr(data, 'path') else None,
                tb_str     = tb,
            )

    setattr(node, method_name, safe_process)
    return node

```

```
=====
```

```
=== FILE: .\core\logging.py ===
```

```
#core/logging.py

import os

import threading

import datetime


class WebNodeLogger:

    # Log levels

    DEBUG = 10

    INFO = 20

    WARN = 30

    ERROR = 40

    def __init__(self,

                  log_dir,

                  max_bytes=10*1024*1024,

                  backup_count=5):

        self.log_dir = log_dir

        self.max_bytes = max_bytes

        self.backup_count = backup_count

        self._level = self.INFO

        self._lock = threading.Lock()

        os.makedirs(log_dir, exist_ok=True)

    def set_level(self, level: int):

        self._level = level

        return self

    def _should_log(self, level):

        return level >= self._level
```



```

def _rotate_if_needed(self, filepath):
    """
    Simple rotation:

    If file > max_bytes:

    rename .log → .log.1
    rename .log.1 → .log.2
    ... up to backup_count

    delete oldest

    """

    if not os.path.exists(filepath):
        return

    if os.path.getsize(filepath) < self.max_bytes:
        return

    # Rotate old backups
    for i in range(self.backup_count - 1, 0, -1):
        old = f"{filepath}.{i}"
        new = f"{filepath}.{i+1}"
        if os.path.exists(old):
            if os.path.exists(new):
                os.remove(new)

            os.rename(old, new)

    # Current → .1
    if os.path.exists(filepath):
        new1 = f"{filepath}.1"
        if os.path.exists(new1):
            os.remove(new1)

        os.rename(filepath, new1)

def _write(self, level_name, message, filepath):
    with self._lock:

```

```

self._rotate_if_needed(filepath)

timestamp = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')

line = f"[{timestamp}] [{level_name}] {message}\n"

with open(filepath, 'a', encoding='utf-8') as f:

    f.write(line)

```

```

def debug(self, message):

    if self._should_log(self.DEBUG):

        path = os.path.join(self.log_dir, 'debug.log')

        self._write('DEBUG', message, path)

```

```

def info(self, message):

    if self._should_log(self.INFO):

        path = os.path.join(self.log_dir, 'access.log')

        self._write('INFO', message, path)

```

```

def warn(self, message):

    if self._should_log(self.WARN):

        path = os.path.join(self.log_dir, 'access.log')

        self._write('WARN', message, path)

```

```

def error(self, message):

    if self._should_log(self.ERROR):

        path = os.path.join(self.log_dir, 'error.log')

        self._write('ERROR', message, path)

```

```

def request(self, method, path, status=200, duration_ms=None, ip=None, user_agent=None):

    """

    Log a structured request entry.

    Format:

    [timestamp] [INFO] GET /shop 200 12ms 127.0.0.1 Chrome/...

    """

    parts = [method, path, str(status)]

```

```

if duration_ms is not None:
    parts.append(f"{duration_ms}ms")

if ip:
    parts.append(ip)

if user_agent:
    parts.append(user_agent[:80])

```

```

message = ' '.join(parts)

self.info(message)

```

Module-level singleton

```
_logger = None
```

```
def get_logger():
```

```
    global _logger
```

```
    if _logger is None:
```

```
        import settings
```

```
        log_dir = settings.LOGGING.get('LOG_DIR', os.path.join(os.path.dirname(os.path.abspath(__file__)),
'logs'))
```

```
        _logger = WebNodeLogger(log_dir)
```

```
    return _logger
```

```
=====
```

```
=== FILE: .\core\sessions.py ===
```

```
"""
```

core/sessions.py – WebNode Framework Session Management

Cookie-based session system.

- Session IDs are generated using `secrets.token_urlsafe(32)`
- Sessions are stored in memory (Python dict) with 24-hour expiry

- Thread-safe using `threading.Lock()`
- Cookie name: `wn_session`

Usage:

```
from core.sessions import get_session_id, set_session_cookie

# In RequestWrapper (http_requests_node.py): get_session_id(request)
# In _send_response (server_node.py): set_session_cookie(response, session_id)
"""

import secrets

import time

import threading

# -----
# Session Store
# -----

SESSION_STORE = {}          # { session_id: { '_created': timestamp, ...data } }
_lock = threading.RLock()  # Re-entrant – safe for nested acquisitions in same thread
SESSION_EXPIRY = 86400      # 24 hours in seconds
COOKIE_NAME = 'wn_session'

# -----
# Internal Helpers
# -----

def _cleanup_expired():
    """
    Remove sessions older than SESSION_EXPIRY.

    CONTRACT: Must be called while holding _lock (assumes lock is already acquired).
    """
    now = time.time()
```

```

expired = [
    sid for sid, data in SESSION_STORE.items()
    if now - data.get('_created', now) > SESSION_EXPIRY
]

for sid in expired:
    del SESSION_STORE[sid]

```

```

def _create_session():
    """
    Create a brand-new session and return its ID.
    Called from within get_session_id() which already holds _lock.
    Uses _lock via RLock re-entrancy – safe to call while lock is held.
    """
    session_id = secrets.token_urlsafe(32)
    with _lock: # RLock: safe even if caller already holds it
        SESSION_STORE[session_id] = {'_created': time.time()}
    return session_id

```

```

# -----
# Public API
# -----

```

```

def get_session_id(request) -> str:
    """
    Returns the session ID for this request.

```

Priority:

1. request.session_id (set by RequestWrapper from Cookie header)
2. If session_id exists in SESSION_STORE → return it
3. Otherwise create a new session and mark request._new_session = True

Signature is identical to the old IP-based `get_session_id(request)`

so all existing logic files (`cart_logic.py` etc.) work unchanged.

"""

with `_lock`:

`_cleanup_expired()`

RequestWrapper sets `request.session_id` from the Cookie header

`existing_id = getattr(request, 'session_id', None)`

if `existing_id`:

with `_lock`:

if `existing_id` in `SESSION_STORE`:

`# Refresh the session timestamp (rolling expiry)`

`SESSION_STORE[existing_id]['_created'] = time.time()`

`return existing_id`

No valid session found → create a new one

`new_id = _create_session()`

Mark on the request so `server_node.py` knows to Set-Cookie

try:

`request._new_session = True`

`request.session_id = new_id`

except (`AttributeError`, `TypeError`):

`pass # WSGI wrapper or raw handler – best effort`

`return new_id`

def `set_session_cookie(response, session_id: str)`:

"""

Adds a Set-Cookie header to a Response object.

Called from `server_node._send_response()` when `request._new_session` is True.

Cookie attributes:

HttpOnly – not accessible via JavaScript (XSS protection)

SameSite=Strict – CSRF protection

Path=/ – valid for entire site

Max-Age=86400 – 24 hours

"""

```
cookie_value = (
    f"{COOKIE_NAME}={session_id}; "
    f"HttpOnly; SameSite=Strict; Path=/; Max-Age={SESSION_EXPIRY}"
)
response.headers['Set-Cookie'] = cookie_value
```

```
def get_session_data(session_id: str) -> dict:
    """Return the data dict for a session (excluding internal keys)."""
    with _lock:
        data = SESSION_STORE.get(session_id, {})
        return {k: v for k, v in data.items() if not k.startswith('_')}
```

```
def set_session_data(session_id: str, key: str, value):
    """Store a key-value pair in the session."""
    with _lock:
        if session_id not in SESSION_STORE:
            SESSION_STORE[session_id] = {'_created': time.time()}
        SESSION_STORE[session_id][key] = value
```

```
def delete_session(session_id: str):
    """Remove a session from the store (e.g. on logout)."""
    with _lock:
        SESSION_STORE.pop(session_id, None)
```

```
=====
```

```
=== FILE: .\core\test.py ===
```

```
import time  
print(time.now())
```

```
=====
```

```
=== FILE: .\core\testing.py ===
```

```
"""
```

```
core/testing.py – Gravity-Flow Node Testing Framework
```

```
Har node ko ALAG se test karo bina poori server chalaye.
```

```
AI-Friendly Design:
```

- AI ek node ka test likh sakta hai without knowing other nodes
- MockRequest banao, node mein daalo, output check karo
- Clear pass/fail output with node name

```
Usage:
```

```
from core.testing import NodeTestCase, MockRequest, run_tests
```

```
class MyTest(NodeTestCase):
```

```
    def test_url_match(self):
```

```
        from nodes.url_node import URLNode
```

```
        node = URLNode('/users/<id>')
```

```
        req = MockRequest(path='/users/42')
```

```
        # Manually check matching
```

```
        params = node._match('/users/42')
```

```
        self.assertEqual(params, {'id': '42'}, 'Dynamic URL param extraction')
```



```

def test_logic_node(self):

    from nodes.logic_node import LogicNode

    def my_logic(req):

        return {'name': req.query_params.get('n', 'World')}

    node = LogicNode(my_logic)

    req = MockRequest(query_params={'n': 'Ali'})

    result = node.process(req)

    self.assertEqual(result, {'name': 'Ali'}, 'LogicNode output')

```

```

run_tests(MyTest)

```

```

"""

```

```

import time

```

```

import traceback

```

```

import datetime

```

```

# -----
# MockRequest - Fake request for testing nodes
# -----

```

```

class MockRequest:

```

```

    """

```

```

    A fake RequestWrapper for unit testing nodes.

```

```

    Pass it to any node's process() method without starting a server.

```

```

    Example:

```

```

    req = MockRequest(

        method='POST',

        path='/login',

        params={'username': ['ali'], 'password': ['1234']},

        query_params={'next': '/dashboard'},

```

```

        json_body={'key': 'value'},

        context={'user_id': 1},

        url_params={'id': '42'},

        headers={'Content-Type': 'application/json'},

    )

"""

```

```

class _FakeHandler:

```

```

    client_address = ('127.0.0.1', 9999)

```

```

def __init__(self,

```

```

    method='GET',

```

```

    path='/',

```

```

    params=None,

```

```

    query_params=None,

```

```

    json_body=None,

```

```

    context=None,

```

```

    url_params=None,

```

```

    headers=None):

```

```

    self.method      = method.upper()

```

```

    self.path        = path

```

```

    self.params      = params or {}

```

```

    self.query_params = query_params or {}

```

```

    self.context     = context or {}

```

```

    self.url_params  = url_params or {}

```

```

    self.headers     = MockHeaders(headers or {})

```

```

    self.body_bytes  = b''

```

```

    self.handler     = self._FakeHandler()

```

```

    self._json_body  = json_body

```

```

# Pre-encode JSON body if provided

```

```

if json_body is not None:

```

```

    import json

```

```
self.body_bytes = json.dumps(json_body).encode('utf-8')
```

```
def get_param(self, key, default=None):  
    if key in self.url_params:  
        return self.url_params[key]  
  
    v = self.params.get(key)  
  
    if isinstance(v, list):  
        return v[0] if v else default  
  
    return v if v is not None else default
```

```
def get_json(self):  
    if self._json_body is not None:  
        return self._json_body  
  
    import json  
  
    try:  
        return json.loads(self.body_bytes)  
  
    except Exception:  
        return None
```

```
def get_file(self, key):  
    return None
```

```
@property
```

```
def content_type(self):  
    return self.headers.get('Content-Type', '').lower()
```

```
def __repr__(self):  
    return f'<MockRequest {self.method} {self.path}>'
```

```
class MockHeaders(dict):
```

```
    """Dict subclass with case-insensitive .get()"""
```

```
    def get(self, key, default=None):
```

```
for k, v in self.items():

    if k.lower() == key.lower():

        return v

return default
```

```
def __contains__(self, key):

    return self.get(key) is not None
```

```
# -----
# Test Result
# -----
```

```
class TestResult:

    def __init__(self, test_name, node_name=''):

        self.test_name = test_name

        self.node_name = node_name

        self.passed = False

        self.message = ''

        self.duration = 0.0

        self.error = None

    def __repr__(self):

        status = ' PASS' if self.passed else ' FAIL'

        base = f" {status} {self.test_name}"

        if self.node_name:

            base += f" [{self.node_name}]"

        if not self.passed:

            base += f"\n          → {self.message}"

        base += f" ({self.duration*1000:.1f}ms)"

        return base
```

```
# -----
```

```
# NodeTestCase – Base class for writing node tests
```

```
# -----
```

```
class NodeTestCase:
```

```
    """
```

```
    Base class for all node test suites.
```

```
    Write test_* methods – they are auto-discovered and run.
```

```
    Example:
```

```
        class URLTests(NodeTestCase):
```

```
            def test_exact_match(self):
```

```
                node = URLNode('/home')
```

```
                req = MockRequest(path='/home')
```

```
                result = node.process(req)
```

```
                self.assert_not_none(result, 'Exact match should pass')
```

```
            def test_no_match(self):
```

```
                node = URLNode('/home')
```

```
                req = MockRequest(path='/about')
```

```
                result = node.process(req)
```

```
                self.assert_none(result, 'Wrong path should return None')
```

```
    """
```

```
def assert_equal(self, actual, expected, label=''):
```

```
    if actual != expected:
```

```
        raise AssertionError(
```

```
            f"{label}: Expected {expected!r}, got {actual!r}"
```

```
        )
```

```
def assert_not_equal(self, actual, expected, label=''):
```

```
    if actual == expected:
```

```
        raise AssertionError(f"{label}: Expected values to differ, both are {actual!r}")
```

```

def assert_true(self, expr, label=''):
    if not expr:
        raise AssertionError(f"{label}: Expected True, got {expr!r}")

def assert_false(self, expr, label=''):
    if expr:
        raise AssertionError(f"{label}: Expected False, got {expr!r}")

def assert_none(self, val, label=''):
    if val is not None:
        raise AssertionError(f"{label}: Expected None, got {val!r}")

def assert_not_none(self, val, label=''):
    if val is None:
        raise AssertionError(f"{label}: Expected not-None, got None")

def assert_contains(self, container, item, label=''):
    if item not in container:
        raise AssertionError(f"{label}: {item!r} not found in {container!r}")

def assert_status(self, response, status_code, label=''):
    """Assert a Response object has the given status code."""
    actual = getattr(response, 'status', None)
    if actual != status_code:
        raise AssertionError(
            f"{label}: Expected status {status_code}, got {actual}"
        )

def assert_json_key(self, response, key, label=''):
    """Assert a Response.json() contains a key."""
    import json
    try:

```

```

        data = json.loads(response.body)

        if key not in data:

            raise AssertionError(f"{label}: Key {key!r} missing in JSON response")

    except Exception as e:

        raise AssertionError(f"{label}: Could not parse JSON - {e}")

```

```

# -----
# Test Runner
# -----

```

```

def run_tests(*test_classes, verbose=True):

    """

    Discover and run all test_* methods in given test classes.

    Usage:

        run_tests(URLTests, LogicTests, ModelTests)

    Returns:

        dict with 'passed', 'failed', 'total', 'results'

    """

    results = []

    total    = passed = failed = 0

    print(f"\n{'-'*55}")

    print(f"    Gravity-Flow Node Test Runner")

    print(f"    {datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")

    print(f"{'-'*55}")

    for cls in test_classes:

        instance    = cls()

        class_name = cls.__name__

        methods     = [m for m in dir(instance) if m.startswith('test_')]

```

```

if verbose:

    print(f"\n    {class_name} ({len(methods)} tests)")

for method_name in sorted(methods):

    res = TestResult(

        test_name = method_name.replace('test_', '').replace('_', ' '),

        node_name = class_name,

    )

    total += 1

    t0 = time.perf_counter()

    try:

        getattr(instance, method_name)()

        res.passed = True

        passed += 1

    except AssertionError as e:

        res.passed = False

        res.message = str(e)

        failed += 1

    except Exception as e:

        res.passed = False

        res.message = f"{type(e).__name__}: {e}"

        res.error = traceback.format_exc()

        failed += 1

    finally:

        res.duration = time.perf_counter() - t0

    results.append(res)

    if verbose:

        print(res)

print(f"\n{'-'*55}")

icon = '' if failed == 0 else ''

```



```

print(f" {icon} Results: {passed}/{total} passed | {failed} failed")

print(f"{'-'*55}\n")


return {
    'total' : total,
    'passed' : passed,
    'failed' : failed,
    'results': results,
}


# -----
# Quick Test Utility (for single node ad-hoc testing)
# -----


def test_node(node, input_data, expected_type=None, label=''):
    """
    Quick one-liner to test a single node.

    Example:

    from core.testing import test_node, MockRequest
    from nodes.url_node import URLNode

    result = test_node(
        URLNode('/home'),
        MockRequest(path='/home'),
        expected_type=dict,
        label='URL match test'
    )
    """
    t0 = time.perf_counter()

    try:
        result = node.process(input_data)

```

```

ms      = (time.perf_counter() - t0) * 1000

status = ''

msg      = f"returned {type(result).__name__}"

if expected_type and not isinstance(result, expected_type):

    status = ''

    msg      = f"expected {expected_type.__name__}, got {type(result).__name__}"

    print(f" {status} {label or node._node_name} → {msg} ({ms:.1f}ms)")

    return result

except Exception as e:

    ms = (time.perf_counter() - t0) * 1000

    print(f" {label or node._node_name} → {type(e).__name__}: {e} ({ms:.1f}ms)")

    return None

```

=====

=== FILE: .\core\validators.py ===

core/validators.py

import html

import re

from nodes.response import Response

Exception

class ValidationError(Exception):

def __init__(self, field: str, message: str):

self.field = field

self.message = message

super().__init__(f"{field}: {message}")

```
# -----
```

```
# Individual Validators
```

```
# -----
```

```
def validate_int(
    value,
    field_name: str,
    min_val: int = None,
    max_val: int = None,
    required: bool = True
) -> int:
    """
    Validate and return an integer value.

    Raises ValidationError if:
    - required=True and value is None/empty
    - value cannot be converted to int
    - value < min_val
    - value > max_val

    Returns int on success.
    """
    if value is None or str(value).strip() == '':
        if required:
            raise ValidationError(field_name, f"{field_name} is required")

        return None

    try:
        result = int(str(value).strip())
    except (ValueError, TypeError):
        raise ValidationError(field_name, f"{field_name} must be a number")

    if min_val is not None and result < min_val:
```

```

        raise ValidationError(field_name, f"{field_name} must be at least {min_val}")

    if max_val is not None and result > max_val:
        raise ValidationError(field_name, f"{field_name} must be at most {max_val}")

    return result


def validate_str(
    value,
    field_name: str,
    min_length: int = 0,
    max_length: int = 255,
    required: bool = True,
    strip: bool = True
) -> str:
    """
    Validate and return a safe string.

    - Strips whitespace if strip=True
    - HTML escapes the value (XSS prevention)
    - Raises ValidationError if:
        * required=True and empty
        * length < min_length
        * length > max_length

    Returns html-escaped string on success.
    """
    if value is None:
        if required:
            raise ValidationError(field_name, f"{field_name} is required")

        return ''

```

```

value = str(value)

if strip:
    value = value.strip()

if required and len(value) == 0:
    raise ValidationError(field_name, f"{field_name} is required")

if len(value) < min_length:
    raise ValidationError(field_name, f"{field_name} must be at least {min_length} characters")

if len(value) > max_length:
    raise ValidationError(field_name, f"{field_name} must be at most {max_length} characters")

# XSS prevention – always escape
return html.escape(value)

def validate_email(
    value,
    field_name: str = 'email',
    required: bool = True
) -> str:
    """
    Validate email format.

    - Basic format check: x@x.x
    - Max length 254 chars (RFC 5321)
    - Returns lowercased, stripped, html-escaped email

    Raises ValidationError if invalid.
    """
    value = validate_str(
        value,

```

```

        field_name,
        max_length=254,
        required=required
    )

```

```

if not value and not required:

```

```

    return ''

```

```

# Basic email pattern

```

```

pattern = r'^^[^@\\s]+@[^@\\s]+\\.^[^@\\s]+$'

```

```

if not re.match(pattern, html.unescape(value)):

```

```

    raise ValidationError(field_name, "Invalid email address")

```

```

return value.lower()

```

```

# -----

```

```

# Bulk Form Validator

```

```

# -----

```

```

def validate_form(

```

```

    request,

```

```

    rules: dict

```

```

) -> tuple[dict, list]:

```

```

    """

```

```

    Validate multiple form fields at once.

```

```

    rules format:

```

```

    {

```

```

        'field_name': {

```

```

            'type': 'int' | 'str' | 'email',

```

```

            'required': True | False,

```

```

            'min': value,      (int: min_val, str: min_length)

```

```

        'max': value,          (int: max_val, str: max_length)

    }

}

```

Returns:

```

    (cleaned_data dict, errors list)

"""

cleaned = {}

errors = []

for field, rule in rules.items():

    # Handle both Mock objects and real framework requests

    value = request.get_param(field) if hasattr(request, 'get_param') else request.get(field)

    field_type = rule.get('type', 'str')

    required = rule.get('required', True)

    try:

        if field_type == 'int':

            cleaned[field] = validate_int(

                value,

                field,

                min_val=rule.get('min'),

                max_val=rule.get('max'),

                required=required

            )

        elif field_type == 'email':

            cleaned[field] = validate_email(

                value,

                field,

                required=required

            )

        else: # str default

            cleaned[field] = validate_str(

```

```

        value,

        field,

        min_length=rule.get('min', 0),

        max_length=rule.get('max', 255),

        required=required

    )

except ValidationError as e:

    errors.append(e.message)


return cleaned, errors


# -----
# Safe Helpers (no exception versions)
# -----


def safe_int(

    value,

    default: int = 0,

    min_val: int = None,

    max_val: int = None

) -> int:

    """

    Safely convert to int – returns default instead of raising.

    """

    try:

        result = int(str(value).strip())

        if min_val is not None and result < min_val:

            return default

        if max_val is not None and result > max_val:

            return default

        return result

    except (ValueError, TypeError, AttributeError):

```



```
return default
```

```
def safe_str(
    value,
    default: str = '',
    max_length: int = 255
) -> str:
    """
    Safely get string value – returns default instead of raising.
    HTML escaped for XSS prevention.
    """
    if value is None:
        return default
    try:
        result = html.escape(str(value).strip())
        if len(result) > max_length:
            return default
        return result
    except Exception:
        return default
```

```
=====
```

```
=== FILE: .\core\__init__.py ===
```

```
# core package – Database & Logging utilities
```

```
=====
```

```
=== FILE: .\nodes\base_node.py ===
```

"""

nodes/base_node.py – Gravity-Flow Framework

BaseNode : Foundation class for ALL nodes.

Key Design Principles (AI-Friendly Architecture):

1. Single Responsibility – har node ek kaam karta hai
2. Chain Pattern – nodes doubly linked list ki tarah connect hote hain
3. Error Isolation – ek node fail ho to sirf wahi crash ho, baaki nahi
4. Named Nodes – har node ka ek readable name hota hai (debug ke liye)

Graph Flow:

```
node_A.connect(node_B).connect(node_C)
```

```
node_A.process(data) → node_B.process(data) → node_C.process(data)
```

"""

```
import traceback
```

```
class BaseNode:
```

```
    """
```

```
    Base class for all nodes in the framework.
```

```
    Attributes:
```

```
        next_node : BaseNode | None – next node in the chain
```

```
        prev_node : BaseNode | None – previous node (back-reference)
```

```
        _node_name : str – human-readable name for error reporting
```

```
        _debug : bool – if True, errors are caught and reported; else re-raised
```

```
    Sub-classes should override process() and call super().process(result)
```

```
    at the end to pass data to the next node.
```

```
    """
```

```

def __init__(self):
    self.next_node = None
    self.prev_node = None

    # Auto-generated readable name: "URLNode_at_<short_id>"
    self._node_name = f"{self.__class__.__name__}_at_{hex(id(self))[-4:]}"

    self._debug = True # Catch + report errors by default

    self._fallback = None # ← ADD THIS

```

```

def set_fallback(self, handler):

```

```

    """

```

```

    Set a fallback handler for errors.

```

```

    handler signature:

```

```

        def my_fallback(request, error):
            return Response.server_error(
                "Custom error message"
            )

```

```

    Usage:

```

```

        node.set_fallback(my_handler)

```

```

        # or chained:

```

```

        URLNode('/shop')\
            .set_fallback(handler)\
            .connect(logic)

```

```

    Returns self for chaining.

```

```

    """

```

```

        self._fallback = handler

```

```

        return self

```

```

# -----

```

```

# Node Naming (for clear error messages)

```

```

# -----

```

```
def set_name(self, name):
    """
    Give this node a human-readable name for error reporting.

    Example:
        URLNode('/users').set_name('user_url')
    """
    self._node_name = name

    return self # allow chaining: URLNode('/').set_name('index').connect(...)
```

```
# -----
# Connection
# -----
```

```
def connect(self, node):
    """
    Connect this node to the next node in the graph chain.

    Returns the next node to allow fluent chaining:
        node_a.connect(node_b).connect(node_c)
    """
    self.next_node = node
    node.prev_node = self
    return node
```

```
# -----
# Processing (with error isolation)
# -----
```

```
def process(self, data):
    """
    Process data and pass to next node.

    Errors are caught and handled
    gracefully via _on_error().
```

```
"""
```

```
if self.next_node:
```

```
    try:
```

```
        return self.next_node.process(data)
```

```
    except Exception as e:
```

```
        return self._on_error(
```

```
            e, data,
```

```
            node=self.next_node
```

```
        )
```

```
    return data
```

```
def _on_error(self, error, data, node=None):
```

```
    """
```

```
    Handle error from this node
```

```
    or next_node.
```

```
    Priority:
```

```
    1. If _fallback is set → call it
```

```
    2. Otherwise → NodeErrorReporter
```

```
    3. Always return valid response
```

```
    """
```

```
    target = node or self
```

```
    # Priority 1: Custom fallback
```

```
    if self._fallback is not None:
```

```
        try:
```

```
            return self._fallback(data, error)
```

```
        except Exception as fb_err:
```

```
            # Fallback itself failed
```

```
            print(
```

```
                f"[BaseNode] Fallback "
```

```
                f"error in "
```

```
                f"{target._node_name}: "
```

```
        f"{fb_err}"
    )
```

```
# Priority 2: NodeErrorReporter
```

```
import traceback
```

```
tb_str = traceback.format_exc()
```

```
try:
```

```
    from core.errors import NodeErrorReporter
```

```
    return NodeErrorReporter.report(
```

```
        node_name=target._node_name,
```

```
        node_type=type(target).__name__,
```

```
        error=error,
```

```
        request=data if hasattr(data, 'path') else None,
```

```
        tb_str=tb_str,
```

```
    )
```

```
except Exception:
```

```
    # Absolute fallback
```

```
    print(
```

```
        f"[NodeError] "
```

```
        f"{type(target).__name__}: "
```

```
        f"{error}"
```

```
    )
```

```
    return (
```

```
        f"<h1>500 - "
```

```
        f"{type(target).__name__} "
```

```
        f"Error</h1>"
```

```
        f"<pre>{tb_str}</pre>"
```

```
    )
```

```
def safe_process(self, data):
```

```
    """
```

```
    Wrapped version of process() – always catches errors.
```

```
    Used as the entry point for the graph.
```

```

"""

try:
    return self.process(data)
except Exception as e:
    return self._handle_error(e, data, node=self)

def _handle_error(self, error, data, node=None):
    """
    Handle a node error: log it, print it, return HTML error page.
    """
    target = node or self
    tb_str = traceback.format_exc()

    # Lazy import to avoid circular imports
    try:
        from core.errors import NodeErrorReporter

        return NodeErrorReporter.report(
            node_name = target._node_name,
            node_type = type(target).__name__,
            error      = error,
            request    = data if hasattr(data, 'path') else None,
            tb_str     = tb_str,
        )
    except Exception:
        # Absolute fallback – never crash the reporter
        print(f"[NodeError] {type(target).__name__}: {error}")
        return f"<h1>500 – {type(target).__name__} Error</h1><pre>{tb_str}</pre>"

# -----
# Graph Introspection (AI-Readable)
# -----

def describe(self):

```

```
"""
```

Return a structured dict describing this node.

Used by AI agents and the Node Editor to understand graph structure.

```
"""
```

```
return {
    "name"      : self._node_name,
    "type"      : self.__class__.__name__,
    "has_next"  : self.next_node is not None,
    "has_prev"  : self.prev_node is not None,
    "next_name" : self.next_node._node_name if self.next_node else None,
}
```

```
def describe_chain(self):
```

```
    """
```

Return a list describing the entire chain from this node forward.

Useful for AI to understand the full flow.

Example output:

```
[
    {"name": "server_1", "type": "ServerNode", ...},
    {"name": "http_parser", "type": "HTTPRequestsNode", ...},
    {"name": "url_home", "type": "URLNode", ...},
    ...
]
```

```
"""
```

```
chain = []
```

```
current = self
```

```
seen = set()
```

```
while current is not None:
```

```
    nid = id(current)
```

```
    if nid in seen:
```

```
        break # Cycle guard
```

```
    seen.add(nid)
```



```
        chain.append(current.describe())

        current = current.next_node

    return chain
```

```
def __repr__(self):
    return f"<{self.__class__.__name__} name={self._node_name!r}>"
```

```
=====
```

```
=== FILE: .\nodes\context_node.py ===
```

```
from nodes.base_node import BaseNode
```

```
class ContextNode(BaseNode):
```

```
    """
```

```
    Executes a callable logic function to update the request context.
```

```
    Passes the request object to the next node.
```

```
    """
```

```
def __init__(self, context_func):
```

```
    super().__init__()
```

```
    self.context_func = context_func
```

```
def process(self, request):
```

```
    """
```

```
    Executes logic, merges result into request.context, and passes request forward.
```

```
    """
```

```
    result = self.context_func(request)
```

```
    if isinstance(result, dict):
```

```
        request.context.update(result)
```

```
    return super().process(request)
```

=====

=== FILE: .\nodes\css_node.py ===

"""

nodes/css_node.py – WebNode Framework

CSSNode: Writes CSS code directly to /static/ directory.

Used from the Node Editor to author and deploy CSS stylesheets

inline, without manually editing files.

Usage in main.py (auto-generated by compiler):

```
cssnode = CSSNode('shop.css', '''body { margin: 0; }''')  
cssnode.apply() # writes /static/shop.css at deploy time
```

"""

import os

import settings

from nodes.base_node import BaseNode

class CSSNode(BaseNode):

"""

A standalone CSS style node.

- Stores CSS code and a target filename.
- On apply(), writes the CSS to settings.STATIC_ROOT/<filename>.
- Does NOT participate in the request/response chain (no in-port).
- Can be chained to a RenderNode for auto-injection (future).

"""

```
def __init__(self, filename: str, css_code: str = ''):
```

```
    super().__init__()
```

```
    self.filename = filename
```

```
self.css_code = css_code
```

```
def apply(self):
```

```
    """Write CSS code to the static directory."""
```

```
    if not self.filename:
```

```
        return
```

```
    # Ensure filename ends with .css
```

```
    fname = self.filename if self.filename.endswith('.css') else self.filename + '.css'
```

```
    dest = os.path.join(settings.STATIC_ROOT, fname)
```

```
    os.makedirs(settings.STATIC_ROOT, exist_ok=True)
```

```
    with open(dest, 'w', encoding='utf-8') as f:
```

```
        f.write(self.css_code)
```

```
    print(f'[CSSNode] Written: {dest}')
```

```
def process(self, request):
```

```
    """CSSNode doesn't process requests – it's a static asset node."""
```

```
    return super().process(request)
```

```
def __repr__(self):
```

```
    return f'<CSSNode filename={self.filename!r}>'
```

```
=====
```

```
=== FILE: .\nodes\http_requests_node.py ===
```

```
"""
```

```
nodes/http_requests_node.py – Gravity-Flow Framework
```

```
HTTPRequestNode : Converts raw http.server handler into a clean RequestWrapper.
```

```
RequestWrapper   : Mimics a real request object with helpers for params, JSON, files.
```

```
"""
```

```
import urllib.parse
```

```

import json as _json

import io

from nodes.base_node import BaseNode


class HTTPRequestsNode(BaseNode):
    """
    Converts the raw HTTP handler → RequestWrapper.

    Graph position: ServerNode → HTTPRequestsNode → RouterNode
    """

    def __init__(self):
        super().__init__()

    def process(self, handler):
        # If already wrapped by a security node placed before us, skip re-wrapping
        if hasattr(handler, 'context'):
            return super().process(handler)

        request = RequestWrapper(handler)

        # Store back-reference so server_node can retrieve _new_session / session_id
        handler._current_request = request

        return super().process(request)


class RequestWrapper:
    """
    A clean request object built from the low-level http.server handler.

    Attributes:
        path          : str    – URL path without query string (e.g. '/users')
        method        : str    – 'GET', 'POST', 'PUT', 'PATCH', 'DELETE'
        headers       : dict-like – HTTP request headers
    """

```

```
query_params : dict - parsed URL query string (?key=value)

params       : dict - parsed form body (POST/PUT/PATCH)

context      : dict - shared context dict passed between nodes

body_bytes   : bytes - raw request body

url_params   : dict - dynamic URL params e.g. {'id': '42'} from /users/<id>
```

HTTP Method → CRUD Mapping:

```
GET    → Read    (idempotent)

POST   → Create

PUT    → Update (full replace, idempotent)

PATCH → Update (partial)

DELETE → Delete (idempotent)
```

"""

```
def __init__(self, handler):

    self.handler = handler

    parsed_url = urllib.parse.urlparse(handler.path)

    self.path = parsed_url.path

    self.headers = handler.headers

    self.method = handler.command

    # URL query string: ?n=5&foo=bar → {'n': '5', 'foo': 'bar'}
    self.query_params = {
        k: v[0] if len(v) == 1 else v
        for k, v in urllib.parse.parse_qs(parsed_url.query).items()
    }

    self.params = {}          # form-encoded body params

    self.context = {}        # shared context dict for ContextNode/ModelNode

    self.body_bytes = b""

    self.url_params = {}     # dynamic URL segments set by URLNode
```

```

# --- Session: read wn_session cookie -----
self._new_session = False      # True if no existing session found
self.session_id = None         # filled below
raw_cookie = self.headers.get('Cookie', '') or ''
for crumb in raw_cookie.split(';'):
    crumb = crumb.strip()

    if crumb.startswith('wn_session='):
        self.session_id = crumb[len('wn_session='):].strip()
        break

# session validation / creation is deferred to get_session_id()
# (called lazily by logic nodes via static/helpers.py)
# -----

# Parse body for methods that carry a payload
if self.method in ('POST', 'PUT', 'PATCH', 'DELETE'):
    self._parse_body()

# -----
# Body Parsing
# -----

def _parse_body(self):
    content_length = int(self.headers.get('Content-Length', 0) or 0)
    if content_length <= 0:
        return

    self.body_bytes = self.handler.rfile.read(content_length)
    content_type = self.headers.get('Content-Type', '')

    if 'application/json' in content_type:
        # JSON body is lazy-parsed via get_json()
        pass

    elif 'application/x-www-form-urlencoded' in content_type:
        decoded = self.body_bytes.decode('utf-8', errors='replace')

```

```

self.params = urllib.parse.parse_qs(decoded)

elif 'multipart/form-data' in content_type:

    # Multipart – manual boundary parser (cgi removed in Python 3.13)

    boundary = None

    for part in content_type.split(';'):

        part = part.strip()

        if part.startswith('boundary='):

            boundary = part[9:].strip().encode()

            break

    if boundary:

        self._files = {}

        delimiter = b'--' + boundary

        end_delimiter = delimiter + b'--'

        parts_raw = self.body_bytes.split(delimiter)

        for raw_part in parts_raw[1:]:

            if raw_part.startswith(b'--') or len(raw_part) < 4:

                continue

            # Separate headers from body

            if b'\r\n\r\n' in raw_part:

                header_sec, body_sec = raw_part.split(b'\r\n\r\n', 1)

            elif b'\n\n' in raw_part:

                header_sec, body_sec = raw_part.split(b'\n\n', 1)

            else:

                continue

            # Strip trailing boundary delimiter

            body_sec = body_sec.rstrip(b'\r\n')

            # Parse Content-Disposition

            header_text = header_sec.decode('utf-8', errors='replace')

            name = None

            filename = None

            for hline in header_text.split('\r\n'):

                if 'Content-Disposition' in hline:

```

```

        for seg in hline.split(';'):
            seg = seg.strip()
            if seg.startswith('name='):
                name = seg[5:].strip('')
            elif seg.startswith('filename='):
                filename = seg[9:].strip('')

    if name is None:
        continue

    if filename is not None:
        # File upload – store as simple object
        class _UploadedFile:
            def __init__(self, fname, data):
                self.filename = fname
                self._data = data
                self.file = io.BytesIO(data)

            def read(self):
                return self._data

        self._files[name] = _UploadedFile(filename, body_sec)
    else:
        self.params[name] = [body_sec.decode('utf-8', errors='replace')]

# -----
# Public Getters
# -----

@property
def content_type(self):
    """Returns Content-Type header value (lowercase)."""
    return self.headers.get('Content-Type', '').lower()

def get_param(self, key, default=None):
    """Get a form-body or URL-param value by key."""
    # Check url_params first (dynamic route segments)

```



```

if key in self.url_params:

    return self.url_params[key]

val_list = self.params.get(key)

if val_list:

    return val_list[0]

return default

```

```

def get_json(self):
    """
    Parse and return the JSON body as a Python dict/list.
    Returns None if body is not valid JSON.

    Example (LogicNode):

        data = request.get_json()

        name = data.get('name', '')
    """
    try:

        return _json.loads(self.body_bytes.decode('utf-8'))

    except (ValueError, UnicodeDecodeError):

        return None

```

```

def get_file(self, key):
    """
    Get an uploaded file (from multipart/form-data) by field name.
    Returns a file-like object with:

        .filename : str    - original filename

        .read()   : bytes  - file contents

    Example (LogicNode):

        upload = request.get_file('avatar')

        if upload:

            data = upload.file.read()
    """

```

```
return getattr(self, '_files', {}).get(key)
```

```
def __repr__(self):
```

```
    return f'<Request {self.method} {self.path}>'
```

```
=====
```

```
=== FILE: .\nodes\logic_node.py ===
```

```
from nodes.base_node import BaseNode
```

```
from nodes.response import Response
```

```
import sys
```

```
class LogicNode(BaseNode):
```

```
    """
```

```
    Executes a callable logic function.
```

```
    If the function returns a Response (redirect, JSON, 404, etc.),
```

```
    it short-circuits immediately – bypassing any downstream nodes.
```

```
    If it returns a dict, the dict is merged into request.context and  
    processing continues to the next node.
```

```
    """
```

```
    def __init__(self, logic_func):
```

```
        super().__init__()
```

```
        self.logic_func = logic_func
```

```
    def process(self, request):
```

```
        """
```

```
        Executes business logic.
```

```
        - Response object → return immediately (redirect / JSON / error)
```

```
        - dict → merge into request.context, continue chain
```

```
        - anything else → continue chain unchanged
```

```
        """
```

```

result = self.logic_func(request)

# Short-circuit: Response objects bypass the rest of the chain
if isinstance(result, Response):
    return result

if isinstance(result, dict):
    request.context.update(result)

return super().process(request)

```

```

=====

```

```

=== FILE: .\nodes\model_node.py ===

```

```

from nodes.base_node import BaseNode

from core.db import Database

```

```

class ModelNode(BaseNode):
    """
    Model Component of MVC.

    Interacts with the Database.
    """
    def __init__(self, query, params_mapping=None, context_key='data', is_write=False):
        super().__init__()
        self.query = query
        self.params_mapping = params_mapping or [] # List of param keys to fetch from request
        self.context_key = context_key
        self.is_write = is_write
        self.db = Database()

    def process(self, request):
        """

```

Executes the query and stores result in request.context (if read).

Now supports BULK insert if params resolve to a list of lists.

"""

1. Prepare Parameters

query_params = []

is_bulk = False

if self.params_mapping:

Check if the FIRST param maps to a list (Bulk Operation Mode)

This is a simple heuristic: if params_mapping has 1 key and that key holds a list of tuples/lists.

first_key = self.params_mapping[0]

val = request.context.get(first_key)

if len(self.params_mapping) == 1 and isinstance(val, list):

BULK MODE: The context variable IS the list of rows

query_params = val

is_bulk = True

else:

STANDARD MODE: Fetch each param

for key in self.params_mapping:

val = request.get_param(key)

if val is None:

val = request.context.get(key)

query_params.append(val)

2. Execute Query

if self.is_write:

try:

if is_bulk:

self.db.executemany(self.query, query_params)

request.context[f'{self.context_key}_count'] = len(query_params)

else:

self.db.execute(self.query, tuple(query_params))

```

        # Optional: Store success flag
        request.context[f'{self.context_key}_success'] = True
    except Exception as e:
        request.context['error'] = str(e)
else:
    results = self.db.fetchall(self.query, tuple(query_params))
    # Store in context
    request.context[self.context_key] = results

    return super().process(request)

```

=====

=== FILE: .\nodes\response.py ===

"""

nodes/response.py – Gravity-Flow Response Object

Har node ab plain string ki jagah Response return kar sakta hai.

Isse status code, content-type, aur custom headers control mein aate hain.

Usage examples:

```

    return Response('<h1>Hello</h1>')
    return Response.json({'status': 'ok'})
    return Response.redirect('/login')
    return Response.not_found('User not found')
    return Response.forbidden('Access denied')
"""

import json as _json
import html as _html

```

```
class Response:
```

```
    """
```

```
    A structured HTTP response object.
```

```
    Can be returned from any node (LogicNode, ContextNode, etc.)
```

```
    FrameworkHandler automatically detects and serializes it.
```

```
    """
```

```
def __init__(self, body='', status=200, content_type='text/html; charset=utf-8', headers=None):
```

```
    """
```

```
    Args:
```

```
        body          : str or bytes – response body
```

```
        status        : int          – HTTP status code (200, 201, 302, 404 ...)
```

```
        content_type  : str          – MIME type header value
```

```
        headers       : dict         – extra headers to send, e.g. {'Location': '/foo'}
```

```
    """
```

```
    self.body = body
```

```
    self.status = status
```

```
    self.content_type = content_type
```

```
    self.headers = headers or {}
```

```
# -----
```

```
# Factory helpers
```

```
# -----
```

```
@staticmethod
```

```
def json(data, status=200):
```

```
    """Return a JSON response.
```

```
    Example:
```

```
        return Response.json({'users': []})
```

```
    """
```

```
    body = _json.dumps(data, ensure_ascii=False, indent=2)
```

```
    return Response(body=body, status=status, content_type='application/json; charset=utf-8')
```

```

@staticmethod
def redirect(url, status=302):
    """Return an HTTP redirect.

    Example:

        return Response.redirect('/dashboard')

    """
    body = f'<html><body>Redirecting to <a href="{url}">{_html.escape(url)}</a></body></html>'
    return Response(body=body, status=status, content_type='text/html; charset=utf-8',
                    headers={'Location': url})

```

```

@staticmethod
def not_found(message='404 Not Found'):
    """Return a 404 response."""
    body = (
        f'<!DOCTYPE html><html><head><title>404 Not Found</title>'
        f'<style>body{{font-family:sans-serif;text-align:center;padding:60px;'
        f'background:#0f172a;color:#e2e8f0}}'
        f'h1{{color:#f87171;font-size:4rem;margin:0}}p{{color:#94a3b8}}</style></head>'
        f'<body><h1>404</h1><p>{_html.escape(message)}</p></body></html>'
    )
    return Response(body=body, status=404)

```

```

@staticmethod
def forbidden(message='403 Forbidden'):
    """Return a 403 response."""
    body = (
        f'<!DOCTYPE html><html><head><title>403 Forbidden</title>'
        f'<style>body{{font-family:sans-serif;text-align:center;padding:60px;'
        f'background:#0f172a;color:#e2e8f0}}'
        f'h1{{color:#fb923c;font-size:4rem;margin:0}}p{{color:#94a3b8}}</style></head>'
        f'<body><h1>403</h1><p>{_html.escape(message)}</p></body></html>'
    )
    return Response(body=body, status=403)

```

```

@staticmethod
def server_error(message='500 Internal Server Error'):
    """Return a 500 response."""
    body = (
        f'<!DOCTYPE html><html><head><title>500 Server Error</title>'
        f'<style>body{{font-family:sans-serif;text-align:center;padding:60px;'
        f'background:#0f172a;color:#e2e8f0}}'
        f'h1{{color:#f43f5e;font-size:4rem;margin:0}}p{{color:#94a3b8}}</style></head>'
        f'<body><h1>500</h1><p>{_html.escape(message)}</p></body></html>'
    )
    return Response(body=body, status=500)

# -----
# Helpers
# -----

def to_bytes(self):
    """Convert body to bytes for writing to socket."""
    if isinstance(self.body, bytes):
        return self.body
    return self.body.encode('utf-8')

def __repr__(self):
    return f'<Response status={self.status} content_type={self.content_type!r}>'

class StreamingResponse(Response):
    """
    A response that streams data to the browser progressively via Server-Sent Events (SSE).
    """
    _active_streams = {}

    def __init__(self, generator, headers=None):

```



```

super().__init__(body='', status=200, content_type='text/html; charset=utf-8', headers=headers)

self.generator = generator

self.is_stream = True

self.stream_id = f"gfs_{id(self)}"

StreamingResponse._active_streams[self.stream_id] = self.generator

```

```

def __str__(self):

    # Native proxy into HTML templates. Instantiates a background SSE EventSource JS socket!

    return (

        f"<div id='{self.stream_id}' style='display:contents;'></div>"

        f"<script>"

        f"new EventSource('/__gf_stream__?id={self.stream_id}').onmessage = function(e){{"

        f"  document.getElementById('{self.stream_id}').innerHTML = JSON.parse(e.data);"

        f"}};"

        f"</script>"

    )

```

```

def __repr__(self):

    return f'<StreamingResponse status={self.status}>'

```

=====

=== FILE: .\nodes\route_node.py ===

```

from nodes.base_node import BaseNode

```

```

class RouterNode(BaseNode):

    """

    Router Node that manages multiple route branches.

    It iterates through a list of route chains and executes the first one that matches.

    """

    def __init__(self, routes):

```

```
super().__init__()

self.routes = routes
```

```
def process(self, request):

    for route in self.routes:

        # route is expected to be a URLNode (start of a chain)

        result = route.process(request)

        if result is not None:

            return result

    return None
```

```
=====
```

```
=== FILE: .\nodes\server_node.py ===
```

```
"""
```

```
nodes/server_node.py – Gravity-Flow Framework
```

```
ServerNode : Root graph node. Holds host/port config.
```

```
FrameworkHandler : Actual HTTP handler. Runs the node graph, applies  
                    middleware chain, and serializes Response objects.
```

```
"""
```

```
import http.server

import importlib

import os

import settings

from nodes.base_node import BaseNode


import mimetypes

mimetypes.init()

mimetypes.add_type('application/javascript', '.js')

mimetypes.add_type('text/css', '.css')

mimetypes.add_type('text/html', '.html')
```

```
from nodes.response import Response
```

```
from core.sessions import set_session_cookie
```

```
# -----
```

```
# Utility: import_string
```

```
# -----
```

```
def import_string(dotted_path):
```

```
    """Import a class from a dotted module path string."""
```

```
    try:
```

```
        module_path, class_name = dotted_path.rsplit('.', 1)
```

```
        module = importlib.import_module(module_path)
```

```
        return getattr(module, class_name)
```

```
    except (ValueError, ImportError, AttributeError) as e:
```

```
        print(f"[Import Error] {dotted_path}: {e}")
```

```
        return None
```

```
# -----
```

```
# ServerNode
```

```
# -----
```

```
class ServerNode(BaseNode):
```

```
    """
```

```
    Root Node – configures host/port and initiates the request graph.
```

```
    Graph Flow:
```

```
        ServerNode → HTTPRequestsNode → RouterNode → URLNode → ... → RenderNode
```

```
    """
```

```
    def __init__(self, host='127.0.0.1', port=8000):
```

```
        super().__init__()
```

```

        self.host = host

        self.port = port

def start_flow(self, handler):

    """Triggered by FrameworkHandler – passes raw handler into the graph."""

    return self.process(handler)


# -----
# FrameworkHandler
# -----


class FrameworkHandler(http.server.SimpleHTTPRequestHandler):

    """

    HTTP Handler injected into Python's http.server.

    Responsibilities:

    1. Build middleware chain (once, lazily cached)

    2. Run node graph for every request

    3. Serialize Response objects (or plain strings) to HTTP wire format

    """

    server_node = None          # Injected from main.py

    _middleware_chain = None    # Lazily built once

    protocol_version = "HTTP/1.1" # Required for Transfer-Encoding: chunked


# -----
# Middleware Chain (lazy build)
# -----


@classmethod
def _build_middleware(cls):

    """

    Read settings.MIDDLEWARE list, instantiate each class with a

```

get_response callable, and return the outermost middleware callable.

This is called once the first time a request comes in.

```
"""
```

```
middleware_classes = []
```

```
for path in getattr(settings, 'MIDDLEWARE', []):
```

```
    klass = import_string(path)
```

```
    if klass:
```

```
        middleware_classes.append(klass)
```

```
# Inner-most handler – just a no-op placeholder (graph handles real work)
```

```
def _inner(request):
```

```
    return None
```

```
# Wrap from inside out
```

```
handler = _inner
```

```
for klass in reversed(middleware_classes):
```

```
    try:
```

```
        handler = klass(handler)
```

```
    except Exception as e:
```

```
        print(f"[Middleware Warning] Could not load {klass}: {e}")
```

```
cls._middleware_chain = handler
```

```
return handler
```

```
# -----
```

```
# Core Request Dispatcher
```

```
# -----
```

```
def handle_graph_request(self, method):
```

```
    """
```

```
    Unified handler for all HTTP methods.
```

```
    1. Build middleware chain (once)
```

```
    2. Run middleware → node graph
```

3. Serialize Response or string to HTTP

"""

0. Intercept Server-Sent Events background stream

```
if self.path.startswith('/__gf_stream__'):
```

```
    self._handle_sse_stream()
```

```
    return
```

Lazy-build middleware chain

```
if FrameworkHandler._middleware_chain is None:
```

```
    FrameworkHandler._build_middleware()
```

```
if not self.server_node:
```

```
    self._send_response(Response.server_error("ServerNode not configured."))
```

```
    return
```

Run middleware (logging, security headers, etc.)

```
try:
```

```
    FrameworkHandler._middleware_chain(self)
```

```
except Exception:
```

```
    pass # Middleware errors are non-fatal; graph still runs
```

Run the node graph

```
try:
```

```
    result = self.server_node.start_flow(self)
```

```
except Exception as e:
```

```
    print(f"[Graph Error] {e}")
```

```
    self._send_response(Response.server_error(str(e)))
```

```
    return
```

Serialize result

```
if result is None:
```

```
    # No route matched → check if it's a static file
```

```
    if self.path.startswith(settings.STATIC_URL):
```

```
        self._send_response(self._serve_static(self.path))
```

```

        else:

            self._send_response(Response.not_found(f"No route matched: {self.path}"))

elif isinstance(result, Response):

    self._send_response(result)

elif isinstance(result, str):

    # Backward-compatible: plain string → 200 HTML

    self._send_response(Response(body=result, status=200))

else:

    # Anything else → try JSON serialize

    import json

    try:

        self._send_response(Response.json(result))

    except Exception:

        self._send_response(Response(body=str(result)))

# Allowed static file extensions and their MIME types (whitelist only)

_STATIC_MIME_TYPES = {

    '.css' : 'text/css',

    '.js'  : 'application/javascript',

    '.png' : 'image/png',

    '.jpg' : 'image/jpeg',

    '.jpeg' : 'image/jpeg',

    '.svg' : 'image/svg+xml',

    '.ico' : 'image/x-icon',

    '.woff' : 'font/woff',

    '.woff2' : 'font/woff2',

    '.ttf' : 'font/ttf',

    '.webp' : 'image/webp',

    '.gif' : 'image/gif',

}

def _serve_static(self, path: str) -> Response:

    """

```

Securely serve a file from settings.STATIC_ROOT.

Security model (8 layers):

1. Strip STATIC_URL prefix to get relative path
2. `os.path.normpath()` to collapse `'..'` sequences
3. Block any path that starts with `'..'` or `'/'` after normalization
4. Build absolute full path from STATIC_ROOT
5. JAIL CHECK: `abs(full_path)` must be inside `abs(STATIC_ROOT)`
6. File must exist and be a regular file
7. Extension must be in `_STATIC_MIME_TYPES` whitelist
8. Read and return file bytes with correct MIME type

Always returns a Response – never raises.

```
"""
```

```
try:
```

```
    static_prefix = settings.STATIC_URL # e.g. '/static/'
```

```
    # Step 1: Strip the URL prefix
```

```
    rel_path = path[len(static_prefix):]
```

```
    # Strip query string if present
```

```
    if '?' in rel_path:
```

```
        rel_path = rel_path.split('?')[0]
```

```
    # Step 2: Normalize (collapses ../.. etc.)
```

```
    safe_path = os.path.normpath(rel_path)
```

```
    # Step 3: Block obvious traversal attempts
```

```
    if safe_path.startswith('..') or safe_path.startswith('/'): 
```

```
        if settings.DEBUG:
```

```
            print(f"[Static] Traversal blocked: {path!r}")
```

```
        return Response.forbidden('Access denied.')
```



```

# Step 4: Build the candidate full path

full_path = os.path.join(settings.STATIC_ROOT, safe_path)


# Step 5: Jail check – must stay inside STATIC_ROOT

abs_static = os.path.abspath(settings.STATIC_ROOT)

abs_full = os.path.abspath(full_path)

if not abs_full.startswith(abs_static + os.sep) and abs_full != abs_static:

    if settings.DEBUG:

        print(f"[Static] Jail escape blocked: {abs_full!r}")

    return Response.forbidden('Access denied.')


# Step 6: MIME type from whitelist only – checked BEFORE file existence

# (so disallowed extensions always return 403, even if the file is missing,

# which also prevents probing whether sensitive files exist)

ext = os.path.splitext(full_path)[1].lower()

content_type = self._STATIC_MIME_TYPES.get(ext)

if content_type is None:

    if settings.DEBUG:

        print(f"[Static] Blocked non-whitelisted extension {ext!r}: {path!r}")

    return Response.forbidden(f'File type not allowed: {ext}')


# Step 7: Must exist and be a regular file (not a dir, symlink to dir, etc.)

if not os.path.isfile(full_path):

    return Response.not_found(f'Static file not found: {safe_path}')


# Step 8: Read and serve

with open(full_path, 'rb') as f:

    body = f.read()


return Response(body=body, content_type=content_type, status=200)


except Exception as e:

    if settings.DEBUG:

```

```

        print(f"[Static] Unexpected error serving {path!r}: {e}")

        return Response.server_error('Error serving static file.')

def _send_response(self, response: Response):
    """Write a Response object to the HTTP socket."""

    # Attach session cookie when a new session was created this request.

    # HTTPRequestsNode.process() stores the RequestWrapper on self._current_request.
    req = getattr(self, '_current_request', None)

    if getattr(req, '_new_session', False):
        sid = getattr(req, 'session_id', None)

        if sid:
            set_session_cookie(response, sid)

            req._new_session = False # only emit cookie once per response

    body_bytes = response.to_bytes()
    self.send_response(response.status)
    self.send_header('Content-Type', response.content_type)
    self.send_header('Content-Length', str(len(body_bytes)))
    # Custom headers (e.g. Location for redirects, Set-Cookie)
    for key, value in response.headers.items():
        self.send_header(key, value)

    # Security headers (always applied)
    self.send_header('X-Content-Type-Options', 'nosniff')
    self.send_header('X-Frame-Options', 'SAMEORIGIN')
    self.end_headers()
    self.wfile.write(body_bytes)

def _handle_sse_stream(self):
    """Streams background data over a persistent SSE connection."""

    from nodes.response import StreamingResponse

    import urllib.parse

    import json

    query = urllib.parse.parse_qs(urllib.parse.urlparse(self.path).query)

```

```
stream_id = query.get('id', [None])[0]
```

```
if not stream_id or stream_id not in StreamingResponse._active_streams:
```

```
    self.send_response(404)
```

```
    self.end_headers()
```

```
    return
```

```
gen = StreamingResponse._active_streams[stream_id]
```

```
self.send_response(200)
```

```
self.send_header('Content-Type', 'text/event-stream')
```

```
self.send_header('Cache-Control', 'no-cache')
```

```
self.send_header('Connection', 'keep-alive')
```

```
self.send_header('Access-Control-Allow-Origin', '*')
```

```
self.end_headers()
```

```
try:
```

```
    for item in gen:
```

```
        # SSE data must be a single string (no embedded raw newlines breaking format)
```

```
        data_str = json.dumps(str(item))
```

```
        # EventSource requires "data: <string>\n\n"
```

```
        self.wfile.write(f"data: {data_str}\n\n".encode('utf-8'))
```

```
        self.wfile.flush()
```

```
except Exception as e:
```

```
    if settings.DEBUG:
```

```
        print(f"[SSE Error] {e}")
```

```
finally:
```

```
    if stream_id in StreamingResponse._active_streams:
```

```
        del StreamingResponse._active_streams[stream_id]
```

```
# -----
```

```
# HTTP Method Handlers
```

```
# -----
```

```

def do_GET(self):    self.handle_graph_request('GET')
def do_POST(self):  self.handle_graph_request('POST')
def do_PUT(self):   self.handle_graph_request('PUT')
def do_PATCH(self): self.handle_graph_request('PATCH')
def do_DELETE(self): self.handle_graph_request('DELETE')

# Suppress default request logs (our middleware handles logging)
def log_message(self, fmt, *args):
    pass

```

=====

=== FILE: .\nodes\template_node.py ===

"""

nodes/template_node.py – Gravity-Flow Framework

RenderNode : Template rendering node. Reads HTML files and injects context.

Template Syntax (in HTML files):

```

{{ var }}          – variable substitution (HTML-escaped)
{{ var | safe }}   – variable substitution (raw/unescaped)
{% for x in items %} – loop block
{% endfor %}
{% if condition %}  – conditional block (evaluates Python expression)
{% elif condition %}
{% else %}
{% endif %}
{% extends "base.html" %} – template inheritance (first line of child)
{% block name %}      – define/override a named block
{% endblock %}
{% include "partial.html" %} – include another template

```

Backward compatible: {key} style (no spaces) still works.

```
"""

import os

import re

import ast

import html

import settings

from nodes.base_node import BaseNode


# -----
# Mini Template Engine
# -----


class TemplateEngine:
    """
    Processes Gravity-Flow template syntax.

    Single-pass renderer: inheritance → includes → tags → variables.
    """

    BLOCK_RE = re.compile(r'\{%s*block\s+(\w+)\s*%\}(.*)\{%s*endblock\s*%\}', re.DOTALL)

    EXTENDS_RE = re.compile(r'^\s*\{%s*extends\s+["\']([^\s\']+)["\']\s*%\}')

    INCLUDE_RE = re.compile(r'\{%s*include\s+["\']([^\s\']+)["\']\s*%\}')

    VAR_RE = re.compile(r'\{\{\s*(.+?)\s*\}\}')

    TAG_RE = re.compile(r'\{%s*(.+?)\s*%\}')

    # Legacy {key} style (no spaces) – handled last for backward compat

    LEGACY_RE = re.compile(r'\{(\w+)\}')

    def __init__(self, template_dir):

        self.template_dir = template_dir
```

```

def _read_file(self, name):
    path = os.path.join(self.template_dir, name)
    try:
        with open(path, 'r', encoding='utf-8') as f:
            return f.read()
    except FileNotFoundError:
        return f'<!-- Template not found: {html.escape(name)} -->'

# -----
# Safe Expression Evaluator
# -----

# Whitelist of AST node types safe for template expressions
_SAFE_NODES = (
    ast.Expression,
    # Operators
    ast.BoolOp, ast.BinOp, ast.UnaryOp,
    # Comparisons
    ast.Compare, ast.Subscript,
    # Names & literals
    ast.Attribute, ast.Name, ast.Constant,
    # Collections
    ast.List, ast.Dict, ast.Tuple, ast.Set,
    # Boolean operators
    ast.And, ast.Or, ast.Not,
    # Comparison operators
    ast.Eq, ast.NotEq, ast.Lt, ast.LtE,
    ast.Gt, ast.GtE, ast.In, ast.NotIn,
    # Arithmetic operators
    ast.Add, ast.Sub, ast.Mult, ast.Div, ast.Mod, ast.USub,
    # Misc
    ast.Index, ast.Load, ast.IfExp,
)

```

```
def _safe_eval(self, expr: str, context: dict) -> str:
```

```
    """
```

Safely evaluate a template expression.

Security model:

1. Immediate block if '___' appears anywhere (dunder attack prevention)
2. AST whitelist: only explicitly allowed node types are permitted
3. eval() run with empty __builtins__ as final safety net

Returns '' for any blocked or errored expression.

```
    """
```

```
# Rule 1: Block ALL dunder access (___class___, ___import___, etc.)
```

```
if '___' in expr:
```

```
    if settings.DEBUG:
```

```
        print(f"[Template] Blocked dunder expression: {expr!r}")
```

```
    return ''
```

```
# Rule 2: Parse and walk AST – only whitelisted node types allowed
```

```
try:
```

```
    tree = ast.parse(expr, mode='eval')
```

```
except SyntaxError:
```

```
    return ''
```

```
for node in ast.walk(tree):
```

```
    if not isinstance(node, self._SAFE_NODES):
```

```
        if settings.DEBUG:
```

```
            print(
```

```
                f"[Template] Blocked unsafe AST node "
```

```
                f"{type(node).__name__!r} in expression: {expr!r}"
```

```
            )
```

```
    return ''
```

```

# Rule 3: Compile + eval with empty builtins (belt-and-suspenders)

try:
    result = eval(
        compile(tree, '<template>', 'eval'),
        {'__builtins__': {}},
        context,
    )
    return str(result)
except Exception:
    return ''

def render(self, template_name, context=None):
    if context is None:
        context = {}

    source = self._read_file(template_name)
    rendered_html = self._process(source, context, template_name)
    return self._inject_pyscript(rendered_html)

def _inject_pyscript(self, html_content):
    # Auto-detect Python-First frontend (PyScript)

    # Scan for <script type="py">, <py-script>, py-click, etc.

    if not re.search(r'<script\s+type=[\']*py[\']*|<py-script>|py-[a-z]+=|<py-env>', html_content,
re.IGNORECASE):
        return html_content

    pyscript_tags = (
        '\n    <!-- PyScript Auto-Injected by Gravity-Flow -->\n'
        '    <link rel="stylesheet" href="https://pyscript.net/releases/2024.1.1/core.css" />\n'
        '    <script type="module" src="https://pyscript.net/releases/2024.1.1/core.js"></script>\n'
    )

    # Inject right before </head>

    if re.search(r'</head>', html_content, re.IGNORECASE):
        return re.sub(r'(?i)</head>', f'{pyscript_tags}</head>', html_content, count=1)

```



```
return pyscript_tags + html_content
```

```
def _process(self, source, context, template_name=''):
```

```
    # 1. Template inheritance
```

```
    ext_match = self.EXTENDS_RE.match(source)
```

```
    if ext_match:
```

```
        parent_name = ext_match.group(1)
```

```
        parent_src = self._read_file(parent_name)
```

```
        # Extract child blocks
```

```
        child_blocks = {}
```

```
        for m in self.BLOCK_RE.finditer(source):
```

```
            child_blocks[m.group(1)] = m.group(2)
```

```
        # Fill parent blocks with child overrides
```

```
        def replace_block(m):
```

```
            name = m.group(1)
```

```
            default = m.group(2)
```

```
            return child_blocks.get(name, default)
```

```
        source = self.BLOCK_RE.sub(replace_block, parent_src)
```

```
    # 2. Includes
```

```
    def replace_include(m):
```

```
        inc_name = m.group(1)
```

```
        inc_src = self._read_file(inc_name)
```

```
        return self._process(inc_src, context)
```

```
    source = self.INCLUDE_RE.sub(replace_include, source)
```

```
    # 3. Control flow tags – for / if, and variable substitution inside text nodes
```

```
    source = self._process_tags(source, context)
```

```
    return source
```

```
def _process_vars(self, text, context):
```

```

import re, html

def replace_var(m):
    expr = m.group(1)

    safe = False

    if expr.endswith('| safe') or expr.endswith('|safe'):
        expr = re.sub(r'\|s*safe$', '', expr).strip()

        safe = True

    value = self._safe_eval(expr, context)

    return value if safe else html.escape(value)

def replace_legacy(m):
    key = m.group(1)

    val = context.get(key)

    if val is not None:
        return html.escape(str(val)) if isinstance(val, str) else str(val)

    return m.group(0)

text = self.VAR_RE.sub(replace_var, text)

return self.LEGACY_RE.sub(replace_legacy, text)

def _process_tags(self, source, context):
    """
    Process {% for %}, {% if %}, {% elif %}, {% else %}, {% endif %},
    {% endfor %} – recursive descent tokenizer.
    """

    # Parse source into (type, value) parts directly using finditer
    # This avoids the split/re-join bug where {% %} delimiters are lost.

    parts = []

    pos = 0

    for m in self.TAG_RE.finditer(source):
        if m.start() > pos:
            parts.append(('text', source[pos:m.start()]))

        parts.append(('tag', m.group(1).strip()))

```

```

        pos = m.end()

    if pos < len(source):

        parts.append(('text', source[pos:]))

    return ''.join(self._run_parts(parts, context))

def _eval_tokens(self, tokens, context, start=0):
    """
    Walk tokens linearly; handle for/if blocks via recursion.
    tokens alternates: [text, tag, text, tag, ...]
    """
    result = []
    i = start
    tag_re = self.TAG_RE

    # Re-split into (text, tag) pairs for easier processing
    parts = []
    full = ''.join(tokens)
    pos = 0
    for m in self.TAG_RE.finditer(full):
        parts.append(('text', full[pos:m.start()]))
        parts.append(('tag', m.group(1).strip()))
        pos = m.end()
    parts.append(('text', full[pos:]))

    return self._run_parts(parts, context)

def _run_parts(self, parts, context, idx=0):
    result = []
    while idx < len(parts):
        kind, value = parts[idx]
        if kind == 'text':

```

```

        result.append(self._process_vars(value, context))

        idx += 1

elif kind == 'tag':

    tag = value

    if tag.startswith('for ') and ' in ' in tag:

        # {% for item in collection %}

        loop_expr = tag[4:]    # 'item in collection'

        var_name, iter_expr = [x.strip() for x in loop_expr.split(' in ', 1)]

        try:

            if '__' in iter_expr:

                collection = []

            else:

                collection = eval(iter_expr, {'__builtins__': {}}, context)

        except Exception:

            collection = []

        # Collect body until endfor

        body_parts, idx = self._collect_until(parts, idx + 1, 'endfor')

        for item in collection:

            loop_ctx = dict(context)

            loop_ctx[var_name] = item

            result.append(''.join(self._run_parts(body_parts, loop_ctx)))

    elif tag.startswith('if '):

        condition = tag[3:].strip()

        body_parts, else_parts, idx = self._collect_if(parts, idx + 1)

        try:

            if '__' in condition:

                test = False

            else:

                test = bool(eval(condition, {'__builtins__': {}}, context))

```

```

except Exception:

    test = False

if test:

    result.append(''.join(self._run_parts(body_parts, context)))

elif else_parts:

    result.append(''.join(self._run_parts(else_parts, context)))

elif tag in ('endfor', 'endif', 'endblock', 'else') or tag.startswith('elif '):

    # These are handled by their parent collectors

    break

elif tag.startswith('block '):

    block_name = tag[6:].strip()

    try:

        body_parts, idx = \

            self._collect_until(

                parts, idx + 1,

                'endblock'

            )

        result.append(

            ''.join(

                self._run_parts(

                    body_parts, context

                )

            )

        )

    except Exception as e:

        # Log the error

        if settings.DEBUG:

            print(

                f"[Template] Block error "

                f"in '{block_name}': {e}"

            )

```

```

        # Show HTML comment in DEBUG

        # Silent in production

        if settings.DEBUG:

            result.append(

                f'<!-- Block Error '

                f'[{block_name}]: '

                f'{html.escape(str(e))} -->'

            )

        # Always advance idx safely

        # to prevent infinite loop

        try:

            _, idx = self._collect_until(

                parts, idx + 1,

                'endblock'

            )

        except Exception:

            idx = len(parts)

    else:

        idx += 1 # Unknown tag – skip

    else:

        idx += 1

    return result

```

```

def _collect_until(self, parts, start, end_tag):

    """Collect parts until a specific end tag. Returns (body_parts, next_idx)."""

    depth = 0

    body = []

    i = start

    opener = end_tag.replace('end', '') # e.g. 'for' from 'endfor'

    while i < len(parts):

```

```

kind, value = parts[i]

if kind == 'tag':

    if value == end_tag and depth == 0:

        return body, i + 1

    if value.startswith(opener + ' '):

        depth += 1

    elif value == end_tag:

        depth -= 1

    body.append((kind, value))

    i += 1

return body, i

```

```

def _collect_if(self, parts, start):

    """Collect if body and optional else body."""

    if_body = []

    else_body = []

    in_else = False

    i = start

    depth = 0

    while i < len(parts):

        kind, value = parts[i]

        if kind == 'tag':

            if value == 'endif' and depth == 0:

                return if_body, else_body, i + 1

            if value.startswith('if '):

                depth += 1

            elif value == 'endif':

                depth -= 1

            elif value == 'else' and depth == 0:

                in_else = True

                i += 1

                continue

        if in_else:

```

```

        else_body.append((kind, value))

    else:

        if_body.append((kind, value))

    i += 1

    return if_body, else_body, i

```

```

# -----

# RenderNode

# -----

```

```

class RenderNode(BaseNode):

    """

    Template rendering node.

    Reads an HTML template, injects context from request, returns HTML string.

    """

    # Shared engine instance (reads templates from settings.TEMPLATES_DIR)

    _engine = None

    @classmethod
    def _get_engine(cls):

        if cls._engine is None:

            cls._engine = TemplateEngine(settings.TEMPLATES_DIR)

        return cls._engine

    def __init__(self, template_name):

        super().__init__()

        self.template_name = template_name

    def process(self, request):

        """

        Receive request from upstream node.

```


Extract context from request.context (or request itself if dict).

Render template and return HTML string.

"""

```
if hasattr(request, 'context'):
```

```
    context = request.context
```

```
elif isinstance(request, dict):
```

```
    context = request
```

```
else:
```

```
    context = {}
```

```
engine = self._get_engine()
```

```
return engine.render(self.template_name, context)
```

```
@staticmethod
```

```
def render(template_name, context=None):
```

"""

Static convenience method – render without a node instance.

Example:

```
html = RenderNode.render('index.html', {'title': 'Home'})
```

"""

```
engine = TemplateEngine(settings.TEMPLATES_DIR)
```

```
return engine.render(template_name, context or {})
```

=====

=== FILE: .\nodes\url_node.py ===

"""

nodes/url_node.py – Gravity-Flow Framework

URLNode : Route matching node. Supports both exact and dynamic paths.

Exact match:

```
URLNode('/about')          -> matches only '/about'
```

Dynamic segments:

```
URLNode('/users/<id>')      -> matches '/users/42', url_params = {'id': '42'}
```

```
URLNode('/files/<pk:int>') -> matches '/files/7',    url_params = {'pk': 7} (int cast)
```

```
URLNode('/p/<slug:str>')   -> matches '/p/hello',    url_params = {'slug': 'hello'}
```

Dynamic segments extracted from the URL are stored in `request.url_params` dict.

They can also be accessed via `request.get_param('id')`.

```
"""
```

```
import re
```

```
from nodes.base_node import BaseNode
```

```
# -----
```

```
# Internal: compile a URL pattern into a regex
```

```
# -----
```

```
_TYPE_PATTERNS = {  
    'int': r'(\d+)',  
    'str': r'([^/]+)',  
    '':   r'([^/]+)', # default – same as str  
}
```

```
def _compile_pattern(path_pattern):
```

```
    """
```

Convert a URL pattern string into a (compiled_regex, param_names, param_types) tuple.

Examples:

```
    '/users/<id>'          -> regex: r'^/users/([^/]+)$'   params: ['id']
```

```
    '/items/<pk:int>'      -> regex: r'^/items/(\d+)$'       params: ['pk']
```

```
    '/about'              -> regex: r'^/about$'           params: []
```

```

"""

param_names = []

param_types = []

def replace_segment(m):

    full = m.group(1)          # e.g. 'id' or 'pk:int'

    if ':' in full:

        name, type_hint = full.split(':', 1)

    else:

        name, type_hint = full, ''

    param_names.append(name.strip())

    param_types.append(type_hint.strip())

    pattern = _TYPE_PATTERNS.get(type_hint.strip(), _TYPE_PATTERNS['str'])

    return pattern

# Escape the rest of the path, then un-escape our placeholders

escaped = re.escape(path_pattern)

# re.escape turns < > into \< \> – handle both escaped and raw

escaped = escaped.replace(r'\<', '<').replace(r'\>', '>')

regex_str = re.sub(r'<([^\>]+)>', replace_segment, escaped)

regex_str = f'^{regex_str}$'

return re.compile(regex_str), param_names, param_types

# -----

# URLNode

# -----

class URLNode(BaseNode):

    """

    Route matching node in the graph.

```

```
- Exact match   : URLNode('/about')
- Dynamic      : URLNode('/users/<id>'), URLNode('/items/<pk:int>')
```

```
"""
```

```
def __init__(self, path, methods=None):
    super().__init__()
    self.path_pattern = path
    # Default: GET and POST both allowed. None means allow ALL methods
    self.allowed_methods = methods
    self._regex, self._param_names, self._param_types = _compile_pattern(path)
    self._is_exact = len(self._param_names) == 0
```

```
def _match(self, request_path):
    """
    Try to match request_path against this node's pattern.
    Returns a dict of url params on match, or None on no match.
    """
    if self._is_exact:
        return {} if request_path == self.path_pattern else None

    m = self._regex.match(request_path)
    if not m:
        return None

    url_params = {}
    for i, name in enumerate(self._param_names):
        raw = m.group(i + 1)
        type_hint = self._param_types[i]
        if type_hint == 'int':
            try:
                raw = int(raw)
            except ValueError:
                return None # Path matched pattern but value isn't int
```

```
url_params[name] = raw
```

```
return url_params
```

```
def process(self, request):
```

```
    """
```

```
    Check if request.path matches this node's pattern and if method is allowed.
```

```
    On match: populate request.url_params, pass to next node.
```

```
    No match: return None (RouterNode tries next branch).
```

```
    """
```

```
    # Step 1: Path match
```

```
    url_params = self._match(request.path)
```

```
    if url_params is None:
```

```
        return None
```

```
    # Step 2: Method check
```

```
    if self.allowed_methods is not None:
```

```
        method = request.method.upper()
```

```
        if method not in self.allowed_methods:
```

```
            from nodes.response import Response
```

```
            return Response(
```

```
                body='Method Not Allowed',
```

```
                status=405,
```

```
                headers={
```

```
                    'Allow': ', '.join(self.allowed_methods)
```

```
                }
```

```
            )
```

```
    # Step 3: Pass through
```

```
    request.url_params = url_params
```

```
    return super().process(request)
```

```
def __repr__(self):
```

```
return f'<URLNode pattern={self.path_pattern!r}>'
```

```
=====
```

```
=== FILE: .\nodes\__init__.py ===
```

```
=====
```

```
=== FILE: .\nodes\middleware\chain.py ===
```

```
"""
```

nodes/middleware/chain.py – Gravity-Flow Framework

MiddlewareChain : Reads settings.MIDDLEWARE list, instantiates each class,
and executes them in order around the node graph.

Each middleware class expects Django-style signature:

```
class MyMiddleware:

    def __init__(self, get_response):

        self.get_response = get_response

    def __call__(self, request):

        # before graph

        response = self.get_response(request)

        # after graph

        return response
```

```
"""
```

```
import importlib
```

```
import settings
```

```
def _import_string(dotted_path):
```

```
"""Import a class/function from a dotted string path."""
```

```
try:
```

```
    module_path, class_name = dotted_path.rsplit('.', 1)
```

```
    module = importlib.import_module(module_path)
```

```
    return getattr(module, class_name)
```

```
except (ValueError, ImportError, AttributeError) as e:
```

```
    print(f"[Middleware] Could not import {dotted_path!r}: {e}")
```

```
    return None
```

```
class MiddlewareChain:
```

```
    """
```

```
    Builds and executes the middleware chain defined in settings.MIDDLEWARE.
```

```
Usage in FrameworkHandler:
```

```
    chain = MiddlewareChain.build(final_handler=graph_runner)
```

```
    chain(request)
```

```
The chain executes: MW1 → MW2 → ... → final_handler → ... → MW2 → MW1
```

```
(Django-style, innermost-first wrapping).
```

```
    """
```

```
def __init__(self, handler):
```

```
    self._handler = handler
```

```
def __call__(self, request):
```

```
    return self._handler(request)
```

```
@classmethod
```

```
def build(cls, final_handler):
```

```
    """
```

```
    Build the middleware chain from settings.MIDDLEWARE.
```

Args:

`final_handler : callable(request) → response`

The innermost function (runs the node graph).

Returns:

MiddlewareChain instance wrapping all middleware.

"""

`middleware_list = getattr(settings, 'MIDDLEWARE', [])`

`handler = final_handler`

`for path in reversed(middleware_list):`

`klass = _import_string(path)`

`if klass is None:`

`continue`

`try:`

`handler = klass(handler)`

`except Exception as e:`

`print(f"[Middleware] Could not instantiate {path!r}: {e}")`

`return cls(handler)`

`@staticmethod`

`def resolve_middleware_class(path):`

`"""Utility – import and return a middleware class by its dotted path."""`

`return _import_string(path)`

=====

=== FILE: .\nodes\middleware\common.py ===

`import time`


```

class Middleware:

    def __init__(self, get_response):

        self.get_response = get_response

    def __call__(self, request):

        # Code to be executed for each request before the view (and later middleware) are called.

        response = self.get_response(request)

        # Code to be executed for each request/response after the view is called.

        return response

```

```

class SimpleLoggingMiddleware:

    """
    Logs every request to the console.
    """

    def __init__(self, get_response):

        self.get_response = get_response

    def __call__(self, request):

        start_time = time.time()

        print(f"[Middleware] Request: {request.method} {request.path}")

        response = self.get_response(request)

        duration = time.time() - start_time

        print(f"[Middleware] Response generated in {duration:.4f}s")

        return response

```

```

class SecurityMiddleware:

    """
    Adds security headers to the response.

    NOTE: In our simple implementation, response is just a string.

```

A real framework would allow modifying headers object.

For now, we will verify integration by printing.

```
"""
```

```
def __init__(self, get_response):
```

```
    self.get_response = get_response
```

```
def __call__(self, request):
```

```
    # Process request
```

```
    response = self.get_response(request)
```

```
    # In a real HttpResonse object we would do:
```

```
    # response['X-Content-Type-Options'] = 'nosniff'
```

```
    # Since we return string, we just log ensuring it ran.
```

```
    # customization of the Server handler is needed to inject headers really.
```

```
    return response
```

```
=====
```

```
=== FILE: .\node_editor\canvas.js ===
```

```
// State
```

```
let panX = 0, panY = 0, scale = 1;
```

```
let isPanning = false, startX, startY;
```

```
let activeNode = null;
```

```
let isWiring = false;
```

```
let startPort = null;
```

```
let wires = [];
```

```
let nodeIdCounter = 10; // Reserve < 10 for defaults
```

```
const canvasLayer = document.getElementById('canvas-layer');
```

```
const canvasContainer = document.getElementById('canvas-container');
```

```
const wireLayer = document.getElementById('wire-layer');
```

```

// --- Initialization ---

function init() {

    // Bind existing nodes

    document.querySelectorAll('#canvas-layer .node').forEach(node => bindNodeEvents(node));

    updateAllWires();

    // Create the floating wire disconnect button (once)

    _createWireDisconnectBtn();

    // Bind wire dblclick deletion (legacy fallback)

    wireLayer.addEventListener('dblclick', (e) => {

        if (e.target.classList.contains('wire')) {

            deleteWireByPath(e.target);

        }

    });

    // Load graph automatically

    loadGraphJSON();

}

// --- Panning & Zooming ---

canvasContainer.addEventListener('mousedown', (e) => {

    if (e.target.closest('.node') || e.target.closest('.port')) return;

    if (e.button === 1 || e.button === 0) {

        isPanning = true;

        startX = e.clientX - panX;

        startY = e.clientY - panY;

    }

});

window.addEventListener('mousemove', (e) => {

    if (isPanning) {

```

```

        panX = e.clientX - startX;
        panY = e.clientY - startY;
        updateCanvasTransform();
    }
    if (isWiring && startPort) {
        drawTempWire(e.clientX, e.clientY);
    }
});

window.addEventListener('mouseup', () => {
    isPanning = false;
    stopWiring();
});

canvasContainer.addEventListener('wheel', (e) => {
    // Only zoom when Ctrl (Windows/Linux) or Cmd (macOS) is pressed
    if (e.ctrlKey || e.metaKey) {
        e.preventDefault();

        // Zoom sensitivity: 5% per scroll step
        const zoomIntensity = 0.05;

        // Scroll up (negative deltaY) => zoom in, down => zoom out
        const delta = e.deltaY > 0 ? -1 : 1;

        let factor = Math.exp(delta * zoomIntensity);

        let newScale = scale * factor;

        newScale = Math.min(Math.max(0.2, newScale), 3);

        if (newScale === scale) return;

        // Get mouse position relative to the canvas container
        const rect = canvasContainer.getBoundingClientRect();

        const mouseX = e.clientX - rect.left;

        const mouseY = e.clientY - rect.top;

```

```

    // Compute the canvas (unscaled) coordinates of the point under the mouse

    const canvasX = (mouseX - panX) / scale;

    const canvasY = (mouseY - panY) / scale;


    // Apply the new scale

    scale = newScale;


    // Adjust pan so that the same canvas point stays under the mouse

    panX = mouseX - canvasX * scale;

    panY = mouseY - canvasY * scale;


    // Apply the transformation and redraw wires

    updateCanvasTransform();

}

});

function updateCanvasTransform() {

    canvasLayer.style.transform = `translate(${panX}px, ${panY}px) scale(${scale})`;

    canvasContainer.style.backgroundPosition = `${panX}px ${panY}px`;

    canvasContainer.style.backgroundSize = `${20 * scale}px ${20 * scale}px`;

    updateAllWires();

}


// --- Dynamic Node Event Binding ---

function bindNodeEvents(node) {

    // Bind drag on node-header

    const header = node.querySelector('.node-header');

    if (header) {

        header.addEventListener('mousedown', startNodeDrag);

    }


    node.querySelectorAll('.port').forEach(port => {

        port.addEventListener('mousedown', onPortMouseDown);

    });

}

```

```

    port.addEventListener('mouseup', onPortMouseUp);
  });

  // — Inject X delete button (if not already present) —
  if (!node.querySelector('.node-delete-btn')) {
    const btn = document.createElement('button');
    btn.className = 'node-delete-btn';
    btn.title = 'Delete node';
    btn.textContent = '';
    btn.addEventListener('mousedown', e => e.stopPropagation()); // don't start drag
    btn.addEventListener('click', (e) => {
      e.stopPropagation();
      deleteNode(node.id);
    });
    node.appendChild(btn);
  }
}

// --- Deletion Handlers ---
function deleteNode(nodeId) {
  const nodeEl = document.getElementById(nodeId);
  if (!nodeEl) return;

  wires = wires.filter(w => {
    if (w.sourceNode === nodeId || w.targetNode === nodeId) {
      w.path.remove();
      return false;
    }
    return true;
  });

  nodeEl.remove();
}

```

```

function deleteWire(wireObj) {
    if (!wireObj) return;
    wires = wires.filter(w => {
        if (w === wireObj) {
            w.path.remove();
            return false;
        }
        return true;
    });
}

```

// Keep backward compat for dblclick wire delete

```

function deleteWireByPath(pathEl) {
    const wireObj = wires.find(w => w.path === pathEl);
    deleteWire(wireObj);
}

```

// — Wire Hover Disconnect Button —————

```

let _hoveredWire = null;

```

```

function _createWireDisconnectBtn() {
    if (document.getElementById('wire-disconnect-btn')) return;
    const btn = document.createElement('button');
    btn.id = 'wire-disconnect-btn';
    btn.title = 'Disconnect wire';
    btn.textContent = '-';
    btn.addEventListener('click', () => {
        if (_hoveredWire) {
            deleteWire(_hoveredWire);
            _hoveredWire = null;
            btn.style.display = 'none';
        }
    });
}

```

```

});

// Keep visible while hovering the button itself

btn.addEventListener('mouseenter', () => { btn.style.display = 'flex'; });

btn.addEventListener('mouseleave', () => { btn.style.display = 'none'; _hoveredWire = null; });

canvasContainer.appendChild(btn);

}

```

```

function _getBezierMidpoint(x1, y1, x2, y2) {

    // Midpoint of the cubic bezier used in drawBezier()

    const curvature = Math.abs(x2 - x1) * 0.5;

    // Cubic bezier: B(0.5) for P0,P1,P2,P3 where P1=(x1+c,y1) P2=(x2-c,y2)

    const t = 0.5;

    const mt = 1 - t;

    const cx1 = x1 + curvature, cy1 = y1;

    const cx2 = x2 - curvature, cy2 = y2;

    return {

        x: mt * mt * mt * x1 + 3 * mt * mt * t * cx1 + 3 * mt * t * t * cx2 + t * t * t * x2,

        y: mt * mt * mt * y1 + 3 * mt * mt * t * cy1 + 3 * mt * t * t * cy2 + t * t * t * y2,

    };

}

```

```

function bindWireHover(wireObj) {

    const path = wireObj.path;

    path.style.pointerEvents = 'stroke';

    path.style.cursor = 'pointer';

    path.addEventListener('mouseenter', () => {

        _hoveredWire = wireObj;

        const btn = document.getElementById('wire-disconnect-btn');

        if (!btn) return;

        // Calculate midpoint in canvasContainer coordinates

        const start = getPortCoords(wireObj.fromEl);

```



```

    const end = getPortCoords(wireObj.toEl);

    const mid = _getBezierMidpoint(start.x, start.y, end.x, end.y);

    btn.style.left = `${mid.x}px`;

    btn.style.top = `${mid.y}px`;

    btn.style.display = 'flex';

  });

  path.addEventListener('mouseleave', (e) => {

    // Don't hide if moving to the button itself

    const btn = document.getElementById('wire-disconnect-btn');

    if (btn && e.relatedTarget === btn) return;

    if (btn) btn.style.display = 'none';

    _hoveredWire = null;

  });

}

// --- Drag & Drop from Palette ---

document.querySelectorAll('.palette-node').forEach(item => {

  item.addEventListener('dragstart', (e) => {

    e.dataTransfer.setData('text/plain', e.target.getAttribute('data-type'));

  });

});

canvasContainer.addEventListener('dragover', (e) => {

  e.preventDefault();

});

canvasContainer.addEventListener('drop', (e) => {

  e.preventDefault();

  const nodeType = e.dataTransfer.getData('text/plain');

  if (!nodeType) return;

```

```
const template = document.querySelector(`#node-templates [data-type="${nodeType}"]`);
if (!template) return;

const newNode = template.cloneNode(true);
const newId = `node-${nodeIdCounter++}`;
newNode.id = newId;

newNode.querySelectorAll('.port').forEach(port => {
    port.dataset.node = newId;
});

// Get mouse position relative to canvas container
const rect = canvasContainer.getBoundingClientRect();
const mouseX = e.clientX - rect.left;
const mouseY = e.clientY - rect.top;

// Convert to canvas (unscaled) coordinates using current pan and scale
const canvasX = (mouseX - panX) / scale;
const canvasY = (mouseY - panY) / scale;

// Set initial position
newNode.style.left = `${canvasX}px`;
newNode.style.top = `${canvasY}px`;

canvasLayer.appendChild(newNode);
bindNodeEvents(newNode);

// Store intended position for safety check
newNode.dataset.intendedX = canvasX;
newNode.dataset.intendedY = canvasY;

if (['LogicNode', 'ContextNode', 'RenderNode', 'CSSNode'].includes(nodeType)) {
```

```

        initMonacoEditor(newNode, nodeType);
    }

    // Safety: after a short delay, reapply position if it changed
    setTimeout(() => {
        if (newNode.parentNode) {
            const currentX = parseFloat(newNode.style.left);
            const currentY = parseFloat(newNode.style.top);

            // If node moved more than 1px, reset to intended coordinates
            if (Math.abs(currentX - canvasX) > 1 || Math.abs(currentY - canvasY) > 1) {
                newNode.style.left = `${canvasX}px`;
                newNode.style.top = `${canvasY}px`;

                updateAllWires();
            }
        }
    }, 500);
});

```

```

// --- Monaco Editor ---

```

```

function initMonacoEditor(nodeElement, type) {
    const container = nodeElement.querySelector('.monaco-container');
    if (!container) return;

    let initConfig = {
        value: '',
        language: 'python',
        theme: 'vs-dark',
        minimap: { enabled: false },
        scrollBeyondLastLine: false,
        fontSize: 12
    };

    if (type === 'LogicNode') {

```

```

    initConfig.value = `def process_logic(request):

# Write Python logic here

# e.g., request.context["result"] = "<h1>Hello</h1>"

return {}`;

} else if (type === 'ContextNode') {

    initConfig.value = `def node_logic(request):

# Add global variables here

return {"key": "value"}`;

} else if (type === 'RenderNode') {

    initConfig.value = `<!DOCTYPE html>

<html lang="en">

<head>

    <title>Document</title>

</head>

<body>

    {result}

</body>

</html>`;

    initConfig.language = 'html';

} else if (type === 'CSSNode') {

    initConfig.value = `/* CSS styles for your page */\nbody {\n    font-family: sans-serif;\n    margin: 0;\n    padding: 0;\n}\n\n.container {\n    max-width: 1200px;\n    margin: 0 auto;\n    padding: 20px;\n}`;

    initConfig.language = 'css';

}

if (window.require) {

    require(['vs/editor/editor.main'], function () {

        const editor = monaco.editor.create(container, initConfig);

        nodeElement._monacoEditor = editor;

        const resizeObserver = new ResizeObserver(() => editor.layout());

        resizeObserver.observe(container);

    });

```

```

    }
}

// --- Node Dragging (Canvas) ---

function startNodeDrag(e) {
    if (e.target.classList.contains('port') ||
        e.target.classList.contains('in-port') ||
        e.target.classList.contains('out-port')) return;

    const node = e.target.closest('.node');
    if (!node) return;
    activeNode = node;

    // Use z-index instead of appendChild to prevent Monaco editor from detaching/breaking
    let maxZ = 10;
    document.querySelectorAll('#canvas-layer .node').forEach(n => {
        let z = parseInt(n.style.zIndex || 10);
        if (z > maxZ) maxZ = z;
    });
    activeNode.style.zIndex = maxZ + 1;

    // Use node's current CSS position – works correctly at any zoom level
    const startNodeX = parseFloat(activeNode.style.left) || 0;
    const startNodeY = parseFloat(activeNode.style.top) || 0;
    const startMouseX = e.clientX;
    const startMouseY = e.clientY;

    function onMouseMove(moveEvent) {
        // Delta in screen pixels, divide by scale to get canvas-space delta
        const dx = (moveEvent.clientX - startMouseX) / scale;
        const dy = (moveEvent.clientY - startMouseY) / scale;
        activeNode.style.left = `${startNodeX + dx}px`;
        activeNode.style.top = `${startNodeY + dy}px`;
    }
}

```

```

// Only update wires connected to the active node to prevent layout thrashing
wires.forEach(wire => {
    if (wire.sourceNode === activeNode.id || wire.targetNode === activeNode.id) {
        const start = getPortCoords(wire.fromEl);
        const end = getPortCoords(wire.toEl);
        drawBezier(wire.path, start.x, start.y, end.x, end.y);
    }
});
}

function onMouseUp() {
    window.removeEventListener('mousemove', onMouseMove);
    window.removeEventListener('mouseup', onMouseUp);
    activeNode = null;
}

window.addEventListener('mousemove', onMouseMove);
window.addEventListener('mouseup', onMouseUp);
e.preventDefault();
}

// --- Wiring (SVGs) ---
function onPortMouseDown(e) {
    e.stopPropagation();
    isWiring = true;
    startPort = e.target;

    const tempPath = document.createElementNS("http://www.w3.org/2000/svg", "path");
    tempPath.classList.add('wire');
    tempPath.id = 'temp-wire';
    wireLayer.appendChild(tempPath);
}

```

```
function onPortMouseUp(e) {  
    if (isWiring && startPort && startPort !== e.target) {  
        const isStartOut = startPort.classList.contains('out-port');  
        const isEndIn = e.target.classList.contains('in-port');  
  
        if (isStartOut && isEndIn && startPort.dataset.node !== e.target.dataset.node) {  
            createWire(startPort, e.target);  
        }  
    }  
}
```

```
function stopWiring() {  
    isWiring = false;  
    startPort = null;  
    const temp = document.getElementById('temp-wire');  
    if (temp) temp.remove();  
}
```

```
function getPortCoords(portEl) {  
    const rect = portEl.getBoundingClientRect();  
    const canvasRect = canvasContainer.getBoundingClientRect();  
    return {  
        x: rect.left - canvasRect.left + (rect.width / 2),  
        y: rect.top - canvasRect.top + (rect.height / 2)  
    };  
}
```

```
function drawTempWire(mouseX, mouseY) {  
    if (!startPort) return;  
    const start = getPortCoords(startPort);  
    const canvasRect = canvasContainer.getBoundingClientRect();  
    const endX = mouseX - canvasRect.left;
```

```

const endY = mouseY - canvasRect.top;

drawBezier(document.getElementById('temp-wire'), start.x, start.y, endX, endY);
}

function createWire(fromPort, toPort) {
  const path = document.createElementNS("http://www.w3.org/2000/svg", "path");
  path.classList.add('wire');
  wireLayer.appendChild(path);

  const wireObj = {
    sourceNode: fromPort.dataset.node,
    sourcePort: fromPort.dataset.port,
    targetNode: toPort.dataset.node,
    targetPort: toPort.dataset.port,
    path: path,
    fromEl: fromPort,
    toEl: toPort
  };
  wires.push(wireObj);
  bindWireHover(wireObj); // ← hover disconnect button
  updateAllWires();
}

function updateAllWires() {
  wires.forEach(wire => {
    const start = getPortCoords(wire.fromEl);
    const end = getPortCoords(wire.toEl);
    drawBezier(wire.path, start.x, start.y, end.x, end.y);
  });
}

function drawBezier(pathEl, x1, y1, x2, y2) {

```



```

const curvature = Math.abs(x2 - x1) * 0.5;

const d = `M ${x1} ${y1} C ${x1 + curvature} ${y1}, ${x2 - curvature} ${y2}, ${x2} ${y2}`;

pathEl.setAttribute('d', d);

}

```

```

// --- Graph Serialization & Deserialization ---

```

```

function waitForMonacoEditors(

    timeout = 3000

) {

    return new Promise(resolve => {

        const start = Date.now();

        const check = setInterval(() => {

            // Check all monaco containers

            const pending = document

                .querySelectorAll(

                    '.monaco-container'

                );

            // All loaded if every container

            // either has no editor needed

            // OR has _monacoEditor attached

            const allReady = Array.from(

                pending

            ).every(el => {

                const node = el.closest(

                    '.node'

                );

                if (!node) return true;

                const type = node.dataset.type;

                const needsEditor = [

                    'LogicNode',

                    'ContextNode',

```

```

        'RenderNode',
        'CSSNode'
    ].includes(type);

    return !needsEditor ||

        !!node._monacoEditor;

    });

    if (allReady) {

        clearInterval(check);

        resolve();

        return;

    }

    // Timeout fallback

    if (Date.now() - start > timeout) {

        clearInterval(check);

        resolve(); // resolve anyway

    }

    }, 50);

    });

}

```

```

async function loadGraphJSON() {

    try {

        const res = await fetch('/api/load');

        const data = await res.json();

        if (!data?.nodes?.length) return;

        // Clear canvas

        document.querySelectorAll(

            '#canvas-layer .node'

        ).forEach(n => n.remove());

    }

}

```

```

document.querySelectorAll(
    '#wire-layer .wire'
).forEach(w => w.remove());

wires = [];

let maxId = 10;

// Step 1: Create ALL nodes first
data.nodes.forEach(nodeData => {
    const template = document
        .querySelector(
            '#node-templates ' +
            '[data-type="' +
            nodeData.type + '"'
        );
    if (!template) return;

    const newNode =
        template.cloneNode(true);
    newNode.id = nodeData.id;

    const nId = parseInt(
        nodeData.id.replace('node-', '')
    );
    if (!isNaN(nId) && nId > maxId)
        maxId = nId;

    newNode.querySelectorAll(
        '.port'
    ).forEach(port => {
        port.dataset.node = nodeData.id;
    });
});

```

```

newNode.style.left =
    `${nodeData.x}px`;
newNode.style.top =
    `${nodeData.y}px`;

canvasLayer.appendChild(newNode);
bindNodeEvents(newNode);

// Init Monaco editors
if (['LogicNode',
    'ContextNode',
    'RenderNode'].includes(
        nodeData.type
    )) {
    initMonacoEditor(
        newNode,
        nodeData.type
    );
}
});

nodeIdCounter = maxId + 1;

// Step 2: Wait for Monaco editors
await waitForMonacoEditors();

// Step 3: Load config values
// (guaranteed safe now)
data.nodes.forEach(nodeData => {
    const nodeEl = document
        .getElementById(nodeData.id);
    if (!nodeEl) return;

```

```
const config = nodeData.config

|| {});

const type = nodeData.type;

if (type === 'ServerNode') {

  if (config.ip)

    nodeEl.querySelector(

      '.ip-input'

    ).value = config.ip;

  if (config.port)

    nodeEl.querySelector(

      '.port-input'

    ).value = config.port;

} else if (type === 'URLNode') {

  if (config.path)

    nodeEl.querySelector(

      '.path-input'

    ).value = config.path;

} else if (type === 'ModelNode') {

  if (config.query)

    nodeEl.querySelector(

      '.query-input'

    ).value = config.query;

  if (config.paramsMap)

    nodeEl.querySelector(

      '.params-input'

    ).value = config.paramsMap;

  if (config.contextKey)

    nodeEl.querySelector(

      '.context-input'

    ).value = config.contextKey;

  if (config.isWrite !==

    undefined)
```

```

        nodeEl.querySelector(
            '.is-write-input'
        ).checked = config.isWrite;
    } else if (type === 'RenderNode') {
        if (config.filename)
            nodeEl.querySelector(
                '.filename-input'
            ).value = config.filename;
        if (config.html_code &&
            nodeEl._monacoEditor)
            nodeEl._monacoEditor
                .setValue(
                    config.html_code
                );
    } else if (
        type === 'LogicNode' ||
        type === 'ContextNode'
    ) {
        if (config.code &&
            nodeEl._monacoEditor)
            nodeEl._monacoEditor
                .setValue(config.code);
    } else if (type === 'CSSNode') {
        if (config.css_filename)
            nodeEl.querySelector(
                '.css-filename-input'
            ).value = config.css_filename;
        if (config.css_code &&
            nodeEl._monacoEditor)
            nodeEl._monacoEditor
                .setValue(config.css_code);
    }
});

```

```

// Step 4: Draw connections

// (guaranteed: nodes + editors ready)
data.connections.forEach(conn => {

  const sourceEl = document

    .getElementById(conn.source);

  const targetEl = document

    .getElementById(conn.target);

  if (!sourceEl || !targetEl) {

    console.warn(

      'Missing node for ' +

      'connection:',

      conn.source,

      '->',

      conn.target

    );

    return;

  }

  const fromPort = sourceEl

    .querySelector('.out-port');

  const toPort = targetEl

    .querySelector('.in-port');

  if (fromPort && toPort) {

    createWire(fromPort, toPort);

  }

});

updateAllWires();

} catch (e) {

```

```

        console.error(
            'Failed to load graph:', e
        );
    }
}

// --- Serialization & Backend API ---

function extractGraphJSON() {
    const nodes = [];

    document.querySelectorAll('#canvas-layer .node').forEach(nodeEl => {
        const id = nodeEl.id;

        const type = nodeEl.dataset.type;

        const x = parseFloat(nodeEl.style.left) || 0;

        const y = parseFloat(nodeEl.style.top) || 0;

        let config = {};

        if (type === 'ServerNode') {
            config.ip = nodeEl.querySelector('.ip-input').value;

            config.port = parseInt(nodeEl.querySelector('.port-input').value, 10);
        } else if (type === 'URLNode') {
            config.path = nodeEl.querySelector('.path-input').value;
        } else if (type === 'RenderNode') {
            config.filename = nodeEl.querySelector('.filename-input').value;

            if (nodeEl._monacoEditor) {
                config.html_code = nodeEl._monacoEditor.getValue();
            }
        } else if (type === 'ModelNode') {
            config.query = nodeEl.querySelector('.query-input').value;

            config.paramsMap = nodeEl.querySelector('.params-input').value;

            config.contextKey = nodeEl.querySelector('.context-input').value;

            config.isWrite = nodeEl.querySelector('.is-write-input').checked;
        } else if (type === 'LogicNode' || type === 'ContextNode') {
            if (nodeEl._monacoEditor) {

```



```

        config.code = nodeEl._monacoEditor.getValue();
    }
} else if (type === 'CSSNode') {
    config.css_filename = nodeEl.querySelector('.css-filename-input').value;
    if (nodeEl._monacoEditor) {
        config.css_code = nodeEl._monacoEditor.getValue();
    }
}

nodes.push({ id, type, x, y, config });
});

const connections = wires.map(w => ({
    source: w.sourceNode,
    target: w.targetNode
}));

return { nodes, connections };
}

// Bind Toolbar Buttons
document.getElementById('btn-save').addEventListener('click', async () => {
    const payload = extractGraphJSON();
    console.log("Saving graph:", payload);
    try {
        await fetch('/api/save', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });
        alert('Graph JSON saved locally!');
    } catch (e) {
        alert('Failed to save. Is gravity_backend.py running?');
    }
});

```

```

    }
  });

document.getElementById('btn-deploy').addEventListener('click', async () => {

  const payload = extractGraphJSON();

  try {

    const res = await fetch('/api/deploy', {

      method: 'POST',

      headers: { 'Content-Type': 'application/json' },

      body: JSON.stringify(payload)

    });

    const result = await res.json();

    if (result.status === 'success') {

      document.getElementById('status-indicator').innerText = '● Live';

      document.getElementById('status-indicator').className = 'live';

      alert('Deployed successfully! main.py generated and server is running.');
```

```

    } else {

      alert('Failed to deploy: ' + result.message);

    }

  } catch (e) {

    alert('Cannot connect to backend compiler.');
```

```

  });

document.getElementById('btn-stop').addEventListener('click', async () => {

  try {

    await fetch('/api/stop', { method: 'POST' });

    document.getElementById('status-indicator').innerText = '● Offline';

    document.getElementById('status-indicator').className = '';

  } catch (e) { }

});

document.getElementById('btn-clear').addEventListener('click', () => {
```

```

document.querySelectorAll('#canvas-layer .node').forEach(n => n.remove());

document.querySelectorAll('#wire-layer .wire').forEach(w => w.remove());

wires = [];

nodeIdCounter = 10;

});

// --- Polling & Flow Traversal ---

async function pollServerStatus() {
  try {
    const [statusRes, errorsRes] = await Promise.all([
      fetch('/api/status'),
      fetch('/api/errors').catch(() => null)
    ]);

    const data = await statusRes.json();
    const errorState = errorsRes ? await errorsRes.json().catch(() => ({})) : {};

    const isLive = (data.status === 'live');
    updateVisualFlow(isLive);
    updateNodeErrors(errorState);

    const indicator = document.getElementById('status-indicator');
    if (isLive) {
      indicator.innerText = '● Live';
      indicator.className = 'live';
    } else {
      indicator.innerText = '● Offline';
      indicator.className = '';
    }
  } catch (e) {
    updateVisualFlow(false);
  }
}

```

```

function updateNodeErrors(errorState) {

  // Clear all previous error states first

  document.querySelectorAll('#canvas-layer > .node.status-error').forEach(n => {

    n.classList.remove('status-error');

    const tip = n.querySelector('.__err-tip');

    if (tip) tip.remove();

  });

  // No errors

  if (!errorState || Object.keys(errorState).length === 0) return;

  // For each errored node_type, find matching nodes on canvas and highlight red

  Object.values(errorState).forEach(err => {

    const nodeType = err.node_type;

    document.querySelectorAll(`#canvas-layer > .node[data-type="${nodeType}"]`).forEach(nodeEl => {

      nodeEl.classList.add('status-error');

      // Add error tooltip if not already present

      if (!nodeEl.querySelector('.__err-tip')) {

        const tip = document.createElement('div');

        tip.className = '__err-tip';

        tip.textContent = ` ${err.error_type}: ${err.message}`;

        tip.style.cssText = `

          position:absolute; bottom:-32px; left:50%; transform:translateX(-50%);

          background:#f43f5e; color:#fff; font-size:11px; padding:3px 8px;

          border-radius:6px; white-space:nowrap; z-index:999;

          pointer-events:none; max-width:260px; overflow:hidden;

          text-overflow:ellipsis;

        `;

        nodeEl.style.position = 'relative';

        nodeEl.appendChild(tip);

      }

    });

  });
}

```

```
    });  
  });  
}
```

```
function updateVisualFlow(isLive) {  
  const allNodes = document.querySelectorAll('#canvas-layer > .node');  
  
  allNodes.forEach(n => {  
    n.classList.remove('status-green');  
    n.classList.add('status-red');  
  });  
  
  wires.forEach(w => {  
    w.path.classList.remove('active');  
    w.path.classList.add('error');  
  });  
  
  if (!isLive) return;  
  
  let serverNodeEl = null;  
  for (let i = 0; i < allNodes.length; i++) {  
    if (allNodes[i].dataset.type === 'ServerNode') {  
      serverNodeEl = allNodes[i];  
      break;  
    }  
  }  
  
  if (!serverNodeEl) return;  
  
  const reachableNodes = new Set();  
  const reachableWires = new Set();  
  
  reachableNodes.add(serverNodeEl.id);  
  const queue = [serverNodeEl.id];
```

```

while (queue.length > 0) {
    const curr = queue.shift();
    wires.forEach(w => {
        if (w.sourceNode === curr) {
            reachableWires.add(w);
            if (!reachableNodes.has(w.targetNode)) {
                reachableNodes.add(w.targetNode);
                queue.push(w.targetNode);
            }
        }
    });
}

allNodes.forEach(n => {
    if (reachableNodes.has(n.id)) {
        n.classList.remove('status-red');
        n.classList.add('status-green');
    }
});

wires.forEach(w => {
    if (reachableWires.has(w)) {
        w.path.classList.remove('error');
        w.path.classList.add('active');
    }
});
}

setInterval(pollServerStatus, 2000);

// Start
init();

```

=====

=== FILE: .\node_editor\index.html ===

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Node Editor</title>

<link rel="stylesheet" href="styles.css">

<!-- Monaco Editor Core -->

<script>var require = { paths: { 'vs':
'https://cdnjs.cloudflare.com/ajax/libs/monaco-editor/0.45.0/min/vs' } };</script>

<script
src="https://cdnjs.cloudflare.com/ajax/libs/monaco-editor/0.45.0/min/vs/loader.min.js"></script>

<script
src="https://cdnjs.cloudflare.com/ajax/libs/monaco-editor/0.45.0/min/vs/editor/editor.main.nls.js"></script
>

<script
src="https://cdnjs.cloudflare.com/ajax/libs/monaco-editor/0.45.0/min/vs/editor/editor.main.js"></script>

</head>

<body>

<!-- Top Toolbar -->

<div id="toolbar">

<div class="logo"> Node Editor</div>

<div class="controls">

<button class="btn btn-primary" id="btn-deploy">Deploy & Run</button>

<button class="btn btn-danger" id="btn-stop">Stop Server</button>

<button class="btn btn-secondary" id="btn-save">Save JSON</button>

```

        <button class="btn btn-secondary" id="btn-clear">Clear Canvas</button>

    </div>

    <div id="status-indicator">● Offline</div>

</div>

<!-- Main Workspace -->

<div id="workspace">

    <!-- Sidebar: Node Palette -->

    <div id="sidebar">

        <h3>Library</h3>

        <p style="font-size: 0.75rem; color: #aaa; margin-bottom: 15px;">
            Drag to add nodes.<br>
            Double-click node or wire to delete.
        </p>

        <div class="category">Core</div>

        <div class="palette-node" draggable="true" data-type="ServerNode">Server Node</div>
        <div class="palette-node" draggable="true" data-type="HTTPRequestsNode">HTTP Request</div>
        <div class="palette-node" draggable="true" data-type="URLNode">URL Router</div>
        <div class="palette-node" draggable="true" data-type="LogicNode">Logic (Python)</div>
        <div class="palette-node" draggable="true" data-type="ContextNode">Context (Python)</div>

        <div class="category">Views & DB</div>

        <div class="palette-node" draggable="true" data-type="RenderNode">Render Template</div>
        <div class="palette-node" draggable="true" data-type="CSSNode">CSS Style</div>
        <div class="palette-node" draggable="true" data-type="ModelNode">Model (SQL)</div>

        <div class="category">Security & Logging</div>

        <div class="palette-node" draggable="true" data-type="ActionLoggerNode">Action Logger</div>
        <div class="palette-node" draggable="true" data-type="AntiBotNode">Anti-Bot</div>
        <div class="palette-node" draggable="true" data-type="RateLimitNode">Rate Limit</div>
        <div class="palette-node" draggable="true" data-type="CSRFNode">CSRF Token</div>

        <div class="palette-node" draggable="true" data-type="ScreenProtectionNode">Screen
Protection</div>

```



```
</div>
```

```
<!-- Canvas Area -->
```

```
<div id="canvas-container">
```

```
    <!-- SVG for Wires (Noodles) -->
```

```
    <svg id="wire-layer" width="100%" height="100%"></svg>
```

```
    <!-- Nodes Container (Pan/Zoom layer) -->
```

```
    <div id="canvas-layer">
```

```
        <!-- Nodes will be injected here dynamically -->
```

```
        <!-- Default Server Node Example -->
```

```
        <div class="node server-node status-green" id="node-1" style="left: 100px; top: 100px;"
```

```
            data-type="ServerNode">
```

```
            <div class="node-header">
```

```
                Server Node
```

```
                <div class="port out-port" data-node="node-1" data-port="out"></div>
```

```
            </div>
```

```
            <div class="node-body">
```

```
                <label>IP Address</label>
```

```
                <input type="text" value="127.0.0.1" class="node-input ip-input">
```

```
                <label>Port</label>
```

```
                <input type="number" value="8000" class="node-input port-input">
```

```
                <div class="node-actions">
```

```
                    <button class="node-btn">Live/Stop</button>
```

```
                </div>
```

```
            </div>
```

```
        </div>
```

```
        <!-- Example URL Node -->
```

```
        <div class="node url-node status-red" id="node-2" style="left: 450px; top: 100px;" data-type="URLNode">
```

```
            <div class="node-header">
```

```
                <div class="port in-port" data-node="node-2" data-port="in"></div>
```

```

        URL Router

        <div class="port out-port" data-node="node-2" data-port="out"></div>

    </div>

    <div class="node-body">

        <label>Path</label>

        <input type="text" value="/api/data" class="node-input path-input">

    </div>

</div>

</div>

</div>

</div>

<!-- Node Templates for Drag & Drop -->
<div id="node-templates" style="display: none;">

    <!-- ServerNode Template -->

    <div class="node server-node status-red" data-type="ServerNode">

        <div class="node-header">

            Server Node

            <div class="port out-port" data-port="out"></div>

        </div>

        <div class="node-body">

            <label>IP Address</label>

            <input type="text" value="127.0.0.1" class="node-input ip-input">

            <label>Port</label>

            <input type="number" value="8000" class="node-input port-input">

            <div class="node-actions"><button class="node-btn">Live/Stop</button></div>

        </div>

    </div>

</div>

    <!-- HTTPRequest Template -->

    <div class="node http-node status-red" data-type="HTTPRequestsNode">

```

```

<div class="node-header">

    <div class="port in-port" data-port="in"></div>

    HTTP Request

    <div class="port out-port" data-port="out"></div>

</div>

<div class="node-body">

    <p style="font-size: 0.8rem; color: #666; text-align: center;">Parses raw request</p>

</div>

</div>

```

```

<!-- URLNode Template -->

```

```

<div class="node url-node status-red" data-type="URLNode">

    <div class="node-header">

        <div class="port in-port" data-port="in"></div>

        URL Router

        <div class="port out-port" data-port="out"></div>

    </div>

    <div class="node-body">

        <label>Path</label>

        <input type="text" value="/" class="node-input path-input">

    </div>

</div>

```

```

<!-- LogicNode Template (Monaco) -->

```

```

<div class="node logic-node status-red" data-type="LogicNode" style="width: 400px;">

    <div class="node-header">

        <div class="port in-port" data-port="in"></div>

        Logic (Python)

        <div class="port out-port" data-port="out"></div>

    </div>

    <div class="node-body">

        <label>Function Logic (def node_logic(request):)</label>

        <!-- Monaco instance will mount here -->

    </div>

</div>

```

```

        <div class="monaco-container"></div>

    </div>

</div>

<!-- RenderNode Template -->

<div class="node render-node status-red" data-type="RenderNode" style="width: 400px;">

    <div class="node-header">

        <div class="port in-port" data-port="in"></div>

        Render Template

    </div>

    <div class="node-body">

        <label>Template Filename (Optional)</label>

        <input type="text" placeholder="e.g. index.html" class="node-input filename-input">

        <label>HTML Content</label>

        <div class="monaco-container"></div>

    </div>

</div>

<!-- CSSNode Template -->

<div class="node css-node status-red" data-type="CSSNode" style="width: 420px;">

    <div class="node-header">

        CSS Style

    </div>

    <div class="node-body">

        <label>Filename (saved to /static/)</label>

        <input type="text" placeholder="e.g. shop.css" class="node-input css-filename-input">

        <label>CSS Code</label>

        <div class="monaco-container" style="height: 250px;"></div>

    </div>

</div>

<!-- ModelNode Template -->

<div class="node model-node status-red" data-type="ModelNode" style="width: 320px;">

```

```

<div class="node-header">

    <div class="port in-port" data-port="in"></div>

    Model (SQL Database)

    <div class="port out-port" data-port="out"></div>

</div>

<div class="node-body">

    <label>SQL Query</label>

    <input type="text" value="SELECT * FROM users" class="node-input query-input">

    <label>Params Mapping (comma separated)</label>

    <input type="text" placeholder="e.g., id, name" class="node-input params-input">

    <label>Context Key</label>

    <input type="text" value="data" class="node-input context-input">

    <div style="display:flex; align-items:center; gap:5px; margin-top:5px;">

        <input type="checkbox" class="is-write-input"> <label style="margin:0;">Is Write
Query?</label>

    </div>

</div>

</div>

```

```

<!-- ContextNode Template (Monaco) -->

<div class="node context-node status-red" data-type="ContextNode" style="width: 400px;">

    <div class="node-header">

        <div class="port in-port" data-port="in"></div>

        Context (Python)

        <div class="port out-port" data-port="out"></div>

    </div>

    <div class="node-body">

        <label>Update request.context (def node_logic(request):)</label>

        <div class="monaco-container"></div>

    </div>

</div>

```

```

<!-- Security specific Templates -->

<div class="node logger-node status-green" data-type="ActionLoggerNode">

```

```
<div class="node-header bg-dark">
```

```
    <div class="port in-port" data-port="in"></div>
```

```
    Action Logger
```

```
    <div class="port out-port" data-port="out"></div>
```

```
</div>
```

```
<div class="node-body">
```

```
    <p style="font-size: 0.8rem; color: #666; text-align: center;">Logs requests to  
/core/logs</p>
```

```
</div>
```

```
</div>
```

```
<div class="node antibot-node status-green" data-type="AntiBotNode">
```

```
    <div class="node-header bg-dark">
```

```
        <div class="port in-port" data-port="in"></div>
```

```
        Anti-Bot Filter
```

```
        <div class="port out-port" data-port="out"></div>
```

```
    </div>
```

```
    <div class="node-body">
```

```
        <p style="font-size: 0.8rem; color: #666; text-align: center;">Blocks scrapers/crawlers</p>
```

```
    </div>
```

```
</div>
```

```
<div class="node ratelimit-node status-green" data-type="RateLimitNode">
```

```
    <div class="node-header bg-dark">
```

```
        <div class="port in-port" data-port="in"></div>
```

```
        Rate Limiter
```

```
        <div class="port out-port" data-port="out"></div>
```

```
    </div>
```

```
    <div class="node-body">
```

```
        <p style="font-size: 0.8rem; color: #666; text-align: center;">Prevents spam / DDoS</p>
```

```
    </div>
```

```
</div>
```

```
<div class="node csrf-node status-green" data-type="CSRFNode">
```

```
<div class="node-header bg-dark">

  <div class="port in-port" data-port="in"></div>

  CSRF Shield

  <div class="port out-port" data-port="out"></div>

</div>

<div class="node-body">

  <p style="font-size: 0.8rem; color: #666; text-align: center;">Validates POST token</p>

</div>

</div>

<div class="node screen-node status-green" data-type="ScreenProtectionNode">

  <div class="node-header bg-dark">

    <div class="port in-port" data-port="in"></div>

    Screen Protection

    <div class="port out-port" data-port="out"></div>

  </div>

  <div class="node-body">

    <p style="font-size: 0.8rem; color: #666; text-align: center;">Blocks screenshots/copy</p>

  </div>

</div>

</div>

<!-- Right Click Menu (Optional Future Use) -->

<div id="context-menu" class="hidden">

  <div class="menu-item" id="delete-node">Delete Node</div>

</div>

<script src="canvas.js"></script>

</body>

</html>
```

=====

=== FILE: .\node_editor\node_backend.py ===

```
import http.server
import socketserver
import json
import os
import subprocess
import signal
import sys
import socket
```

```
PORT = 8080 # Node Editor port
EDITOR_DIR = os.path.dirname(os.path.abspath(__file__))
FRAMEWORK_DIR = os.path.dirname(EDITOR_DIR)
```

```
import mimetypes
mimetypes.init()
mimetypes.add_type('application/javascript', '.js')
mimetypes.add_type('text/css', '.css')
mimetypes.add_type('text/html', '.html')
```

```
active_process = None
```

```
def _extract_function_name(code: str) -> str:
    """Extracts the first top-level function name from Python code robustly."""
    # Method 1: AST parsing (most reliable)
    import ast
    try:
        tree = ast.parse(code)
        for node in ast.walk(tree):
            if isinstance(node, ast.FunctionDef):
```



```

        return node.name

except SyntaxError:

    pass

# Method 2: Regex fallback

import re

match = re.search(

    r'^def\s+([a-zA-Z_][a-zA-Z0-9_]*)\s*\(',

    code,

    re.MULTILINE

)

if match:

    return match.group(1)

# Method 3: Raise clear error

raise ValueError(

    f"No valid function definition found "

    f"in LogicNode/ContextNode code.\n"

    f"Code preview: {code[:100]}"

)

def is_port_free(port, host='127.0.0.1'):

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:

        try:

            s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

            s.bind((host, port))

            return True

        except OSError:

            return False

def wait_for_server(port, timeout=10, host='127.0.0.1'):

    import time

    start = time.time()

```

```
while time.time() - start < timeout:

    try:

        with socket.create_connection((host, port), timeout=1):

            return True

    except (ConnectionRefusedError, OSError):

        time.sleep(0.3)

return False
```

```
class EditorHandler(http.server.SimpleHTTPRequestHandler):

    def __init__(self, *args, **kwargs):

        super().__init__(*args, directory=EDITOR_DIR, **kwargs)

    def do_GET(self):

        if self.path == '/api/status':

            self.handle_status()

        elif self.path == '/api/load':

            self.handle_load()

        elif self.path == '/api/errors':

            self.handle_errors()

        else:

            super().do_GET()

    def do_POST(self):

        if self.path == '/api/save':

            self.handle_save()

        elif self.path == '/api/deploy':

            self.handle_deploy()

        elif self.path == '/api/stop':

            self.handle_stop()

        else:

            self.send_error(404, "API endpoint not found")

    def handle_status(self):
```

```

global active_process

is_live = False

if active_process and active_process.poll() is None:

    is_live = True


self.send_response(200)

self.send_header('Content-type', 'application/json')

self.end_headers()

self.wfile.write(json.dumps({'status': 'live' if is_live else 'offline'}).encode())


def handle_load(self):

    graph_path = os.path.join(EDITOR_DIR, 'graph.json')

    if os.path.exists(graph_path):

        with open(graph_path, 'r') as f:

            graph_data = json.load(f)

    else:

        graph_data = {"nodes": [], "connections": []}


self.send_response(200)

self.send_header('Content-type', 'application/json')

self.send_header('Access-Control-Allow-Origin', '*')

self.end_headers()

self.wfile.write(json.dumps(graph_data).encode())


def handle_errors(self):

    """Serve live error state – which node types are currently erroring."""

    state_file = os.path.join(FRAMEWORK_DIR, 'core', 'logs', 'error_state.json')

    if os.path.exists(state_file):

        with open(state_file, 'r', encoding='utf-8') as f:

            state = json.load(f)

    else:

        state = {}

    self.send_response(200)

```

```
self.send_header('Content-type', 'application/json')
self.send_header('Access-Control-Allow-Origin', '*')
self.end_headers()
self.wfile.write(json.dumps(state).encode())
```

```
def handle_save(self):
```

```
    content_length = int(self.headers['Content-Length'])
    post_data = self.rfile.read(content_length)
    graph_data = json.loads(post_data.decode('utf-8'))
```

```
    with open(os.path.join(EDITOR_DIR, 'graph.json'), 'w') as f:
        json.dump(graph_data, f, indent=4)
```

```
    self.send_response(200)
    self.send_header('Content-type', 'application/json')
    self.end_headers()
    self.wfile.write(json.dumps({'status': 'success'}).encode())
```

```
def handle_stop(self):
```

```
    global active_process
    if active_process:
        active_process.terminate()
        active_process = None
```

```
    self.send_response(200)
    self.send_header('Content-type', 'application/json')
    self.end_headers()
    self.wfile.write(json.dumps({'status': 'stopped'}).encode())
```

```
def handle_deploy(self):
```

```
    global active_process
    content_length = int(self.headers['Content-Length'])
    post_data = self.rfile.read(content_length)
```

```

graph_data = json.loads(post_data.decode('utf-8'))

# 1. Save JSON

with open(os.path.join(EDITOR_DIR, 'graph.json'), 'w') as f:

    json.dump(graph_data, f, indent=4)

# 2. Compile JSON to main.py

try:

    self.compile_graph(graph_data)

except Exception as e:

    self.send_response(400)

    self.send_header('Content-type', 'application/json')

    self.end_headers()

    self.wfile.write(json.dumps({'status': 'error', 'message': str(e)}).encode())

    return

# 3. Restart Server

if active_process:

    active_process.terminate()

    active_process.wait()

# 3.5 Forcefully kill any stray background processes holding the port

try:

    port = 8000

    for n in graph_data.get('nodes', []):

        if n['type'] == 'ServerNode':

            port = int(n.get('config', {}).get('port', 8000))

            break

            kill_cmd = f"Get-NetTCPConnection -LocalPort {port} -ErrorAction SilentlyContinue |
Select-Object -ExpandProperty OwningProcess | ForEach-Object {{ Stop-Process -Id $_ -Force }}"

            subprocess.run(["powershell", "-Command", kill_cmd], capture_output=True)

except Exception:

    pass

```

```

# Check port is free before starting

port = 8000 # get from graph_data

for n in graph_data.get('nodes', []):

    if n['type'] == 'ServerNode':

        port = int(n.get('config', {}).get('port', 8000))

        break

# Wait for port to be free

# (old process may take time to die)

import time

for _ in range(20): # max 2 seconds

    if is_port_free(port):

        break

    time.sleep(0.1)

main_py_path = os.path.join(FRAMEWORK_DIR, 'main.py')

active_process = subprocess.Popen([sys.executable, main_py_path], cwd=FRAMEWORK_DIR)

# Verify server actually started

if not wait_for_server(port, timeout=8):

    # Server failed to start

    self.send_response(200)

    self.send_header('Content-type', 'application/json')

    self.end_headers()

    self.wfile.write(

        json.dumps({

            'status': 'error',

            'message': f'Server failed to start on port {port}. Check main.py for errors.'

        }).encode()

    )

    return

# Only NOW return success

```

```
self.send_response(200)

self.send_header('Content-type', 'application/json')

self.end_headers()

self.wfile.write(json.dumps({'status': 'success'}).encode())
```

```
def compile_graph(self, graph_data):

    nodes = {n['id']: n for n in graph_data['nodes']}

    connections = graph_data['connections']

    # Find starting node (ServerNode)

    server_node_id = next((n['id'] for n in nodes.values() if n['type'] == 'ServerNode'), None)

    if not server_node_id:

        raise ValueError("No Server Node found in graph!")

    # Find mapping of source -> list of targets

    outgoing_map = {}

    for c in connections:

        source_id = c['source']

        target_id = c['target']

        if source_id not in outgoing_map:

            outgoing_map[source_id] = []

        outgoing_map[source_id].append(target_id)

    # BFS to find all reachable nodes

    reachable_ids = []

    queue = [server_node_id]

    while queue:

        curr = queue.pop(0)

        if curr not in reachable_ids:

            reachable_ids.append(curr)

            for tgt in outgoing_map.get(curr, []):

                queue.append(tgt)
```

```

# Gather Imports based on reachable node types

types_used = set(nodes[nid]['type'] for nid in reachable_ids)

needs_router = any(len(targets) > 1 for src, targets in outgoing_map.items() if src in
reachable_ids)

imports = [

    "import socketserver",

    "from nodes.server_node import ServerNode",

    "from nodes.http_requests_node import HTTPRequestsNode"

]

if 'URLNode' in types_used: imports.append("from nodes.url_node import URLNode")
if 'RenderNode' in types_used: imports.append("from nodes.template_node import RenderNode")
if 'LogicNode' in types_used: imports.append("from nodes.logic_node import LogicNode")
if 'ModelNode' in types_used:
    imports.append("from nodes.model_node import ModelNode")
    imports.append("from core.db import Database")
if needs_router:
    imports.append("from nodes.route_node import RouterNode")
if 'ContextNode' in types_used: imports.append("from nodes.context_node import ContextNode")
if 'ActionLoggerNode' in types_used: imports.append("from plugins.logger import ActionLoggerNode")
if 'AntiBotNode' in types_used: imports.append("from plugins.security import AntiBotNode")
if 'RateLimitNode' in types_used: imports.append("from plugins.security import RateLimitNode")
if 'CSRFNode' in types_used: imports.append("from plugins.security import CSRFNode")
    if 'ScreenProtectionNode' in types_used: imports.append("from plugins.security import
ScreenProtectionNode")
if 'CSSNode' in types_used: imports.append("from nodes.css_node import CSSNode")

imports.append("from nodes.response import Response")

code_lines = [

    "# AUTO-GENERATED BY NODE EDITOR COMPILER",

    "\n".join(imports),

    "\n# Initialize Database if needed"

```



```
]
```

```
if 'ModelNode' in types_used:
```

```
    code_lines.append("db = Database('node.db')")
```

```
    code_lines.append("db.connect()")
```

```
code_lines.append("\n# Nodes Instantiation")
```

```
# Initialize variables
```

```
chain_vars = []
```

```
logic_counter = 1
```

```
for nid in reachable_ids:
```

```
    node = nodes[nid]
```

```
    ntype = node['type']
```

```
    config = node.get('config', {})
```

```
    var_name = f"{ntype.lower()}_{nid.replace('-', '_)}"
```

```
    chain_vars.append(var_name)
```

```
    if ntype == 'ServerNode':
```

```
        code_lines.append(f"{var_name} = ServerNode(host='{config.get('ip', '127.0.0.1')}',  
port={config.get('port', 8000)})")
```

```
    elif ntype == 'URLNode':
```

```
        code_lines.append(f"{var_name} = URLNode('{config.get('path', '/')}')")
```

```
    elif ntype == 'RenderNode':
```

```
        html_code = config.get('html_code', '<h1>Hello World!</h1>')
```

```
        custom_filename = config.get('filename', '').strip()
```

```
        auto_filename = custom_filename if custom_filename else f"auto_template_{nid}.html"
```

```
os.makedirs(os.path.join(FRAMEWORK_DIR, 'templates'), exist_ok=True)
```

```
auto_filepath = os.path.join(FRAMEWORK_DIR, 'templates', auto_filename)
```

```
with open(auto_filepath, 'w', encoding='utf-8') as tf:
```

```
    tf.write(html_code)
```

```

        code_lines.append(f"{var_name} = RenderNode('{auto_filename}'))"

    elif ntype == 'ModelNode':

        code_lines.append(f"{var_name} = ModelNode(db, query='{config.get('query', '')}',
params_mapping=[x.strip() for x in '{config.get('paramsMap', '')}'.split(',')],
context_key='{config.get('contextKey', 'data')}', is_write={config.get('isWrite', False)})")

    elif ntype == 'LogicNode':

        logic_counter += 1

        func_code = config.get('code', 'def process_logic(request):\n    return {}')

        func_def_name = _extract_function_name(func_code)

        code_lines.append(f"\n{func_code}\n")

        code_lines.append(f"{var_name} = LogicNode({func_def_name})")

    elif ntype == 'ContextNode':

        logic_counter += 1

        func_code = config.get('code', 'def node_logic(request):\n    return {}')

        func_def_name = _extract_function_name(func_code)

        code_lines.append(f"\n{func_code}\n")

        code_lines.append(f"{var_name} = ContextNode({func_def_name})")

    elif ntype == 'HTTPRequestNode':

        code_lines.append(f"{var_name} = HTTPRequestsNode()")

    elif ntype == 'ActionLoggerNode':

        code_lines.append(f"{var_name} = ActionLoggerNode()")

    elif ntype == 'AntiBotNode':

        code_lines.append(f"{var_name} = AntiBotNode()")

    elif ntype == 'RateLimitNode':

        code_lines.append(f"{var_name} = RateLimitNode()")

    elif ntype == 'CSRFNode':

        code_lines.append(f"{var_name} = CSRFNode()")

    elif ntype == 'ScreenProtectionNode':

        code_lines.append(f"{var_name} = ScreenProtectionNode()")

    elif ntype == 'CSSNode':

        css_filename = config.get('css_filename', 'style.css')

        css_code = config.get('css_code', '').replace("'", "\'").replace('\n', '\\n')

        code_lines.append(f"{var_name} = CSSNode('{css_filename}', '{css_code}'))")

        code_lines.append(f"{var_name}.apply() # Writes CSS to /static/{css_filename}")

```

```

else:

    code_lines.append(f"{var_name} = BaseNode()") # Fallback

# Connect Chain

code_lines.append("\n# Node Connection Chain")

router_counter = 1

processed = set()

for nid in reachable_ids:

    if nid in processed:

        continue

    targets = outgoing_map.get(nid, [])

    if not targets:

        continue

    src_var = f"{nodes[nid]['type'].lower()}_{nid.replace('-', '_)}"

    if len(targets) == 1:

        tgt_var = f"{nodes[targets[0]]['type'].lower()}_{targets[0].replace('-', '_)}"

        code_lines.append(f"{src_var}.connect({tgt_var})")

    else:

        route_list = [

            f"{nodes[t]['type'].lower()}_{t.replace('-', '_)}"

            for t in targets

        ]

        router_var = f"router_node_auto_{router_counter}"

        router_counter += 1

        code_lines.append(f"{router_var} = RouterNode(['', '.join(route_list)])")

        code_lines.append(f"{src_var}.connect({router_var})")

    processed.add(nid)

```

```

# Start Server

code_lines.append("\n# Start Server")

server_var = chain_vars[0]

code_lines.append(f"print('Node Editor Generated Server starting...')")

code_lines.append(f"print(f'  -> http://{{{server_var}.host}}:{{{server_var}.port}}')")

code_lines.append('from nodes.server_node import FrameworkHandler')

code_lines.append(f"FrameworkHandler.server_node = {server_var}")

code_lines.append(f"FrameworkHandler._middleware_chain = None  # Reset cache on redeploy")

code_lines.append(f"from http.server import HTTPServer")

code_lines.append(f"from socketserver import ThreadingMixIn")

code_lines.append(f"class ThreadedHTTPServer(ThreadingMixIn, HTTPServer): pass")

        code_lines.append(f"with  ThreadedHTTPServer(({server_var}.host,  {server_var}.port),
FrameworkHandler) as httpd:")

        code_lines.append(f"    try:")

        code_lines.append(f"        httpd.serve_forever()")

        code_lines.append(f"    except KeyboardInterrupt:")

        code_lines.append(f"        pass")


# Write main.py

with open(os.path.join(FRAMEWORK_DIR, 'main.py'), 'w') as f:

    f.write("\n".join(code_lines))


if __name__ == '__main__':

    from http.server import HTTPServer

    from socketserver import ThreadingMixIn

    class ThreadedHTTPServer(ThreadingMixIn, HTTPServer): pass

    with ThreadedHTTPServer(("", PORT), EditorHandler) as httpd:

        print(f"Node Editor running at http://localhost:{PORT}")

        httpd.serve_forever()


=====


=== FILE: .\node_editor\styles.css ===

```

```
:root {

  --bg-dark: #1e1e1e;

  --bg-panel: #2d2d2d;

  --border-color: #3e3e42;

  --text-main: #d4d4d4;

  --text-muted: #858585;

  --accent: #007acc;


  /* Status Colors based on user image */

  --status-green: #10b981;

  /* Emerald */

  --status-red: #ef4444;

  /* Red */

  --port-color: #000000;
}


* {

  box-sizing: border-box;

  margin: 0;

  padding: 0;

  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}


body {

  background-color: var(--bg-dark);

  color: var(--text-main);

  overflow: hidden;

  display: flex;

  flex-direction: column;

  height: 100vh;
}
```

```
/* Toolbar */
```

```
#toolbar {  
    height: 50px;  
    background-color: var(--bg-panel);  
    border-bottom: 1px solid var(--border-color);  
    display: flex;  
    align-items: center;  
    justify-content: space-between;  
    padding: 0 20px;  
    z-index: 10;  
}
```

```
.logo {  
    font-weight: bold;  
    font-size: 1.2rem;  
    color: white;  
}
```

```
.controls button {  
    background-color: #3c3c3c;  
    color: white;  
    border: 1px solid #555;  
    padding: 6px 12px;  
    margin-right: 8px;  
    border-radius: 4px;  
    cursor: pointer;  
    font-size: 0.9rem;  
}
```

```
.controls button:hover {  
    background-color: #505050;  
}
```

```
.btn-primary {  
    background-color: var(--accent) !important;  
    border-color: var(--accent) !important;  
}
```

```
.btn-primary:hover {  
    background-color: #005f9e !important;  
}
```

```
.btn-danger {  
    background-color: #b91c1c !important;  
    border-color: #991b1b !important;  
}
```

```
#status-indicator {  
    color: var(--text-muted);  
    font-size: 0.9rem;  
}
```

```
#status-indicator.live {  
    color: var(--status-green);  
}
```

```
/* Workspace */
```

```
#workspace {  
    display: flex;  
    flex: 1;  
    overflow: hidden;  
}
```

```
/* Sidebar */
```

```
#sidebar {  
    width: 250px;
```

```
background-color: var(--bg-panel);

border-right: 1px solid var(--border-color);

padding: 15px;

overflow-y: auto;

z-index: 5;

}
```

```
#sidebar h3 {

    font-size: 1rem;

    margin-bottom: 15px;

    color: white;

}
```

```
.category {

    font-size: 0.8rem;

    text-transform: uppercase;

    color: var(--text-muted);

    margin: 15px 0 5px 0;

    letter-spacing: 0.05em;

}
```

```
.palette-node {

    background-color: #3b3b3b;

    border: 1px solid #555;

    padding: 10px;

    margin-bottom: 8px;

    border-radius: 6px;

    cursor: grab;

    font-size: 0.9rem;

    transition: background 0.2s;

}
```

```
.palette-node:hover {
```



```
background-color: #4a4a4a;

border-color: #666;

}

/* Canvas */

#canvas-container {

    flex: 1;

    position: relative;

    overflow: hidden;

    background-color: #1a1a1a;

    background-image:

        linear-gradient(rgba(255, 255, 255, 0.03) 1px, transparent 1px),

        linear-gradient(90deg, rgba(255, 255, 255, 0.03) 1px, transparent 1px);

    background-size: 20px 20px;

    cursor: grab;

}

#canvas-container:active {

    cursor: grabbing;

}

#wire-layer {

    position: absolute;

    top: 0;

    left: 0;

    z-index: 1;

    pointer-events: none;

}

.wire {

    fill: none;

    stroke: #888;

    stroke-width: 4;
```

```
    transition: stroke 0.3s;

    pointer-events: stroke;

    cursor: pointer;
}

.wire:hover {

    stroke: #b91c1c;
}

.wire.active {

    stroke: var(--accent);
}

.wire.error {

    stroke: var(--status-red);

    stroke-dasharray: 5, 5;

    animation: dash 1s linear infinite;
}

@keyframes dash {

    to {

        stroke-dashoffset: -10;

    }
}

#canvas-layer {

    position: absolute;

    top: 0;

    left: 0;

    width: 100%;

    height: 100%;

    z-index: 2;

    transform-origin: 0 0;
```

```
}
```

```
/* Nodes */
```

```
.node {  
    position: absolute;  
  
    width: 280px;  
  
    background-color: white;  
  
    border: 5px solid var(--border-color);  
  
    border-radius: 4px;  
  
    color: black;  
  
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.5);  
  
    cursor: default;  
  
    user-select: none;  
}
```

```
/* Status Borders (Green/Red) */
```

```
.node.status-green {  
    border-color: var(--status-green);  
  
    box-shadow: 0 0 20px rgba(16, 185, 129, 0.6);  
}
```

```
.node.status-red {  
    border-color: var(--status-red);  
  
    box-shadow: 0 0 20px rgba(239, 68, 68, 0.6);  
}
```

```
/* Runtime Node Error – pulsing red glow (overrides status-green) */
```

```
.node.status-error {  
    border-color: #f43f5e !important;  
  
    box-shadow: 0 0 0 3px #f43f5e, 0 0 24px rgba(244, 63, 94, 0.8) !important;  
  
    animation: node-error-pulse 1.2s ease-in-out infinite;  
}
```

```
@keyframes node-error-pulse {  
  0% {  
    box-shadow: 0 0 0 3px #f43f5e, 0 0 20px rgba(244, 63, 94, 0.7);  
  }  
  
  50% {  
    box-shadow: 0 0 0 5px #f43f5e, 0 0 36px rgba(244, 63, 94, 1.0);  
  }  
  
  100% {  
    box-shadow: 0 0 0 3px #f43f5e, 0 0 20px rgba(244, 63, 94, 0.7);  
  }  
}
```

```
.node-header {  
  background-color: transparent;  
  padding: 10px;  
  font-weight: bold;  
  text-align: center;  
  border-bottom: 2px solid black;  
  cursor: grab;  
  position: relative;  
}
```

```
.node-header:active {  
  cursor: grabbing;  
}
```

```
.node-body {  
  padding: 15px;  
  display: flex;  
  flex-direction: column;
```

```
    gap: 10px;
}
```

```
.node-body label {
    font-size: 0.8rem;
    color: #444;
}
```

```
.node-input {
    padding: 6px;
    border: 1px solid black;
    border-radius: 2px;
    font-family: monospace;
    width: 100%;
}
```

```
.node-actions {
    display: flex;
    justify-content: flex-end;
    margin-top: 5px;
}
```

```
.node-btn {
    background: white;
    border: 1px solid black;
    border-radius: 12px;
    padding: 4px 12px;
    cursor: pointer;
    font-size: 0.8rem;
}
```

```
.node-btn:hover {
    background: #eee;
}
```

```
}
```

```
/* Ports */
```

```
.port {  
    width: 14px;  
    height: 14px;  
    background-color: white;  
    border: 4px solid var(--port-color);  
    border-radius: 50%;  
    position: absolute;  
    top: 50%;  
    transform: translateY(-50%);  
    cursor: crosshair;  
    z-index: 10;  
}
```

```
.port:hover {  
    transform: translateY(-50%) scale(1.2);  
}
```

```
.in-port {  
    left: -9px;  
}
```

```
.out-port {  
    right: -9px;  
}
```

```
/* Monaco Container inside Logic Node */
```

```
.monaco-container {  
    height: 150px;  
    width: 100%;  
    border: 1px solid #ccc;
```

```

    margin-top: 5px;
}

/* — Node Delete Button (X) ————— */

.node-delete-btn {
    position: absolute;
    top: -11px;
    right: -11px;
    width: 22px;
    height: 22px;
    background: #f43f5e;
    color: #fff;
    border: 2px solid #fff;
    border-radius: 50%;
    font-size: 13px;
    font-weight: bold;
    line-height: 18px;
    text-align: center;
    cursor: pointer;
    display: none;
    /* hidden by default */
    align-items: center;
    justify-content: center;
    z-index: 200;
    box-shadow: 0 2px 6px rgba(0, 0, 0, 0.4);
    transition: transform 0.15s, background 0.15s;
    user-select: none;
}

.node:hover .node-delete-btn {
    display: flex;
    /* show on node hover */
}

```

```
.node-delete-btn:hover {  
    background: #be123c;  
    transform: scale(1.2);  
}
```

```
/* — Wire Disconnect Button (-) ————— */
```

```
#wire-disconnect-btn {  
    position: absolute;  
    width: 22px;  
    height: 22px;  
    background: #ef4444;  
    color: #fff;  
    border: 2px solid #fff;  
    border-radius: 50%;  
    font-size: 16px;  
    font-weight: bold;  
    line-height: 18px;  
    text-align: center;  
    cursor: pointer;  
    display: none;  
    align-items: center;  
    justify-content: center;  
    z-index: 300;  
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.5);  
    transform: translate(-50%, -50%);  
    pointer-events: all;  
    transition: transform 0.15s, background 0.15s;  
    user-select: none;  
}
```

```
#wire-disconnect-btn:hover {  
    background: #b91c1c;
```



```
        transform: translate(-50%, -50%) scale(1.25);
    }
}
```

=====

=== FILE: .\plugins\logger.py ===

#plugins/logger.py

from nodes.base_node import BaseNode

import settings

class ActionLoggerNode(BaseNode):

"""

Logs every request using the core WebNodeLogger.

Format: [TIMESTAMP] [INFO] METHOD PATH STATUS DURATION IP USER_AGENT

"""

def __init__(self):

super().__init__()

def process(self, request):

if not settings.LOGGING.get('ENABLED', True):

return super().process(request)

import time

start = time.perf_counter()

Get request info BEFORE processing

client_ip = request.handler.client_address[0]

method = request.method

path = request.path

ua = request.headers.get('User-Agent', 'Unknown')

Pass to next node

```

result = super().process(request)

# Log AFTER processing (has timing)
duration_ms = round((time.perf_counter() - start) * 1000)

from core.logging import get_logger
logger = get_logger()
logger.request(
    method=method,
    path=path,
    duration_ms=duration_ms,
    ip=client_ip,
    user_agent=ua
)

return result

```

=====

=== FILE: .\plugins\security.py ===

"""

plugins/security.py – Gravity-Flow Framework Security Nodes

RateLimitNode : Block IPs exceeding request rate limits.

CSRFNode : Per-session CSRF token validation (real tokens, not hardcoded).

AntiBotNode : Block known bots/scrapers by User-Agent.

ScreenProtectionNode: Inject JS/CSS to block screenshots, copy, right-click.

NOTE: Security nodes work at ANY position in the graph – before OR after HTTPRequestsNode.

"""

```
from nodes.base_node import BaseNode
```

```
import time
```

```

import time

import secrets

# threading is already imported above in the file if following standard lib

import threading

import settings


# -----

# Internal helpers – handle both raw FrameworkHandler and RequestWrapper

# -----


def _get_client_ip(request):

    """Return client IP regardless of whether request is raw handler or RequestWrapper."""

    # RequestWrapper: has .handler.client_address

    if hasattr(request, 'handler') and hasattr(request.handler, 'client_address'):

        return request.handler.client_address[0]

    # Raw FrameworkHandler (http.server): has .client_address directly

    if hasattr(request, 'client_address'):

        return request.client_address[0]

    return '0.0.0.0'


def _get_header(request, name, default=''):

    """Get a header value from either RequestWrapper or raw FrameworkHandler."""

    # RequestWrapper: request.headers.get(name)

    if hasattr(request, 'headers') and request.headers is not None:

        return request.headers.get(name, default) or default

    return default


def _get_method(request):

    """Get HTTP method from either RequestWrapper or raw FrameworkHandler."""

    if hasattr(request, 'method'):

```

```

        return request.method

    if hasattr(request, 'command'):

        return request.command

    return 'GET'


def _get_context(request):

    """Get or create context dict on request (works on raw handler too)."""

    if not hasattr(request, 'context'):

        request.context = {}

    return request.context


def _get_param(request, key, default=None):

    """Get a form param safely – raw handler has no get_param()."""

    if hasattr(request, 'get_param'):

        return request.get_param(key, default)

    return default


def _ensure_request_wrapper(request):

    """

    If request is a raw FrameworkHandler (placed before HTTPRequestsNode),
    convert it to a RequestWrapper – same as HTTPRequestsNode.process() does.

    This allows security nodes to work at ANY position in the graph.

    """

    # Already a RequestWrapper (has .context, .params, etc.)
    if hasattr(request, 'context'):

        return request

    # Raw FrameworkHandler – convert it

    try:

        from nodes.http_requests_node import RequestWrapper

```

```

        return RequestWrapper(request)

except Exception as e:

    print(f"[Security] Could not wrap handler: {e}")

    return request


# -----

# Rate Limit Node

# -----


class RateLimitNode(BaseNode):

    """

    Blocks IPs that exceed request limits.

    Config via settings.SECURITY:

        RATE_LIMIT_MAX    - max requests per window (default: 50)

        RATE_LIMIT_WINDOW - window in seconds          (default: 60)

    """

    def __init__(self):

        super().__init__()

        self._registry = {}    # {ip: [timestamps]}

    def process(self, request):

        request = _ensure_request_wrapper(request) # works before OR after HTTPRequestsNode

        if not settings.SECURITY.get('RATE_LIMIT_ENABLED', True):

            return super().process(request)

        client_ip = _get_client_ip(request)

        now      = time.time()

        window   = settings.SECURITY.get('RATE_LIMIT_WINDOW', 60)

        limit    = settings.SECURITY.get('RATE_LIMIT_MAX', 50)

        history = self._registry.get(client_ip, [])

```

```
history = [t for t in history if t > now - window]
```

```
if len(history) >= limit:
```

```
    print(f" [RateLimit] {client_ip} exceeded {limit} req/{window}s")
```

```
    return (
```

```
        '<!DOCTYPE html><html><body style="font-family:sans-serif;text-align:center;'
```

```
        'padding:60px;background:#0f172a;color:#e2e8f0">'
```

```
        '<h1 style="color:#f87171;font-size:3rem">429</h1>'
```

```
        '<p>Too Many Requests. Please wait.</p></body></html>'
```

```
    )
```

```
history.append(now)
```

```
self._registry[client_ip] = history
```

```
return super().process(request)
```

```
# -----
```

```
# CSRF Node (per-session real tokens)
```

```
# -----
```

```
class CSRFNode(BaseNode):
```

```
    """
```

```
    Per-session CSRF protection (Fixed Memory Leak).
```

```
    """
```

```
    _token_store = {}
```

```
    _lock = threading.RLock()
```

```
    TOKEN_EXPIRY = 3600
```

```
    @classmethod
```

```
    def _cleanup_expired(cls):
```

```
        # Called while _lock is held
```

```
        now = time.time()
```

```

expired = [
    ip for ip, data in cls._token_store.items()
    if now - data['created'] > cls.TOKEN_EXPIRY
]

for ip in expired:
    del cls._token_store[ip]

```

```
@classmethod
```

```

def get_or_create_token(cls, client_ip):
    with cls._lock:
        cls._cleanup_expired()
        now = time.time()
        existing = cls._token_store.get(client_ip)
        if existing and now - existing['created'] < cls.TOKEN_EXPIRY:
            return existing['token']

        token = secrets.token_hex(32)
        cls._token_store[client_ip] = {
            'token': token,
            'created': now
        }
        return token

```

```

def process(self, request):
    request = _ensure_request_wrapper(request)
    if not settings.SECURITY.get('CSRF_ENABLED', True):
        return super().process(request)

    client_ip = _get_client_ip(request)
    method = _get_method(request)

    if method == 'GET':
        token = CSRFNode.get_or_create_token(client_ip)

```

```

_get_context(request)['csrf_token'] = token

return super().process(request)

elif method == 'POST':

    expected = CSRFNode.get_or_create_token(client_ip)

    submitted = _get_param(request, 'csrf_token')

    if not expected or not submitted:

        print(f" [CSRF] Missing token from {client_ip}")

        return (

            '<!DOCTYPE html><html><body style="font-family:sans-serif;text-align:center;'

            'padding:60px;background:#0f172a;color:#e2e8f0">'

            '<h1 style="color:#fb923c;font-size:3rem">403</h1>'

            '<p>CSRF token missing.</p></body></html>'

        )

    # Constant-time comparison

    if not secrets.compare_digest(expected, submitted):

        print(f" [CSRF] Token mismatch from {client_ip}")

        return (

            '<!DOCTYPE html><html><body style="font-family:sans-serif;text-align:center;'

            'padding:60px;background:#0f172a;color:#e2e8f0">'

            '<h1 style="color:#fb923c;font-size:3rem">403</h1>'

            '<p>CSRF validation failed. Please go back and try again.</p></body></html>'

        )

    # Do NOT rotate token to allow AJAX and multiple tabs

    _get_context(request)['csrf_token'] = expected

    return super().process(request)

else:

    return super().process(request)

```



```

# -----
# Anti-Bot Node
# -----

class AntiBotNode(BaseNode):
    """
    Blocks requests from known bots and scrapers based on User-Agent.
    """

    _BOT_KEYWORDS = [
        'curl', 'wget', 'python-requests', 'python-urllib',
        'scrapy', 'bot', 'spider', 'crawler', 'http-kit',
        'java/', 'go-http', 'libwww', 'lwp-',
    ]

    def process(self, request):
        request = _ensure_request_wrapper(request) # works before OR after HTTPRequestsNode

        if not settings.SECURITY.get('ANTI_SCRAPING_ENABLED', True):
            return super().process(request)

        user_agent = _get_header(request, 'User-Agent', '').lower()

        if any(kw in user_agent for kw in self._BOT_KEYWORDS):
            print(f" [AntiBot] Blocked: {user_agent[:60]}")
            return (
                '<!DOCTYPE html><html><body style="font-family:sans-serif;text-align:center;'
                'padding:60px;background:#0f172a;color:#e2e8f0">'
                '<h1 style="color:#fb923c;font-size:3rem">403</h1>'
                '<p>Automated requests are not allowed.</p></body></html>'
            )

        return super().process(request)

```

```
# -----  
# Screen Protection Node  
# -----
```

```
class ScreenProtectionNode(BaseNode):
```

```
    """
```

```
    Injects JS + CSS into the response HTML to:
```

- Disable right-click context menu
- Disable text selection
- Show black overlay on window blur (screen switch)
- Show black overlay on PrintScreen key

```
    """
```

```
    _PROTECTION_SCRIPT = """
```

```
<style>
```

```
    body {
```

```
        user-select: none;
```

```
        -webkit-user-select: none;
```

```
        -moz-user-select: none;
```

```
        -ms-user-select: none;
```

```
    }
```

```
    #__gf_overlay {
```

```
        position: fixed; top: 0; left: 0;
```

```
        width: 100%; height: 100%;
```

```
        background: #000;
```

```
        color: #fff;
```

```
        display: none;
```

```
        align-items: center;
```

```
        justify-content: center;
```

```
        flex-direction: column;
```

```
        z-index: 2147483647;
```

```

    font-family: sans-serif;

}

</style>

<div id="__gf_overlay">

    <h2 style="margin-bottom:8px"> Protected Content</h2>

    <p style="color:#aaa;font-size:0.9rem">Click the window to continue</p>

</div>

<script>

(function() {

    var overlay = document.getElementById('__gf_overlay');

    document.addEventListener('contextmenu', function(e) { e.preventDefault(); });

    document.addEventListener('keyup', function(e) {

        if (e.key === 'PrintScreen') {

            overlay.style.display = 'flex';

            setTimeout(function() { overlay.style.display = 'none'; }, 3000);

        }

    });

    document.addEventListener('keydown', function(e) {

        // Block Ctrl+S, Ctrl+U, Ctrl+P

        if (e.ctrlKey && ['s', 'u', 'p'].includes(e.key.toLowerCase())) {

            e.preventDefault();

        }

    });

    window.addEventListener('blur', function() { overlay.style.display = 'flex'; });

    window.addEventListener('focus', function() { overlay.style.display = 'none'; });

})();

</script>

"""

```

```

def process(self, request):

    request = _ensure_request_wrapper(request) # works before OR after HTTPRequestsNode

    if not settings.SECURITY.get('SCREEN_PROTECTION_ENABLED', True):

        return super().process(request)

```

```
result = super().process(request)
```

```
if isinstance(result, str) and '</body>' in result:
```

```
    return result.replace('</body>', self._PROTECTION_SCRIPT + '</body>')
```

```
return result
```

```
=====
```

```
=== FILE: .\plugins\__init__.py ===
```

```
=====
```

```
=== FILE: .\static\api_logic.py ===
```

```
import json
```

```
from nodes.response import Response
```

```
from static.helpers import db, get_session_id
```

```
def api_cart_add_logic(req):
```

```
    session_id = get_session_id(req)
```

```
    product_id = req.get_param('product_id')
```

```
    quantity = int(req.get_param('quantity') or 1)
```

```
    if not product_id:
```

```
        return json.dumps({'error': 'No product_id'})
```

```
    product_id = int(product_id)
```

```
    rows = db.fetchall("SELECT * FROM products WHERE id=?", (product_id,))
```

```
    if not rows:
```

```
        return json.dumps({'error': 'Product not found'})
```

```
    existing = db.fetchall(
```

```

        "SELECT id, quantity FROM cart WHERE session_id=? AND product_id=?",
        (session_id, product_id)
    )
    if existing:
        new_qty = existing[0]['quantity'] + quantity
        db.execute("UPDATE cart SET quantity=? WHERE id=?", (new_qty, existing[0]['id']))
    else:
        db.execute(
            "INSERT INTO cart (session_id, product_id, quantity) VALUES (?, ?, ?)",
            (session_id, product_id, quantity)
        )
    cart_count = db.fetchall(
        "SELECT COALESCE(SUM(quantity),0) as cnt FROM cart WHERE session_id=?",
        (session_id,)
    )[0]['cnt']
    return Response.json({'cart_count': cart_count, 'status': 'added'})

```

```

def api_cart_count_logic(req):
    session_id = get_session_id(req)
    cnt = db.fetchall(
        "SELECT COALESCE(SUM(quantity),0) as cnt FROM cart WHERE session_id=?",
        (session_id,)
    )[0]['cnt']
    return Response.json({'count': cnt})

```

=====

=== FILE: .\static\cart_logic.py ===

```

import urllib.parse

from nodes.response import Response

```

```
from static.helpers import db, get_session_id, compute_cart_totals
```

```
def cart_logic(req):

    session_id = get_session_id(req)

    cart_items = db.fetchall("""

        SELECT c.id as cart_id, c.quantity, p.name, p.price, p.image_url, p.category

        FROM cart c JOIN products p ON c.product_id = p.id

        WHERE c.session_id = ?

        ORDER BY c.id

    """, (session_id,))

    totals = compute_cart_totals(cart_items)

    flash_success = req.query_params.get('added', '')

    flash_name = req.query_params.get('name', '')

    totals.update({

        'cart_items':    cart_items,

        'cart_count':    totals['item_count'],

        'csrf_token':    req.context.get('csrf_token', ''),

        'flash_success': f"'{flash_name}' added to cart!" if flash_success else '',

        'flash_error':   '',

    })

    return totals
```

```
def cart_add_logic(req):

    session_id = get_session_id(req)

    product_id = req.get_param('product_id')

    quantity    = int(req.get_param('quantity') or 1)

    if not product_id:

        return Response.redirect('/cart')

    product_id = int(product_id)

    # Check if product exists

    rows = db.fetchall("SELECT * FROM products WHERE id=?", (product_id,))

    if not rows:

        return Response.redirect('/cart')
```

```

product = rows[0]

# Check if already in cart (same session)

existing = db.fetchall(

    "SELECT id, quantity FROM cart WHERE session_id=? AND product_id=?",

    (session_id, product_id)

)

if existing:

    new_qty = existing[0]['quantity'] + quantity

    db.execute("UPDATE cart SET quantity=? WHERE id=?", (new_qty, existing[0]['id']))

else:

    db.execute(

        "INSERT INTO cart (session_id, product_id, quantity) VALUES (?, ?, ?)",

        (session_id, product_id, quantity)

    )

# Redirect back with flash message

name_enc = urllib.parse.quote(product['name'])

return Response.redirect(f'/cart?added=1&name={name_enc}')

def cart_add_from_shop_logic(req):

    session_id = get_session_id(req)

    product_id = req.get_param('product_id')

    quantity = int(req.get_param('quantity') or 1)

    if not product_id:

        return Response.redirect('/shop')

    product_id = int(product_id)

    # Check if product exists

    rows = db.fetchall("SELECT * FROM products WHERE id=?", (product_id,))

    if not rows:

        return Response.redirect('/shop')

```

```

# Check if already in cart
existing = db.fetchall(
    "SELECT id, quantity FROM cart WHERE session_id=? AND product_id=?",
    (session_id, product_id)
)

redirect_url = req.get_param('redirect_url') or '/shop'

if existing:
    new_qty = existing[0]['quantity'] + quantity
    db.execute("UPDATE cart SET quantity=? WHERE id=?", (new_qty, existing[0]['id']))
else:
    db.execute(
        "INSERT INTO cart (session_id, product_id, quantity) VALUES (?, ?, ?)",
        (session_id, product_id, quantity)
    )

# Redirect back locally
return Response.redirect(redirect_url)

def cart_remove_logic(req):
    cart_id = req.get_param('cart_id')
    if cart_id:
        db.execute("DELETE FROM cart WHERE id=?", (int(cart_id),))
    return Response.redirect('/cart')

=====

=== FILE: .\static\checkout_logic.py ===

import urllib.parse

from nodes.response import Response

```



```
from static.helpers import db, get_session_id, compute_cart_totals
```

```
def checkout_logic(req):
```

```
    session_id = get_session_id(req)
```

```
    cart_items = db.fetchall("""
```

```
        SELECT c.id as cart_id, c.quantity, p.name, p.price, p.image_url, p.category
```

```
        FROM cart c JOIN products p ON c.product_id = p.id
```

```
        WHERE c.session_id = ?
```

```
        ORDER BY c.id
```

```
    """, (session_id,))
```

```
    if not cart_items:
```

```
        return Response.redirect('/cart')
```

```
    totals = compute_cart_totals(cart_items)
```

```
    totals.update({
```

```
        'cart_items': cart_items,
```

```
        'cart_count': totals['item_count'],
```

```
        'csrf_token': req.context.get('csrf_token', ''),
```

```
        'flash_error': req.query_params.get('error', ''),
```

```
    })
```

```
    return totals
```

```
def checkout_place_logic(req):
```

```
    session_id = get_session_id(req)
```

```
    name = req.get_param('name', '').strip()
```

```
    email = req.get_param('email', '').strip()
```

```
    address = req.get_param('address', '').strip()
```

```
    phone = req.get_param('phone', '').strip()
```

```
    if not name or not email or not address:
```

```
        return Response.redirect('/checkout?error=Please+fill+all+required+fields')
```

```
# Get cart items
```

```

cart_items = db.fetchall("""
    SELECT c.id as cart_id, c.quantity, p.id as product_id, p.name, p.price
    FROM cart c JOIN products p ON c.product_id = p.id
    WHERE c.session_id = ?
""", (session_id,))

if not cart_items:
    return Response.redirect('/cart?error=Cart+is+empty')

totals = compute_cart_totals(cart_items)
total = totals['total_raw']

# Insert order
db.execute(
    "INSERT INTO orders (session_id, name, email, address, phone, total) VALUES (?, ?, ?, ?, ?, ?)",
    (session_id, name, email, address, phone, total)
)

# Get the order id
order_row = db.fetchall("SELECT id FROM orders WHERE session_id=? ORDER BY id DESC LIMIT 1",
(session_id,))
order_id = order_row[0]['id'] if order_row else '??'

# Insert order items
items_data = [(order_id, item['product_id'], item['quantity'], item['price']) for item in cart_items]
db.executemany(
    "INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES (?, ?, ?, ?)",
    items_data
)

# Clear the cart
db.execute("DELETE FROM cart WHERE session_id=?", (session_id,))

# Store order info in context for redirect
name_enc = urllib.parse.quote(name)

```

```

email_enc    = urllib.parse.quote(email)
addr_enc     = urllib.parse.quote(address)
total_enc    = urllib.parse.quote(f"{total:.2f}")
return Response.redirect(
    f'/order/success?order_id={order_id}&name={name_enc}'
    f'&email={email_enc}&address={addr_enc}&total={total_enc}'
)

```

```

def order_success_logic(req):
    session_id = get_session_id(req)
    order_id    = req.query_params.get('order_id', '???')
    name        = req.query_params.get('name', 'Customer')
    email       = req.query_params.get('email', '')
    address     = req.query_params.get('address', '')
    total       = req.query_params.get('total', '0.00')
    return {
        'order_id':        order_id,
        'customer_name':   name,
        'customer_email':  email,
        'customer_address': address,
        'order_total':     total,
        'cart_count':      0,
        'csrf_token':      req.context.get('csrf_token', ''),
    }

```

=====

=== FILE: .\static\frontend_logic.py ===

```

from pyscript import document, window

```

```

def change_text(event):

    # Get the HTML element by ID

    text_box = document.getElementById("demo-text")


    # Change its content just like JS's innerText

    text_box.innerText = "Wow! HTML widget updated from a Static Python File! "


    # Change styling dynamically

    text_box.style.color = "#10b981" # Emerald green

    text_box.style.fontWeight = "bold"


def show_alert(event):

    # Call JS functions directly from Python if needed

    window.alert("Hello from Python Logic in static folder!")


=====


=== FILE: .\static\helpers.py ===


# static/helpers.py

from core.db import Database

from core.sessions import get_session_id # cookie-based session (replaces IP-based)

from core.validators import (

    validate_int,

    validate_str,

    validate_email,

    validate_form,

    safe_int,

    safe_str,

    ValidationError

)

```

```
db = Database()
```

```
# get_session_id is re-exported here so all existing logic files can still do:
```

```
# from static.helpers import get_session_id, db, compute_cart_totals
```

```
# without any changes.
```

```
__all__ = ['db', 'get_session_id', 'compute_cart_totals']
```

```
def compute_cart_totals(cart_items):
```

```
    subtotal_raw = sum(item['price'] * item['quantity'] for item in cart_items)
```

```
    shipping      = 0.0 if subtotal_raw >= 50.0 else 5.99
```

```
    tax           = round(subtotal_raw * 0.08, 2)
```

```
    total         = round(subtotal_raw + shipping + tax, 2)
```

```
    # Add subtotal per line
```

```
    for item in cart_items:
```

```
        item['subtotal'] = f"{item['price'] * item['quantity']:.2f}"
```

```
    return {
```

```
        'subtotal':      f"{subtotal_raw:.2f}",
```

```
        'subtotal_raw':  subtotal_raw,
```

```
        'tax':           f"{tax:.2f}",
```

```
        'total':         f"{total:.2f}",
```

```
        'total_raw':     total,
```

```
        'item_count':    sum(item['quantity'] for item in cart_items),
```

```
    }
```

```
=====
```

```
=== FILE: .\static\shop.css ===
```

```
/* =====
```

```
ShopNova – Premium E-Commerce Stylesheet
```

```
Dark glassmorphism theme with vibrant accents
```

```
===== */
```

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&display=swap');
```

```
:root {  
  
  --bg: #080c14;  
  
  --bg2: #0d1526;  
  
  --surface: rgba(18, 28, 50, 0.85);  
  --surface2: rgba(30, 45, 75, 0.6);  
  --border: rgba(99, 148, 255, 0.15);  
  --border-hover: rgba(99, 148, 255, 0.4);  
  --primary: #4f80ff;  
  --primary-glow: rgba(79, 128, 255, 0.3);  
  --accent: #a855f7;  
  --accent2: #06b6d4;  
  --success: #10b981;  
  --danger: #ef4444;  
  --warning: #f59e0b;  
  --text: #e2e8f0;  
  --text-muted: #64748b;  
  --text-sub: #94a3b8;  
  --radius: 16px;  
  --radius-sm: 10px;  
  --radius-lg: 24px;  
  --shadow: 0 20px 60px -10px rgba(0,0,0,0.7);  
  --shadow-card: 0 8px 30px rgba(0,0,0,0.4);  
  --transition: 0.25s cubic-bezier(0.4, 0, 0.2, 1);  
  
}
```

```
*, *::before, *::after {  
  
  box-sizing: border-box;  
  
  margin: 0;  
  
  padding: 0;  
  
}
```

```
html { scroll-behavior: smooth; }
```

```
body {
```

```
    font-family: 'Inter', system-ui, sans-serif;
```

```
    background-color: var(--bg);
```

```
    color: var(--text);
```

```
    min-height: 100vh;
```

```
    background-image:
```

```
        radial-gradient(ellipse 80% 50% at 20% 0%, rgba(79,128,255,0.08) 0%, transparent 60%),
```

```
        radial-gradient(ellipse 60% 40% at 80% 100%, rgba(168,85,247,0.08) 0%, transparent 60%);
```

```
    line-height: 1.6;
```

```
}
```

```
/* — SCROLLBAR ————— */
```

```
::-webkit-scrollbar { width: 6px; }
```

```
::-webkit-scrollbar-track { background: var(--bg); }
```

```
::-webkit-scrollbar-thumb { background: var(--primary); border-radius: 3px; }
```

```
/* — NAVBAR ————— */
```

```
.navbar {
```

```
    position: sticky;
```

```
    top: 0;
```

```
    z-index: 100;
```

```
    background: rgba(8, 12, 20, 0.85);
```

```
    backdrop-filter: blur(20px);
```

```
    -webkit-backdrop-filter: blur(20px);
```

```
    border-bottom: 1px solid var(--border);
```

```
    padding: 0 2rem;
```

```
}
```

```
.navbar-inner {
```

```
    max-width: 1300px;
```

```
margin: 0 auto;

display: flex;

align-items: center;

justify-content: space-between;

height: 64px;

gap: 2rem;

}
```

```
.logo {

  font-size: 1.5rem;

  font-weight: 800;

  text-decoration: none;

  background: linear-gradient(135deg, var(--primary), var(--accent));

  -webkit-background-clip: text;

  -webkit-text-fill-color: transparent;

  background-clip: text;

  letter-spacing: -0.5px;

  white-space: nowrap;

}
```

```
.logo span { color: var(--accent2); -webkit-text-fill-color: unset; background: none; }
```

```
.nav-links {

  display: flex;

  gap: 0.5rem;

  list-style: none;

}
```

```
.nav-links a {

  display: block;

  padding: 0.5rem 1rem;

  color: var(--text-sub);

  text-decoration: none;
```



```
border-radius: var(--radius-sm);

font-size: 0.9rem;

font-weight: 500;

transition: var(--transition);
}

.nav-links a:hover, .nav-links a.active {

  color: var(--text);

  background: var(--surface2);
}

.cart-btn {

  display: flex;

  align-items: center;

  gap: 0.5rem;

  background: var(--primary);

  color: #fff !important;

  padding: 0.5rem 1.25rem !important;

  border-radius: 30px !important;

  font-weight: 600 !important;

  transition: var(--transition) !important;

  text-decoration: none;

  box-shadow: 0 0 20px var(--primary-glow);
}

.cart-btn:hover {

  background: #6b97ff !important;

  transform: translateY(-1px);

  box-shadow: 0 0 30px var(--primary-glow) !important;
}

.cart-badge {

  background: var(--accent);
```

```
    color: #fff;

    border-radius: 50%;

    width: 20px;

    height: 20px;

    display: flex;

    align-items: center;

    justify-content: center;

    font-size: 0.7rem;

    font-weight: 700;

}

/* — LAYOUT ————— */

.container {

    max-width: 1300px;

    margin: 0 auto;

    padding: 2rem 2rem;

}

/* — HERO ————— */

.hero {

    text-align: center;

    padding: 5rem 1rem 3rem;

    position: relative;

    overflow: hidden;

}

.hero::before {

    content: '';

    position: absolute;

    width: 600px;

    height: 600px;

    background: radial-gradient(circle, rgba(79,128,255,0.1) 0%, transparent 70%);

    top: -150px;
```

```
    left: 50%;  
  
    transform: translateX(-50%);  
  
    pointer-events: none;  
}
```

```
.hero-eyebrow {  
    display: inline-block;  
  
    background: var(--surface2);  
  
    border: 1px solid var(--border);  
  
    color: var(--primary);  
  
    padding: 0.35rem 1rem;  
  
    border-radius: 30px;  
  
    font-size: 0.8rem;  
  
    font-weight: 600;  
  
    letter-spacing: 0.5px;  
  
    text-transform: uppercase;  
  
    margin-bottom: 1.5rem;  
}
```

```
.hero h1 {  
  
    font-size: clamp(2.5rem, 5vw, 4rem);  
  
    font-weight: 800;  
  
    line-height: 1.1;  
  
    letter-spacing: -1.5px;  
  
    margin-bottom: 1rem;  
  
    background: linear-gradient(135deg, #fff 30%, var(--primary) 70%);  
  
    -webkit-background-clip: text;  
  
    -webkit-text-fill-color: transparent;  
  
    background-clip: text;  
}
```

```
.hero p {  
  
    color: var(--text-sub);  
}
```

```
    font-size: 1.1rem;

    max-width: 600px;

    margin: 0 auto 2rem;
}

.hero-actions { display: flex; gap: 1rem; justify-content: center; flex-wrap: wrap; }

/* — CATEGORIES BAR ————— */

.categories {

    display: flex;

    gap: 0.75rem;

    flex-wrap: wrap;

    justify-content: center;

    margin-bottom: 3rem;
}

.cat-pill {

    background: var(--surface);

    border: 1px solid var(--border);

    color: var(--text-sub);

    padding: 0.5rem 1.25rem;

    border-radius: 30px;

    font-size: 0.85rem;

    font-weight: 500;

    cursor: pointer;

    transition: var(--transition);

    text-decoration: none;
}

.cat-pill:hover, .cat-pill.active {

    background: var(--primary);

    border-color: var(--primary);

    color: #fff;
}
```

```
        box-shadow: 0 0 15px var(--primary-glow);
    }

/* — PRODUCT GRID ————— */

.section-title {
    font-size: 1.75rem;
    font-weight: 700;
    margin-bottom: 0.5rem;
    letter-spacing: -0.5px;
}

.section-subtitle {
    color: var(--text-muted);
    margin-bottom: 2rem;
    font-size: 0.95rem;
}

.products-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
    gap: 1.5rem;
    margin-bottom: 4rem;
}

/* — PRODUCT CARD ————— */

.product-card {
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: var(--radius-lg);
    overflow: hidden;
    transition: var(--transition);
    display: flex;
    flex-direction: column;
```

```
    position: relative;

    cursor: pointer;

    text-decoration: none;

    color: inherit;
}
```

```
.product-card:hover {

    transform: translateY(-6px);

    border-color: var(--border-hover);

    box-shadow: 0 20px 50px rgba(79,128,255,0.15);
}
```

```
.product-card-img {

    width: 100%;

    height: 200px;

    background: var(--surface2);

    display: flex;

    align-items: center;

    justify-content: center;

    position: relative;

    overflow: hidden;
}
```

```
.product-card-img img {

    width: 80px;

    height: 80px;

    object-fit: contain;

    filter: drop-shadow(0 10px 30px rgba(79,128,255,0.4));

    transition: transform 0.4s ease;
}
```

```
.product-card:hover .product-card-img img { transform: scale(1.15) rotate(-3deg); }
```

```
.product-badge {  
  position: absolute;  
  top: 12px;  
  left: 12px;  
  background: var(--primary);  
  color: #fff;  
  padding: 0.2rem 0.75rem;  
  border-radius: 30px;  
  font-size: 0.72rem;  
  font-weight: 700;  
  letter-spacing: 0.5px;  
  text-transform: uppercase;  
}
```

```
.product-card-body {  
  padding: 1.25rem;  
  flex: 1;  
  display: flex;  
  flex-direction: column;  
}
```

```
.product-cat {  
  font-size: 0.72rem;  
  color: var(--primary);  
  font-weight: 600;  
  text-transform: uppercase;  
  letter-spacing: 0.5px;  
  margin-bottom: 0.4rem;  
}
```

```
.product-name {  
  font-size: 1rem;  
  font-weight: 700;
```

```
margin-bottom: 0.5rem;

color: var(--text);

line-height: 1.4;

}
```

```
.product-desc {

  font-size: 0.82rem;

  color: var(--text-muted);

  flex: 1;

  line-height: 1.5;

  display: -webkit-box;

  -webkit-line-clamp: 2;

  -webkit-box-orient: vertical;

  overflow: hidden;

  margin-bottom: 1rem;

}
```

```
.product-footer {

  display: flex;

  align-items: center;

  justify-content: space-between;

  gap: 0.75rem;

}
```

```
.product-price {

  font-size: 1.3rem;

  font-weight: 800;

  color: var(--text);

}
```

```
.product-price .currency { font-size: 0.9rem; font-weight: 500; color: var(--text-sub); }
```

```
.btn-add {
```



```
background: var(--primary);

color: #fff;

border: none;

padding: 0.6rem 1.1rem;

border-radius: var(--radius-sm);

font-size: 0.85rem;

font-weight: 600;

cursor: pointer;

transition: var(--transition);

white-space: nowrap;

}

.btn-add:hover {

background: #6b97ff;

box-shadow: 0 0 15px var(--primary-glow);

transform: scale(1.03);

}
```

```
/* — PRODUCT DETAIL ————— */
```

```
.product-detail {

display: grid;

grid-template-columns: 1fr 1fr;

gap: 3rem;

align-items: start;

padding: 3rem 0;

}

.product-detail-img {

background: var(--surface);

border: 1px solid var(--border);

border-radius: var(--radius-lg);

aspect-ratio: 1;

display: flex;
```

```
    align-items: center;

    justify-content: center;

    padding: 3rem;
}

.product-detail-img img {

    width: 100%;

    max-width: 250px;

    filter: drop-shadow(0 20px 40px rgba(79,128,255,0.5));

    animation: float 4s ease-in-out infinite;
}

@keyframes float {

    0%, 100% { transform: translateY(0); }

    50% { transform: translateY(-15px); }
}

.product-detail-info { display: flex; flex-direction: column; gap: 1.25rem; }

.product-detail-cat {

    display: inline-block;

    background: rgba(79,128,255,0.15);

    color: var(--primary);

    border: 1px solid rgba(79,128,255,0.3);

    border-radius: 30px;

    padding: 0.3rem 1rem;

    font-size: 0.8rem;

    font-weight: 600;

    letter-spacing: 0.5px;

    text-transform: uppercase;
}

.product-detail-name {
```

```
    font-size: 2rem;

    font-weight: 800;

    letter-spacing: -1px;

    line-height: 1.2;
}
```

```
.product-detail-price {

    font-size: 2.5rem;

    font-weight: 800;

    color: var(--primary);
}
```

```
.product-detail-desc {

    color: var(--text-sub);

    line-height: 1.7;

    font-size: 0.95rem;
}
```

```
.qty-control {

    display: flex;

    align-items: center;

    gap: 1rem;
}
```

```
.qty-label { font-weight: 600; color: var(--text-sub); font-size: 0.9rem; }
```

```
.qty-btns {

    display: flex;

    align-items: center;

    gap: 0.75rem;

    background: var(--surface);

    border: 1px solid var(--border);

    border-radius: var(--radius-sm);
}
```

```
padding: 0.4rem 0.75rem;
}
```

```
.qty-btns button {
  background: none;
  border: none;
  color: var(--text);
  font-size: 1.2rem;
  cursor: pointer;
  width: 28px;
  height: 28px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 6px;
  transition: var(--transition);
  font-weight: 700;
}
```

```
.qty-btns button:hover { background: var(--surface2); }
```

```
.qty-num {
  font-weight: 700;
  font-size: 1rem;
  min-width: 30px;
  text-align: center;
}
```

```
/* — BUTTONS ————— */
```

```
.btn {
  display: inline-flex;
  align-items: center;
  justify-content: center;
```

```
    gap: 0.5rem;

    padding: 0.85rem 2rem;

    border-radius: var(--radius-sm);

    font-size: 0.95rem;

    font-weight: 600;

    cursor: pointer;

    transition: var(--transition);

    border: none;

    text-decoration: none;

    line-height: 1;

}

.btn-primary {

    background: var(--primary);

    color: #fff;

    box-shadow: 0 0 20px var(--primary-glow);

}

.btn-primary:hover {

    background: #6b97ff;

    box-shadow: 0 0 35px var(--primary-glow);

    transform: translateY(-2px);

}

.btn-outline {

    background: transparent;

    color: var(--text);

    border: 1px solid var(--border);

}

.btn-outline:hover {

    border-color: var(--primary);

    color: var(--primary);

}
```

```
background: rgba(79,128,255,0.08);
}

.btn-danger {
  background: var(--danger);
  color: #fff;
}

.btn-danger:hover { background: #dc2626; transform: translateY(-1px); }

.btn-success {
  background: var(--success);
  color: #fff;
  box-shadow: 0 0 20px rgba(16,185,129,0.3);
}

.btn-success:hover {
  background: #059669;
  box-shadow: 0 0 35px rgba(16,185,129,0.4);
  transform: translateY(-2px);
}

.btn-lg { padding: 1rem 2.5rem; font-size: 1.05rem; }
.btn-full { width: 100%; }

/* — CART ————— */

.cart-layout {
  display: grid;
  grid-template-columns: 1fr 360px;
  gap: 2rem;
  padding-top: 2rem;
}
```

```
.cart-items { display: flex; flex-direction: column; gap: 1rem; }
```

```
.cart-item {  
  display: flex;  
  align-items: center;  
  gap: 1.25rem;  
  background: var(--surface);  
  border: 1px solid var(--border);  
  border-radius: var(--radius);  
  padding: 1.25rem;  
  transition: var(--transition);  
}
```

```
.cart-item:hover { border-color: var(--border-hover); }
```

```
.cart-item-img {  
  width: 70px;  
  height: 70px;  
  background: var(--surface2);  
  border-radius: var(--radius-sm);  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  flex-shrink: 0;  
}
```

```
.cart-item-img img { width: 45px; height: 45px; object-fit: contain; }
```

```
.cart-item-info { flex: 1; }
```

```
.cart-item-name { font-weight: 700; margin-bottom: 0.25rem; }
```

```
.cart-item-cat { font-size: 0.78rem; color: var(--primary); font-weight: 600; }
```

```
.cart-item-price { font-size: 0.9rem; color: var(--text-sub); margin-top: 0.25rem; }
```

```
.cart-item-actions {  
    display: flex;  
    align-items: center;  
    gap: 1rem;  
    flex-shrink: 0;  
}
```

```
.cart-subtotal {  
    font-weight: 700;  
    font-size: 1rem;  
    min-width: 70px;  
    text-align: right;  
}
```

```
.remove-btn {  
    background: rgba(239,68,68,0.1);  
    border: 1px solid rgba(239,68,68,0.3);  
    color: var(--danger);  
    width: 32px;  
    height: 32px;  
    border-radius: 8px;  
    display: flex;  
    align-items: center;  
    justify-content: center;  
    cursor: pointer;  
    transition: var(--transition);  
    font-size: 1rem;  
}
```

```
.remove-btn:hover { background: var(--danger); color: #fff; }
```

```
/* — ORDER SUMMARY CARD ————— */
```

```
.order-summary {
```



```
background: var(--surface);

border: 1px solid var(--border);

border-radius: var(--radius-lg);

padding: 1.75rem;

position: sticky;

top: 80px;

}


.summary-title {

    font-size: 1.1rem;

    font-weight: 700;

    margin-bottom: 1.5rem;

    padding-bottom: 1rem;

    border-bottom: 1px solid var(--border);

}


.summary-row {

    display: flex;

    justify-content: space-between;

    margin-bottom: 0.75rem;

    font-size: 0.9rem;

    color: var(--text-sub);

}


.summary-row.total {

    font-size: 1.2rem;

    font-weight: 700;

    color: var(--text);

    border-top: 1px solid var(--border);

    padding-top: 1rem;

    margin-top: 1rem;

}
```

```
.summary-row.total .amt { color: var(--primary); }
```

```
/* — CHECKOUT FORM ————— */
```

```
.checkout-grid {  
  display: grid;  
  grid-template-columns: 1fr 360px;  
  gap: 2rem;  
  padding-top: 2rem;  
}
```

```
.form-section {  
  background: var(--surface);  
  border: 1px solid var(--border);  
  border-radius: var(--radius-lg);  
  padding: 1.75rem;  
  margin-bottom: 1.5rem;  
}
```

```
.form-section-title {  
  font-size: 1rem;  
  font-weight: 700;  
  margin-bottom: 1.5rem;  
  display: flex;  
  align-items: center;  
  gap: 0.5rem;  
}
```

```
.form-row { display: grid; gap: 1rem; margin-bottom: 1rem; }
```

```
.form-row.cols-2 { grid-template-columns: 1fr 1fr; }
```

```
.form-group { display: flex; flex-direction: column; gap: 0.4rem; }
```

```
.form-label {
```

```
font-size: 0.85rem;

font-weight: 600;

color: var(--text-sub);
}

.form-control {

background: rgba(8, 12, 20, 0.5);

border: 1px solid var(--border);

color: var(--text);

padding: 0.85rem 1rem;

border-radius: var(--radius-sm);

font-size: 0.95rem;

font-family: inherit;

transition: var(--transition);

outline: none;

width: 100%;
}

.form-control:focus {

border-color: var(--primary);

box-shadow: 0 0 0 3px var(--primary-glow);
}

.form-control::placeholder { color: var(--text-muted); }

/* — PAGE HEADERS ————— */

.page-header {

padding: 2.5rem 0 1.5rem;

display: flex;

align-items: center;

justify-content: space-between;

gap: 1rem;

flex-wrap: wrap;
```

```
}
```

```
.page-title {  
    font-size: 1.75rem;  
    font-weight: 800;  
    letter-spacing: -0.5px;  
}
```

```
.breadcrumb {  
    display: flex;  
    gap: 0.5rem;  
    align-items: center;  
    font-size: 0.85rem;  
    color: var(--text-muted);  
}
```

```
.breadcrumb a { color: var(--primary); text-decoration: none; }  
.breadcrumb a:hover { text-decoration: underline; }
```

```
/* — SUCCESS PAGE ————— */
```

```
.success-page {  
    display: flex;  
    align-items: center;  
    justify-content: center;  
    min-height: 70vh;  
    text-align: center;  
}
```

```
.success-card {  
    background: var(--surface);  
    border: 1px solid var(--border);  
    border-radius: var(--radius-lg);  
    padding: 3rem;
```

```
    max-width: 520px;

    width: 100%;

    animation: popIn 0.5s ease-out;
}

@keyframes popIn {
    from { opacity: 0; transform: scale(0.9) translateY(20px); }
    to   { opacity: 1; transform: scale(1) translateY(0); }
}
```

```
.success-icon {
    width: 80px;
    height: 80px;
    background: rgba(16,185,129,0.15);
    border: 2px solid var(--success);
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 2.5rem;
    margin: 0 auto 1.5rem;
    box-shadow: 0 0 30px rgba(16,185,129,0.3);
}
```

```
.success-card h1 {
    font-size: 1.75rem;
    font-weight: 800;
    margin-bottom: 0.75rem;
    background: linear-gradient(135deg, var(--success), #34d399);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    background-clip: text;
}
```

```
.success-card p { color: var(--text-sub); margin-bottom: 0.5rem; }
```

```
.order-id {  
    display: inline-block;  
    background: var(--surface2);  
    border: 1px solid var(--border);  
    border-radius: var(--radius-sm);  
    padding: 0.5rem 1.25rem;  
    font-family: monospace;  
    font-size: 1rem;  
    color: var(--primary);  
    margin: 1rem 0 1.75rem;  
    letter-spacing: 1px;  
}
```

```
/* — FLASH MESSAGE ————— */
```

```
.flash {  
    padding: 1rem 1.25rem;  
    border-radius: var(--radius-sm);  
    margin-bottom: 1rem;  
    font-size: 0.9rem;  
    display: flex;  
    align-items: center;  
    gap: 0.6rem;  
    border: 1px solid;  
    animation: slideDown 0.3s ease-out;  
}
```

```
@keyframes slideDown {  
    from { opacity: 0; transform: translateY(-10px); }  
    to   { opacity: 1; transform: translateY(0); }  
}
```

```
.flash.success {  
    background: rgba(16,185,129,0.1);  
    border-color: rgba(16,185,129,0.3);  
    color: #34d399;  
}
```

```
.flash.error {  
    background: rgba(239,68,68,0.1);  
    border-color: rgba(239,68,68,0.3);  
    color: #fca5a5;  
}
```

```
/* — EMPTY STATE ————— */
```

```
.empty-state {  
    text-align: center;  
    padding: 4rem 2rem;  
}
```

```
.empty-icon {  
    font-size: 4rem;  
    margin-bottom: 1rem;  
    opacity: 0.5;  
}
```

```
.empty-state h2 {  
    font-size: 1.3rem;  
    font-weight: 700;  
    margin-bottom: 0.5rem;  
}
```

```
.empty-state p { color: var(--text-muted); margin-bottom: 1.5rem; }
```

```
/* — FOOTER ————— */
```

```
.footer {  
  
  background: var(--surface);  
  
  border-top: 1px solid var(--border);  
  
  padding: 2rem;  
  
  text-align: center;  
  
  color: var(--text-muted);  
  
  font-size: 0.85rem;  
  
  margin-top: 4rem;  
  
}  
  
.footer a { color: var(--primary); text-decoration: none; }  
.footer a:hover { text-decoration: underline; }
```

```
/* — DIVIDER ————— */
```

```
.divider {  
  
  height: 1px;  
  
  background: var(--border);  
  
  margin: 2rem 0;  
  
}
```

```
/* — STATS BAR ————— */
```

```
.stats-bar {  
  
  display: grid;  
  
  grid-template-columns: repeat(3, 1fr);  
  
  gap: 1rem;  
  
  margin-bottom: 3rem;  
  
}
```

```
.stat-card {  
  
  background: var(--surface);  
  
  border: 1px solid var(--border);  
  
  border-radius: var(--radius);  
  
}
```



```
padding: 1.25rem;

text-align: center;

}
```

```
.stat-num {

    font-size: 1.75rem;

    font-weight: 800;

    background: linear-gradient(135deg, var(--primary), var(--accent));

    -webkit-background-clip: text;

    -webkit-text-fill-color: transparent;

    background-clip: text;

}
```

```
.stat-label { font-size: 0.8rem; color: var(--text-muted); margin-top: 0.25rem; }
```

```
/* — BACK LINK ————— */
```

```
.back-link {

    display: inline-flex;

    align-items: center;

    gap: 0.4rem;

    color: var(--text-sub);

    text-decoration: none;

    font-size: 0.9rem;

    font-weight: 500;

    transition: var(--transition);

}
```

```
.back-link:hover { color: var(--primary); transform: translateX(-3px); }
```

```
/* — PRODUCT FEATURES ————— */
```

```
.feature-list {

    display: flex;

    flex-direction: column;
```

```

        gap: 0.75rem;
    }

    .feature-item {
        display: flex;
        align-items: center;
        gap: 0.75rem;
        font-size: 0.9rem;
        color: var(--text-sub);
    }

    .feature-item::before {
        content: '';
        color: var(--primary);
        font-size: 0.7rem;
    }

    /* — CSRF HIDDEN ————— */
    input[name="csrf_token"] { display: none !important; }

    /* — RESPONSIVE ————— */
    @media (max-width: 900px) {
        .product-detail,
        .cart-layout,
        .checkout-grid { grid-template-columns: 1fr; }
        .stats-bar { grid-template-columns: 1fr 1fr; }
        .form-row.cols-2 { grid-template-columns: 1fr; }
    }

    @media (max-width: 600px) {
        .navbar-inner { padding: 0 1rem; }
        .container { padding: 1rem; }
        .nav-links { display: none; }
    }

```

```

.stats-bar { grid-template-columns: 1fr; }

.hero h1 { font-size: 2rem; }

}

```

```
=====
```

```
=== FILE: .\static\shop_logic.py ===
```

```
from static.helpers import db, get_session_id
```

```

def home_logic(req):

    category    = req.query_params.get('cat', '')

    session_id  = get_session_id(req)

    if category:

        products = db.fetchall(

            "SELECT * FROM products WHERE category=? ORDER BY id",

            (category,)

        )

    else:

        products = db.fetchall("SELECT * FROM products ORDER BY id")

    product_count = db.fetchall("SELECT COUNT(*) as cnt FROM products")[0]['cnt']

    cart_count_row = db.fetchall(

        "SELECT COALESCE(SUM(quantity),0) as cnt FROM cart WHERE session_id=?",

        (session_id,)

    )

    return {

        'products':        products,

        'product_count':   product_count,

        'current_category': category,

        'cart_count':      cart_count_row[0]['cnt'],

        'csrf_token':      req.context.get('csrf_token', ''),

    }

```

```

def product_detail_logic(req):

    pid          = req.get_param('id') or req.url_params.get('id', 0)

    session_id = get_session_id(req)

    rows        = db.fetchall("SELECT * FROM products WHERE id=?", (int(pid),))

    if not rows:

        return {'product_name': 'Not Found', 'product_id': 0, 'product_price': '0',

                'product_description': '', 'product_image': '', 'product_category': '',

                'cart_count': 0, 'csrf_token': ''}

    p = rows[0]

    cart_count_row = db.fetchall(

        "SELECT COALESCE(SUM(quantity),0) as cnt FROM cart WHERE session_id=?",

        (session_id,)

    )

    return {

        'product_id':          p['id'],

        'product_name':        p['name'],

        'product_price':        f"{p['price']:.2f}",

        'product_description':  p['description'],

        'product_image':        p['image_url'],

        'product_category':     p['category'],

        'cart_count':           cart_count_row[0]['cnt'],

        'csrf_token':           req.context.get('csrf_token', ''),

    }

```

=====

=== FILE: .\static_c4test2.css ===

body{color:red}

=====

=== FILE: .\templates\About.html ===

```
<!DOCTYPE html>

<html lang="en">

<head>

    <title>About</title>

</head>

<body>

    {{ result}}

</body>

</html>
```

=====

=== FILE: .\templates\base_shop.html ===

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>{% block title %}ShopNova{% endblock %} – Premium Store</title>

    <meta name="description" content="ShopNova – Discover premium products at unbeatable prices.">

    <link rel="icon" href="data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 100 100'><text y='.9em' font-size='90'></text></svg>">

    <link rel="preconnect" href="https://fonts.googleapis.com">

    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&display=swap" rel="stylesheet">

    <link rel="stylesheet" href="/static/shop.css">

    {% block extra_head %}{% endblock %}

</head>

<body>
```

```
<nav class="navbar">

    <div class="navbar-inner">

        <a href="/" class="logo">Shop<span>Nova</span> </a>

        <ul class="nav-links">

            <li><a href="/" id="nav-home">Home</a></li>

            <li><a href="/shop" id="nav-shop">Shop</a></li>

            <li><a href="/cart" id="nav-cart">Cart</a></li>

        </ul>

        <a href="/cart" class="cart-btn" id="btn-cart-nav">

            Cart <span class="cart-badge" id="cart-count-badge">{{ cart_count }}</span>

        </a>

    </div>

</nav>


<main>

    {% block content %}{% endblock %}

</main>


<footer class="footer">

    <p>© 2025 <strong>ShopNova</strong> · Built with <a href="#">Gravity-Flow Framework</a> · All rights reserved.</p>

</footer>


{% block extra_scripts %}{% endblock %}


</body>

</html>
```

=====

```

{% extends "base_shop.html" %}

{% block title %}Your Cart{% endblock %}


{% block content %}

<div class="container">


    <div class="page-header">

        <h1 class="page-title"> Your Cart</h1>

        <div class="breadcrumb">

            <a href="/">Home</a> > <a href="/shop">Shop</a> > Cart

        </div>

    </div>


    {% if flash_success %}

    <div class="flash success" id="flash-success"> {{ flash_success }}</div>

    {% endif %}

    {% if flash_error %}

    <div class="flash error" id="flash-error"> {{ flash_error }}</div>

    {% endif %}


    {% if cart_items %}

    <div class="cart-layout">


        <!-- Cart Items -->

        <div class="cart-items" id="cart-items-list">

            {% for item in cart_items %}

            <div class="cart-item" id="cart-item-{{ item['cart_id'] }}">

                <div class="cart-item-img">

                    </span>' ">

                </div>

                <div class="cart-item-info">

                    <div class="cart-item-name">{{ item['name'] }}</div>

```

```

        <div class="cart-item-cat">{{ item['category'] }}</div>

        <div class="cart-item-price">${{ item['price'] }} each</div>

    </div>

    <div class="cart-item-actions">

        <div class="cart-subtotal">${{ item['subtotal'] }}</div>

        <div style="font-size:0.85rem;color:var(--text-muted)">×{{ item['quantity'] }}</div>

        <!-- Remove Form (CSRF Protected) -->

        <form method="POST" action="/cart/remove" style="display:inline;" id="remove-form-{{
item['cart_id'] }}">

            <input type="hidden" name="csrf_token" value="{{ csrf_token }}">

            <input type="hidden" name="cart_id" value="{{ item['cart_id'] }}">

            <button type="submit" class="remove-btn" id="btn-remove-{{ item['cart_id'] }}"
title="Remove item"></button>

        </form>

    </div>

</div>

{% endfor %}

</div>

<!-- Order Summary -->

<div class="order-summary" id="order-summary">

    <div class="summary-title"> Order Summary</div>

    <div class="summary-row">

        <span>Subtotal ({{ item_count }} items)</span>

        <span>${{ subtotal }}</span>

    </div>

    <div class="summary-row">

        <span>Shipping</span>

        <span>{% if subtotal_raw >= 50.0 %} Free{% else %}$5.99{% endif %}</span>

    </div>

    <div class="summary-row">

        <span>Tax (8%)</span>

        <span>${{ tax }}</span>

```


</div>

<div class="summary-row total">

Total

\${{ total }}

</div>

Proceed to Checkout →

← Continue Shopping

<div class="divider"></div>

<div style="font-size:0.8rem; color:var(--text-muted); text-align:center;">

Secure checkout · CSRF Protected · Free returns

</div>

</div>

</div>

{% else %}

<div class="empty-state" id="empty-cart">

<div class="empty-icon"></div>

<h2>Your cart is empty</h2>

<p>Add some amazing products to get started!</p>

Start Shopping →

</div>

{% endif %}

</div>

{% endblock %}

=====

=== FILE: .\templates\checkout.html ===

{% extends "base_shop.html" %}

{% block title %}Checkout{% endblock %}

{% block content %}

<div class="container">

<div class="page-header">

<h1 class="page-title"> Checkout</h1>

<div class="breadcrumb">

Home > Cart > Checkout

</div>

</div>

{% if flash_error %}

<div class="flash error" id="flash-error"> {{ flash_error }}</div>

{% endif %}

<form method="POST" action="/checkout/place" id="checkout-form">

<!-- CSRF Token (hidden, auto-injected) -->

<input type="hidden" name="csrf_token" value="{{ csrf_token }}">

<div class="checkout-grid">

<!-- Left: Form Sections -->

<div id="checkout-left">

<!-- Shipping Details -->

<div class="form-section" id="section-shipping">

```
<div class="form-section-title"> Shipping Information</div>
```

```
<div class="form-row cols-2">
```

```
  <div class="form-group">
```

```
    <label class="form-label" for="field-name">Full Name *</label>
```

```
    <input type="text" name="name" id="field-name" class="form-control"
```

```
      placeholder="John Doe" required>
```

```
  </div>
```

```
  <div class="form-group">
```

```
    <label class="form-label" for="field-phone">Phone Number</label>
```

```
    <input type="tel" name="phone" id="field-phone" class="form-control"
```

```
      placeholder="+1 (555) 000-0000">
```

```
  </div>
```

```
</div>
```

```
<div class="form-row">
```

```
  <div class="form-group">
```

```
    <label class="form-label" for="field-email">Email Address *</label>
```

```
    <input type="email" name="email" id="field-email" class="form-control"
```

```
      placeholder="john@example.com" required>
```

```
  </div>
```

```
</div>
```

```
<div class="form-row">
```

```
  <div class="form-group">
```

```
    <label class="form-label" for="field-address">Full Address *</label>
```

```
    <input type="text" name="address" id="field-address" class="form-control"
```

```
      placeholder="123 Main St, City, State, ZIP" required>
```

```
  </div>
```

```
</div>
```

```
</div>
```

```
<!-- Payment (Dummy) -->
```

```

<div class="form-section" id="section-payment">

    <div class="form-section-title"> Payment Details <span
style="font-size:0.75rem;color:var(--text-muted);font-weight:400;">(Demo – not real)</span></div>

    <div class="form-row">

        <div class="form-group">

            <label class="form-label" for="field-card">Card Number</label>

            <input type="text" id="field-card" class="form-control"
                placeholder="4242 4242 4242 4242" maxlength="19">

        </div>

    </div>

    <div class="form-row cols-2">

        <div class="form-group">

            <label class="form-label" for="field-expiry">Expiry Date</label>

            <input type="text" id="field-expiry" class="form-control"
                placeholder="MM/YY" maxlength="5">

        </div>

        <div class="form-group">

            <label class="form-label" for="field-cvv">CVV</label>

            <input type="text" id="field-cvv" class="form-control"
                placeholder="123" maxlength="3">

        </div>

    </div>

    <div style="background:rgba(245,158,11,0.1);border:1px solid
rgba(245,158,11,0.3);border-radius:10px;padding:0.75rem
1rem;font-size:0.82rem;color:#fcd34d;margin-top:0.5rem;">

        This is a <strong>demo store</strong>. No real payment is processed. Use any
test card number.

    </div>

</div>

</div>

```

```
<!-- Right: Summary -->
```

```
<div id="checkout-right">
```

```
  <div class="order-summary" id="checkout-summary">
```

```
    <div class="summary-title"> Order Summary</div>
```

```
    {% for item in cart_items %}
```

```
    <div class="summary-row" id="summary-item-{{ item['cart_id'] }}">
```

```
      <span>{{ item['name'] }} x{{ item['quantity'] }}</span>
```

```
      <span>${{ item['subtotal'] }}</span>
```

```
    </div>
```

```
    {% endfor %}
```

```
  <div class="summary-row" style="margin-top:0.75rem;">
```

```
    <span>Subtotal</span>
```

```
    <span>${{ subtotal }}</span>
```

```
  </div>
```

```
  <div class="summary-row">
```

```
    <span>Shipping</span>
```

```
    <span>{% if subtotal_raw >= 50.0 %}Free{% else %}$5.99{% endif %}</span>
```

```
  </div>
```

```
  <div class="summary-row">
```

```
    <span>Tax (8%)</span>
```

```
    <span>${{ tax }}</span>
```

```
  </div>
```

```
  <div class="summary-row total">
```

```
    <span>Total</span>
```

```
    <span class="amt">${{ total }}</span>
```

```
  </div>
```

```
    <button type="submit" class="btn btn-success btn-full btn-lg" id="btn-place-order" style="margin-top:1.25rem;">
```

```
      Place Order
```

```
  </button>
```

<div

style="margin-top:1rem;text-align:center;font-size:0.78rem;color:var(--text-muted);">

CSRF Protected · SSL Secured · Privacy Guaranteed

</div>

</div>

</div>

</div>

</form>

</div>

{% endblock %}

{% endblock %}

=====

=== FILE: .\templates\demo_frontend.html ===

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<title>Python Frontend Architecture Demo</title>

<style>

body { font-family: sans-serif; padding: 40px; background: #0f172a; color: white; text-align: center; }

.box { background: #1e293b; padding: 30px; border-radius: 12px; max-width: 500px; margin: 0 auto; }

button { background: #3b82f6; color: white; border: none; padding: 12px 24px; border-radius: 6px; cursor: pointer; margin: 10px; font-size: 1rem; }

button:hover { background: #2563eb; }

</style>

</head>

<body>

```

<div class="box">

    <h2>Python-First Framework </h2>

    <p id="demo-text">This text is waiting to be changed by Python...</p>

    <!-- Call functions from static/frontend_logic.py -->

    <button py-click="change_text">Click to Update Text</button>

    <button py-click="show_alert">Show JavaScript Alert via Python</button>

</div>

```

```

<!--

    IMPORT YOUR STATIC PYTHON FILE EXACTLY LIKE JAVASCRIPT

    Framework will Auto-Inject PyScript engine automatically!

-->

<script type="py" src="/static/frontend_logic.py"></script>

```

```

</body>

</html>

```

```

=====

```

```

=== FILE: .\templates\factorial.html ===

```

```

<!DOCTYPE html>

<html>

<head>

<style>

    body { font-family: 'Inter', sans-serif; background: #0f172a; color: white; display:flex;
justify-content:center; align-items:center; height:100vh; margin:0;}

    .card { background: #1e293b; padding: 2rem; border-radius: 12px; box-shadow: 0 10px 15px -3px
rgba(0,0,0,0.5); text-align: center; width: 300px;}

    input { padding: 10px; border-radius: 6px; border: none; outline: none; margin-bottom: 10px; width: 100%;
box-sizing: border-box; font-size: 16px; background: #334155; color: white;}

    button { padding: 10px 20px; background: #3b82f6; color: white; border: none; border-radius: 6px; cursor:
pointer; font-weight: bold; width: 100%; transition: background 0.3s;}

```

```

    button:hover { background: #2563eb; }

    .nav { margin-bottom: 20px; display: flex; gap: 10px; justify-content: center;}

    .nav a { color: #94a3b8; text-decoration: none; font-size: 14px; padding: 5px 10px; border-radius: 4px;
background: #334155;}

    .nav a:hover { background: #475569; color: white; }
</style>

</head>

<body>

    <div class="card">

        <div class="nav">

            <a href="/odd-even">Odd/Even</a>

            <a href="/prime">Prime</a>

            <a href="/factorial">Factorial</a>

        </div>

        <h2>Factorial Calc</h2>

        <form method="GET">

            <input type="number" name="n" placeholder="Enter a number" value="{number}" required>

            <button type="submit">Calculate</button>

        </form>

        <h3 style="margin-top:20px; color:#f59e0b; word-break: break-all;">{result}</h3>

    </div>

</body>

</html>

```

```
=====
```

```
=== FILE: .\templates\live.html ===
```

```
<!DOCTYPE html><html><body><h1>Status:</h1> {{ result | safe }} <h2>Footer</h2></body></html>
```

```
=====
```

```
=== FILE: .\templates\odd_even.html ===
```



```
<!DOCTYPE html>

<html>

<head>

<style>

    body { font-family: 'Inter', sans-serif; background: #0f172a; color: white; display:flex;
justify-content:center; align-items:center; height:100vh; margin:0;}

    .card { background: #1e293b; padding: 2rem; border-radius: 12px; box-shadow: 0 10px 15px -3px
rgba(0,0,0,0.5); text-align: center; width: 300px;}

    input { padding: 10px; border-radius: 6px; border: none; outline: none; margin-bottom: 10px; width: 100%;
box-sizing: border-box; font-size: 16px; background: #334155; color: white;}

    button { padding: 10px 20px; background: #3b82f6; color: white; border: none; border-radius: 6px; cursor:
pointer; font-weight: bold; width: 100%; transition: background 0.3s;}

    button:hover { background: #2563eb; }

    .nav { margin-bottom: 20px; display: flex; gap: 10px; justify-content: center;}

    .nav a { color: #94a3b8; text-decoration: none; font-size: 14px; padding: 5px 10px; border-radius: 4px;
background: #334155;}

    .nav a:hover { background: #475569; color: white; }

</style>

</head>

<body>

    <div class="card">

        <div class="nav">

            <a href="/odd-even">Odd/Even</a>

            <a href="/prime">Prime</a>

            <a href="/factorial">Factorial</a>

        </div>

        <h2>Odd/Even Checker</h2>

        <form method="GET">

            <input type="number" name="n" placeholder="Enter a number" value="{number}" required>

            <button type="submit">Calculate</button>

        </form>

        <h3 style="margin-top:20px; color:#10b981; word-break: break-all;">{result}</h3>

    </div>

</body>

</html>
```

=====

=== FILE: .\templates\order_success.html ===

```
{% extends "base_shop.html" %}

{% block title %}Order Confirmed!{% endblock %}

{% block content %}

<div class="container">

    <div class="success-page" id="success-page">

        <div class="success-card" id="success-card">

            <div class="success-icon" id="success-icon"></div>

            <h1>Order Confirmed!</h1>

            <p>Thank you for your purchase, <strong>{{ customer_name }}</strong>!</p>

            <p style="font-size:0.9rem;">A confirmation will be sent to <strong>{{ customer_email
            }}</strong></p>

            <div class="order-id" id="order-id-display">ORDER #{{ order_id }}</div>

            <div style="background:var(--surface2);border:1px solid
            var(--border);border-radius:var(--radius);padding:1.25rem;margin-bottom:1.5rem;text-align:left;">

                <div style="font-size:0.85rem;color:var(--text-muted);margin-bottom:0.75rem;font-weight:600;">ORDER
                DETAILS</div>

                <div style="display:flex;justify-content:space-between;margin-bottom:0.4rem;font-size:0.9rem;">

                    <span style="color:var(--text-sub);">Shipping to:</span>

                    <span style="font-weight:600;">{{ customer_address }}</span>

                </div>

                <div style="display:flex;justify-content:space-between;font-size:0.9rem;">

                    <span style="color:var(--text-sub);">Order Total:</span>

                    <span style="font-weight:700;color:var(--success);">${{ order_total }}</span>

                </div>

            </div>

        </div>

    <div style="display:flex;gap:0.75rem;flex-direction:column;">
```

```
<a href="/shop" class="btn btn-primary btn-full btn-lg" id="btn-continue-shopping-success">
```

```
Continue Shopping
```

```
</a>
```

```
<a href="/" class="btn btn-outline btn-full" id="btn-go-home-success">< Back to Home</a>
```

```
</div>
```

```
<div style="margin-top:1.5rem;font-size:0.78rem;color:var(--text-muted);">
```

```
Estimated delivery: <strong>3-5 business days</strong>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

```
=====
```

```
=== FILE: .\templates\prime.html ===
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
body { font-family: 'Inter', sans-serif; background: #0f172a; color: white; display:flex; justify-content:center; align-items:center; height:100vh; margin:0;}
```

```
.card { background: #1e293b; padding: 2rem; border-radius: 12px; box-shadow: 0 10px 15px -3px rgba(0,0,0,0.5); text-align: center; width: 300px;}
```

```
input { padding: 10px; border-radius: 6px; border: none; outline: none; margin-bottom: 10px; width: 100%; box-sizing: border-box; font-size: 16px; background: #334155; color: white;}
```

```
button { padding: 10px 20px; background: #3b82f6; color: white; border: none; border-radius: 6px; cursor: pointer; font-weight: bold; width: 100%; transition: background 0.3s;}
```

```
button:hover { background: #2563eb; }
```

```
.nav { margin-bottom: 20px; display: flex; gap: 10px; justify-content: center;}
```

```
.nav a { color: #94a3b8; text-decoration: none; font-size: 14px; padding: 5px 10px; border-radius: 4px; background: #334155;}
```

```
.nav a:hover { background: #475569; color: white; }
```

```

</style>

</head>

<body>

    <div class="card">

        <div class="nav">

            <a href="/odd-even">Odd/Even</a>

            <a href="/prime">Prime</a>

            <a href="/factorial">Factorial</a>

        </div>

        <h2>Prime Checker</h2>

        <form method="GET">

            <input type="number" name="n" placeholder="Enter a number" value="{number}" required>

            <button type="submit">Calculate</button>

        </form>

        <h3 style="margin-top:20px; color:#8b5cf6; word-break: break-all;">{result}</h3>

    </div>

</body>

</html>

```

```

=====

```

```

=== FILE: .\templates\product.html ===

```

```

{% extends "base_shop.html" %}

{% block title %}{{ product_name }}{% endblock %}

{% block content %}

<div class="container">

    <div class="page-header">

        <a href="/shop" class="back-link" id="btn-back-shop">← Back to Shop</a>

        <div class="breadcrumb">

            <a href="/">Home</a> > <a href="/shop">Shop</a> > {{ product_name }}

        </div>

    </div>

{% endblock %}

```

```
</div>
```

```
<div class="product-detail">
```

```
<!-- Left: Image -->
```

```
<div class="product-detail-img">
```

```
</span>')">
```

```
</div>
```

```
<!-- Right: Info -->
```

```
<div class="product-detail-info">
```

```
<span class="product-detail-cat">{{ product_category }}</span>
```

```
<h1 class="product-detail-name">{{ product_name }}</h1>
```

```
<div class="product-detail-price">${{{ product_price }}}</div>
```

```
<p class="product-detail-desc">{{ product_description }}</p>
```

```
<div class="feature-list">
```

```
<div class="feature-item">Free shipping on orders over $50</div>
```

```
<div class="feature-item">30-day hassle-free returns</div>
```

```
<div class="feature-item">1-year manufacturer warranty</div>
```

```
<div class="feature-item">In stock – ships in 24 hours</div>
```

```
</div>
```

```
<!-- Add to Cart Form (CSRF Protected) -->
```

```
<form method="POST" action="/cart/add" id="add-to-cart-form">
```

```
<input type="hidden" name="csrf_token" value="{{ csrf_token }}">
```

```
<input type="hidden" name="product_id" value="{{ product_id }}">
```

```
<div class="qty-control" style="align-items: center; gap: 10px;">
```

```
<label class="qty-label" for="quantity-input">Quantity:</label>
```

```
<input type="number" name="quantity" id="quantity-input" value="1" min="1" max="99"
style="width: 60px; padding: 5px; font-size: 1rem; text-align: center; border: 1px solid #ccc;
border-radius: 4px;">
```

```
</div>
```

```
<div style="display:flex; gap:1rem; margin-top:0.75rem;">
```

```
    <button type="submit" class="btn btn-primary btn-lg" id="btn-add-to-cart" style="flex:1;">
```

```
        Add to Cart
```

```
    </button>
```

```
    <a href="/checkout" class="btn btn-success btn-lg" id="btn-buy-now" style="flex:1;">
```

```
        Buy Now
```

```
    </a>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

```
{% endblock %}
```

```
=====
```

```
=== FILE: .\templates\shop.html ===
```

```
{% extends "base_shop.html" %}
```

```
{% block title %}Shop All Products{% endblock %}
```

```
{% block content %}
```

```
<div class="container">
```

```
<!-- Hero -->
```

```
<section class="hero">
```

```
    <div class="hero-eyebrow"> Free shipping on orders over $50</div>
```

```
<h1>Discover Premium<br>Products</h1>
```

```
<p>Curated collection of cutting-edge electronics, fashion, and lifestyle products delivered to your door.</p>
```

```
<div class="hero-actions">
```

```
<a href="#products" class="btn btn-primary btn-lg" id="btn-shop-now">Shop Now ↓</a>
```

```
<a href="/cart" class="btn btn-outline btn-lg" id="btn-view-cart-hero">View Cart</a>
```

```
</div>
```

```
</section>
```

```
<!-- Stats -->
```

```
<div class="stats-bar">
```

```
<div class="stat-card">
```

```
<div class="stat-num">{{ product_count }}</div>
```

```
<div class="stat-label">Products Available</div>
```

```
</div>
```

```
<div class="stat-card">
```

```
<div class="stat-num">4.9</div>
```

```
<div class="stat-label">Average Rating</div>
```

```
</div>
```

```
<div class="stat-card">
```

```
<div class="stat-num">24h</div>
```

```
<div class="stat-label">Fast Delivery</div>
```

```
</div>
```

```
</div>
```

```
<!-- Category Filters -->
```

```
<div class="categories" id="category-filters">
```

```
<a href="/shop" class="cat-pill {% if not current_category %}active{% endif %}" id="cat-all">All</a>
```

```
<a href="/shop?cat=Electronics" class="cat-pill {% if current_category == 'Electronics' %}active{% endif %}" id="cat-electronics">Electronics</a>
```

```
<a href="/shop?cat=Footwear" class="cat-pill {% if current_category == 'Footwear' %}active{% endif %}" id="cat-footwear">Footwear</a>
```

```
<a href="/shop?cat=Bags" class="cat-pill {% if current_category == 'Bags' %}active{% endif %}" id="cat-bags">Bags</a>
```

```
<a href="/shop?cat=Travel" class="cat-pill {% if current_category == 'Travel' %}active{% endif %}"
```

```

id="cat-travel">Travel</a>

    <a href="/shop?cat=Kitchen" class="cat-pill {% if current_category == 'Kitchen' %}active{% endif
%}" id="cat-kitchen">Kitchen</a>

</div>

<!-- Products Grid -->

<section id="products">

    <h2 class="section-title">Featured Products</h2>

    <p class="section-subtitle">Handpicked for quality, style and value</p>

    {% if products %}

    <div class="products-grid" id="products-grid">

        {% for p in products %}

        <a href="/product/{{ p['id'] }}" class="product-card" id="product-card-{{ p['id'] }}">

            <div class="product-card-img">

                <span class="product-badge">{{ p['category'] }}</span>

                </span>}'"

            </div>

            <div class="product-card-body">

                <div class="product-cat">{{ p['category'] }}</div>

                <div class="product-name">{{ p['name'] }}</div>

                <div class="product-desc">{{ p['description'] }}</div>

                <div class="product-footer">

                    <div class="product-price"><span class="currency">$</span>{{ p['price'] }}</div>

                    <form action="/cart/add_from_shop" method="POST" style="margin:0; padding:0;
display:inline;">

                        <input type="hidden" name="csrf_token" value="{{ csrf_token }}">

                        <input type="hidden" name="product_id" value="{{ p['id'] }}">

                        <input type="hidden" name="quantity" value="1">

                        <button type="submit" class="btn-add" style="border:none; font-family:inherit;
font-size:inherit;">+ Add</button>

                    </form>

                </div>

            </div>

        </div>

    {% endfor %}

</div>

```



```

        </a>

        {% endfor %}

    </div>

    {% else %}

    <div class="empty-state">

        <div class="empty-icon"></div>

        <h2>No products found</h2>

        <p>Try a different category filter</p>

        <a href="/shop" class="btn btn-primary" id="btn-clear-filter">Clear Filter</a>

    </div>

    {% endif %}

</section>

```

```

</div>

```

```

{% endblock %}

```

```

=====

```

```

=== FILE: .\templates\test_pyscript.html ===

```

```

<!DOCTYPE html>

```

```

<html lang="en">

```

```

<head>

```

```

    <meta charset="UTF-8">

```

```

    <title>Python Frontend Test</title>

```

```

</head>

```

```

<body>

```

```

    <h1>Testing Python-First Auto-Injector</h1>

```

```

    <div id="output"></div>

```

```

    <script type="py">

```

```

        from pyscript import document

```

```
document.getElementById("output").innerText = "Hello from PyScript! This page was auto-injected by Gravity-Flow."
```

```
</script>
```

```
</body>
```

```
</html>
```

```
=====
```